



ECOLE
POLYTECHNIQUE
DE BRUXELLES

05. Rendering Equation and Raytracing

INFO-H502

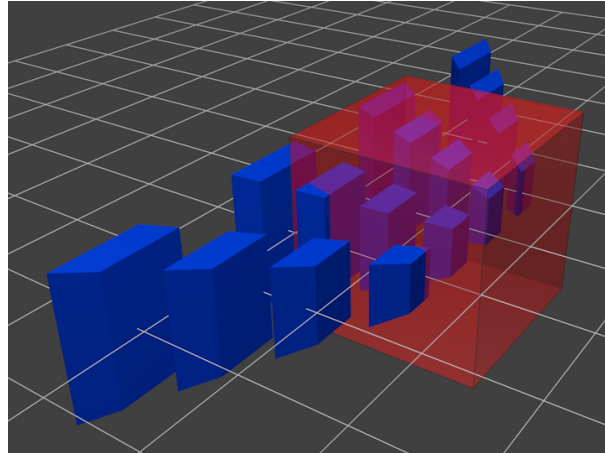
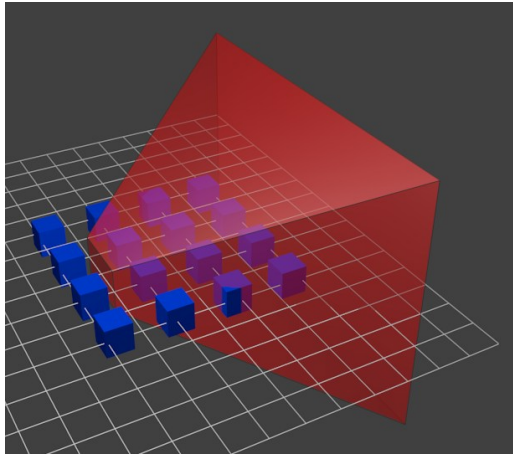
Daniele Bonatto

LISA-VR

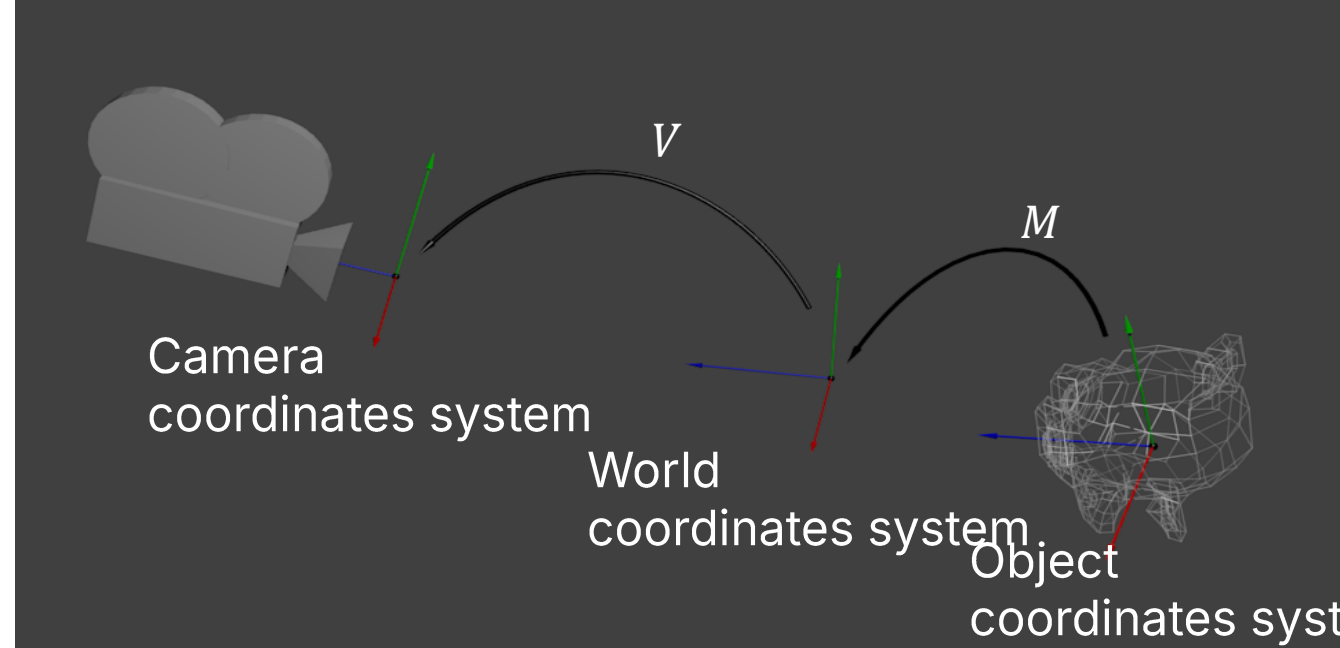
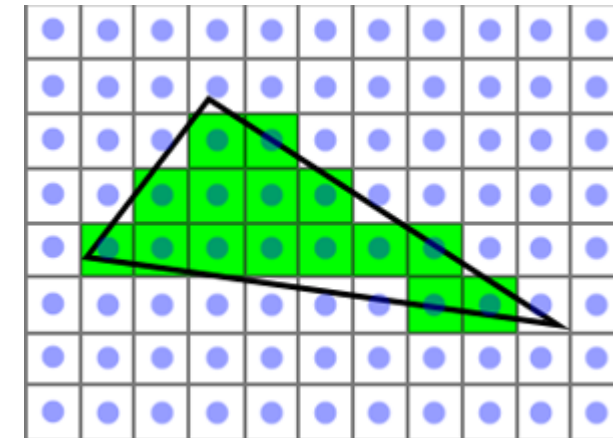
2024

Previously

- We derived the equations for the
 - Model matrix
 - View matrix
 - Projection matrix

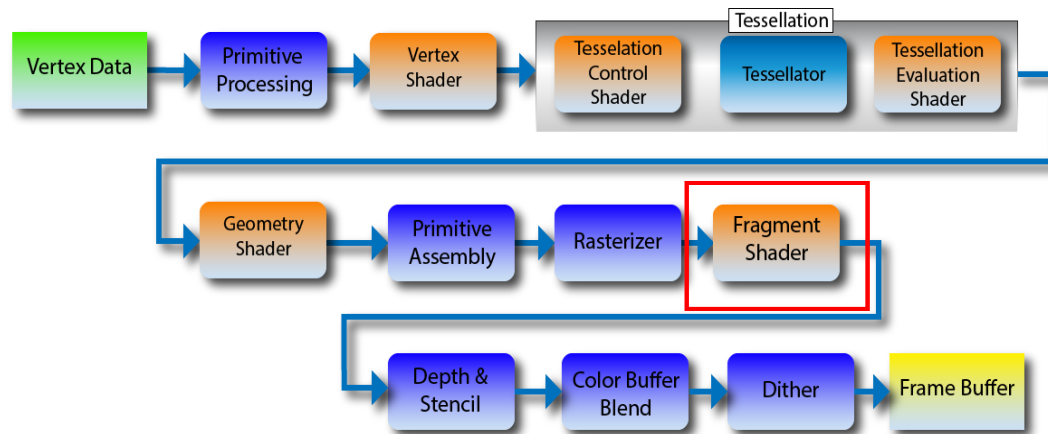


- We saw how rasterization works



This time

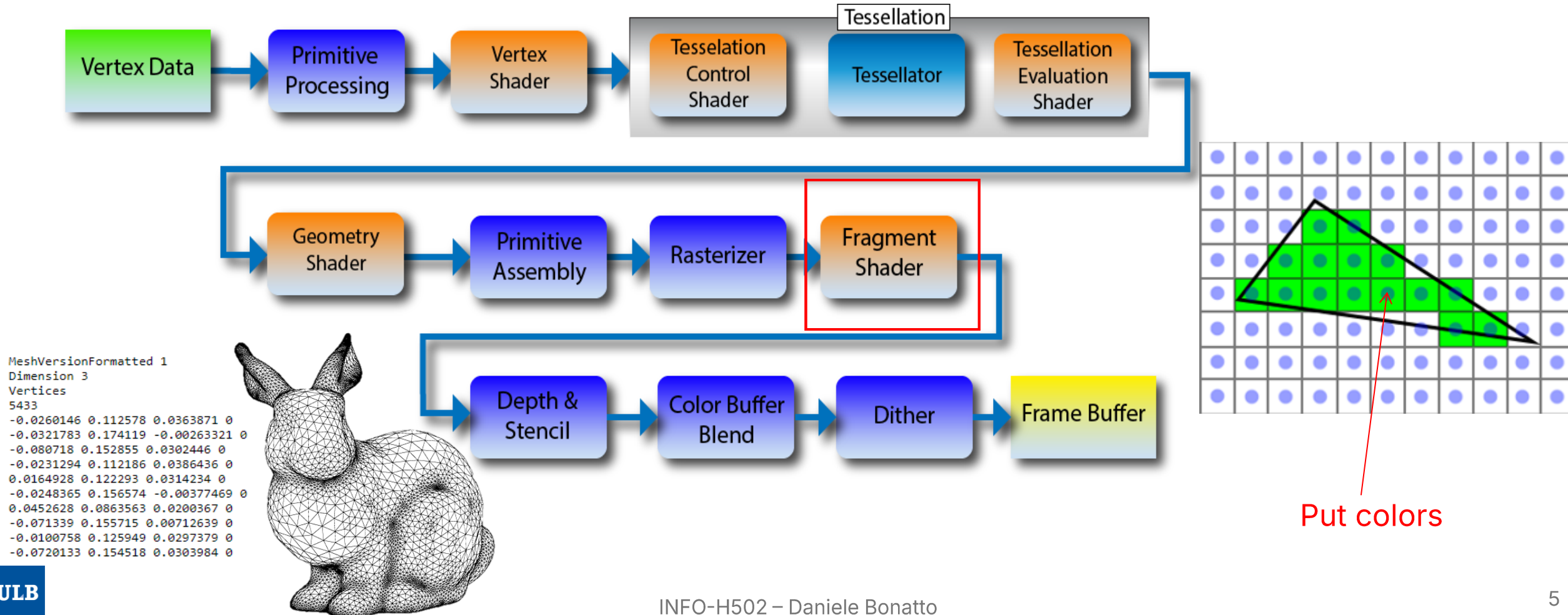
- This time, we focus on the fragment shader and finally put some more interesting colors on the objects !



The Fragment Shader

- We send vertices to the GPU
- Three vertices corresponds to a triangle
- But we still need to put colors on the triangles
- The **Fragment Shader** is responsible to compute the colors inside the triangles.
- It works on *fragments* i.e., small patches of triangles.

OpenGL Pipeline – Fragment Shader

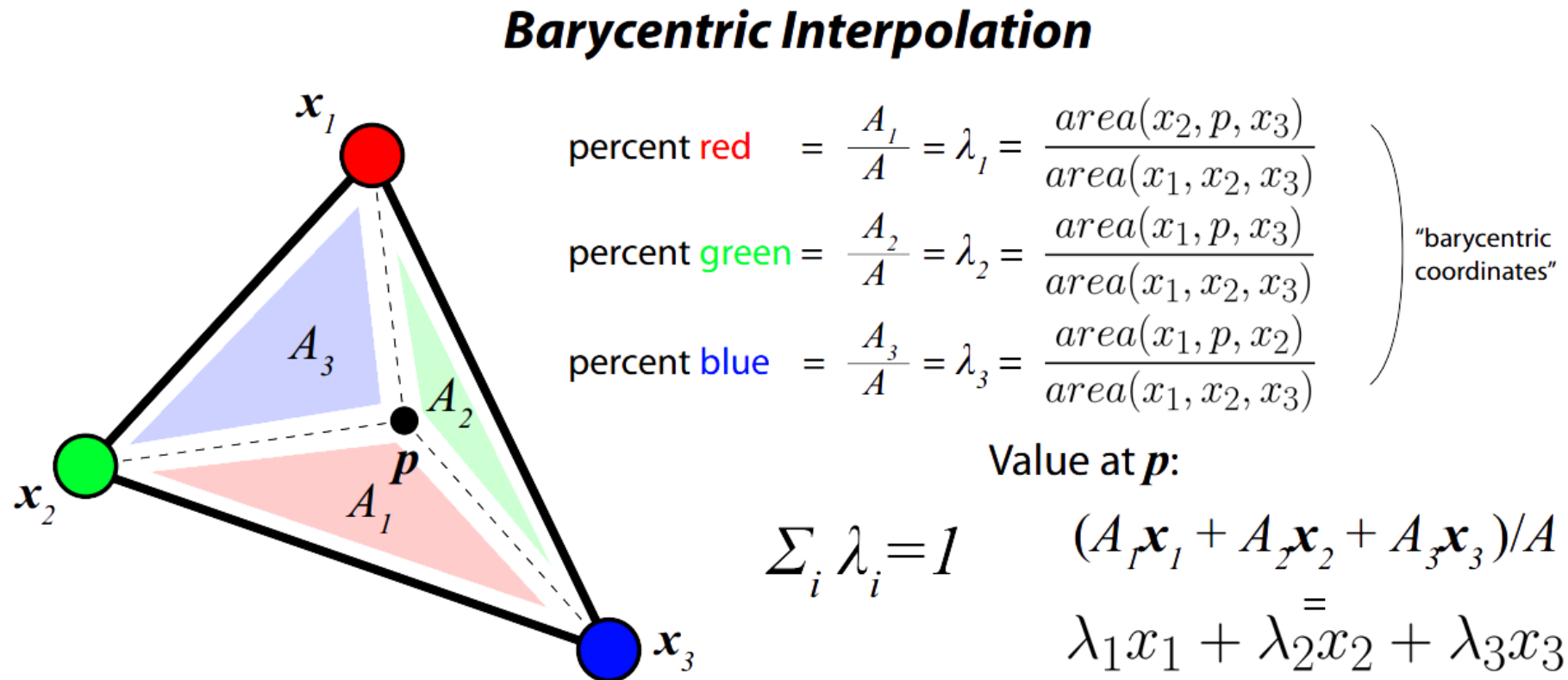


How to pass color data to a triangle ?

- How to pass color data to a triangle?
- We can add color attributes to the data we send to the GPU.
- But this color is only on vertices, we still need to find a way to colorize the fragments

Color interpolation – Vertex to Fragment

- Each vertex can have color as an attribute
- We interpolate between the colors following the Barycentric interpolation



<https://github.com/rarietta/CUDA-Interactive-Rasterizer>

Triangles colorization

- Okay.. This allows to put colors to the triangles..
- By everything feels *flat*
- What is the thing that make our scene less flat ?

Triangles colorization

- Okay.. This allows to put colors to the triangles..
- By everything feels *flat*
- What is the thing that make our scene less flat ?
- Lights !
- We will study how they interact with the triangles to generate beautiful shades.

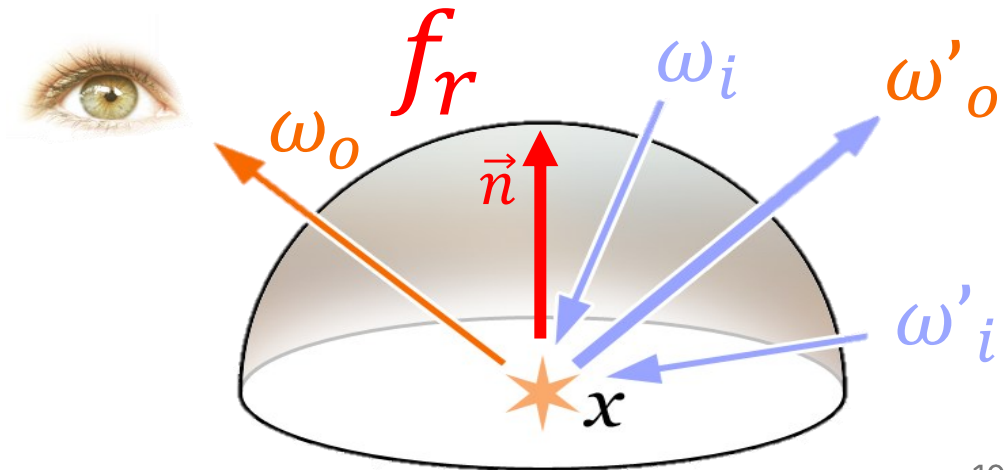
Bidirectional Reflectance Distribution Function (BRDF)

- We need a way to describe how light is reflected of an opaque surface.

- This is modeled by the **bidirectional reflectance distribution function** (BRDF)

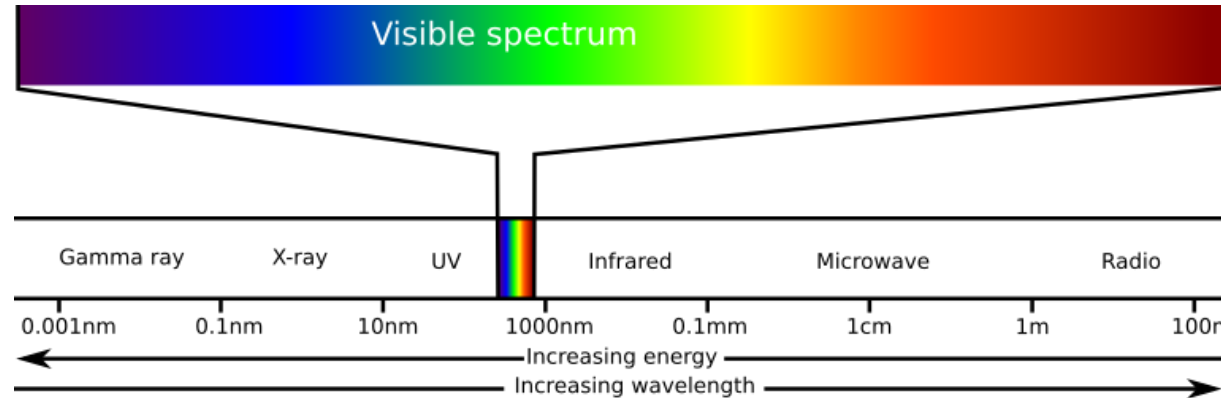
$$f_r(x, \omega_i, \omega_o, \lambda, t)$$

- This function takes the position $\vec{x}$ and incoming light direction ω_i , the outgoing direction ω_o the wavelength λ and the time t .
- It returns the ratio of reflected radiance exiting along ω_o to the irradiance incident on the surface from direction ω_i



BRDF – Wavelength and time

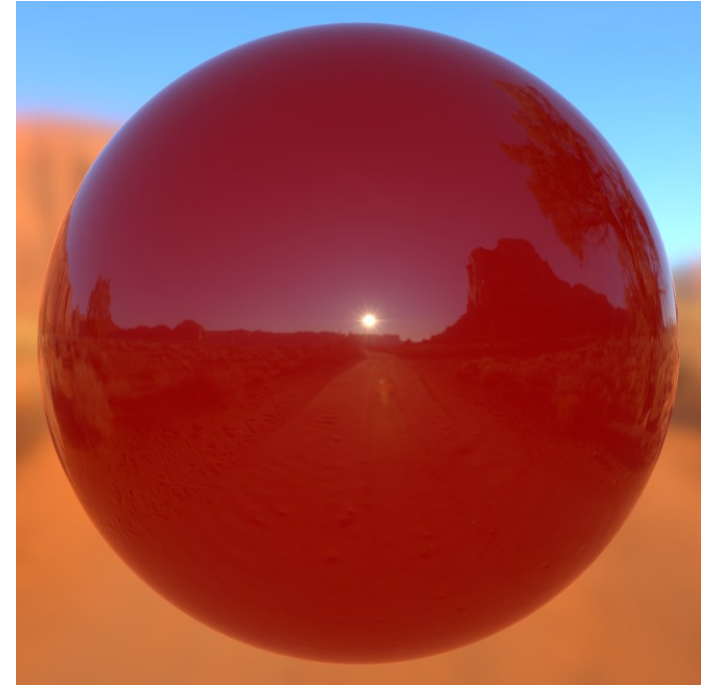
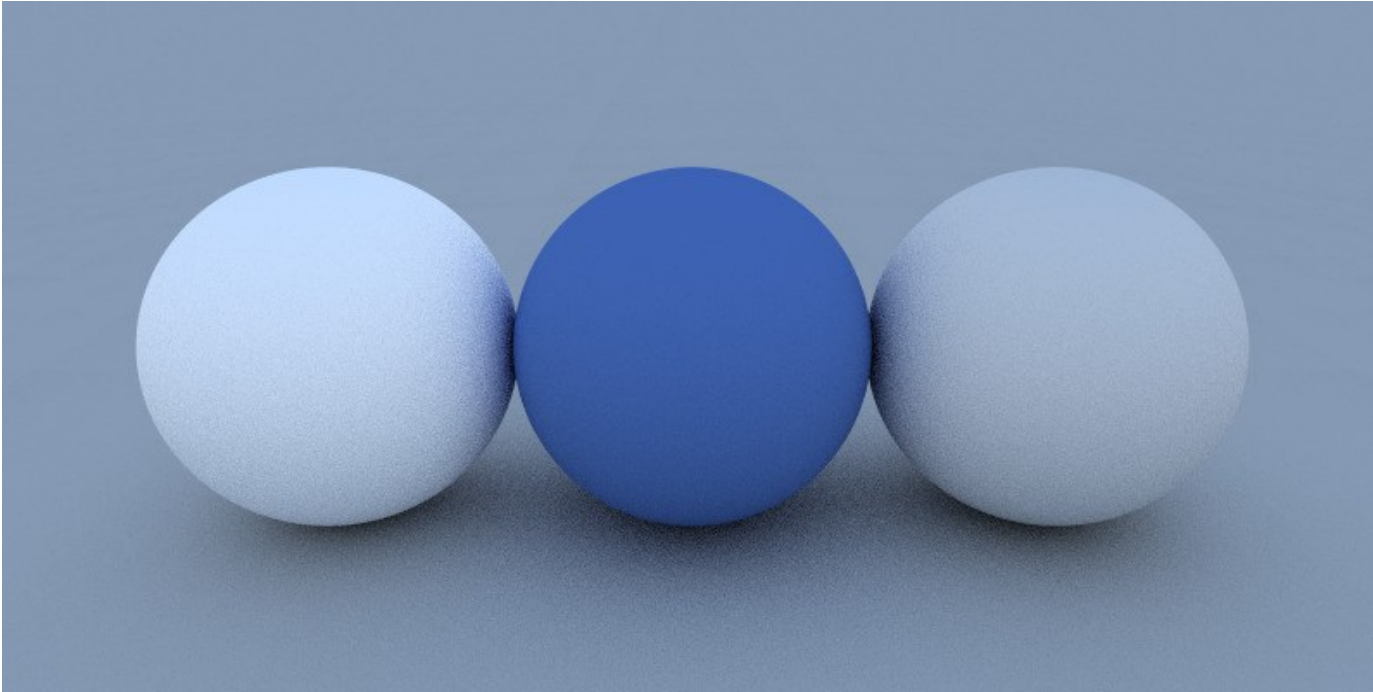
- Materials distribute light in different ways depending on the wavelength λ



- And this can vary in time t
- That's why we have those parameters in the function, but they are often omitted.

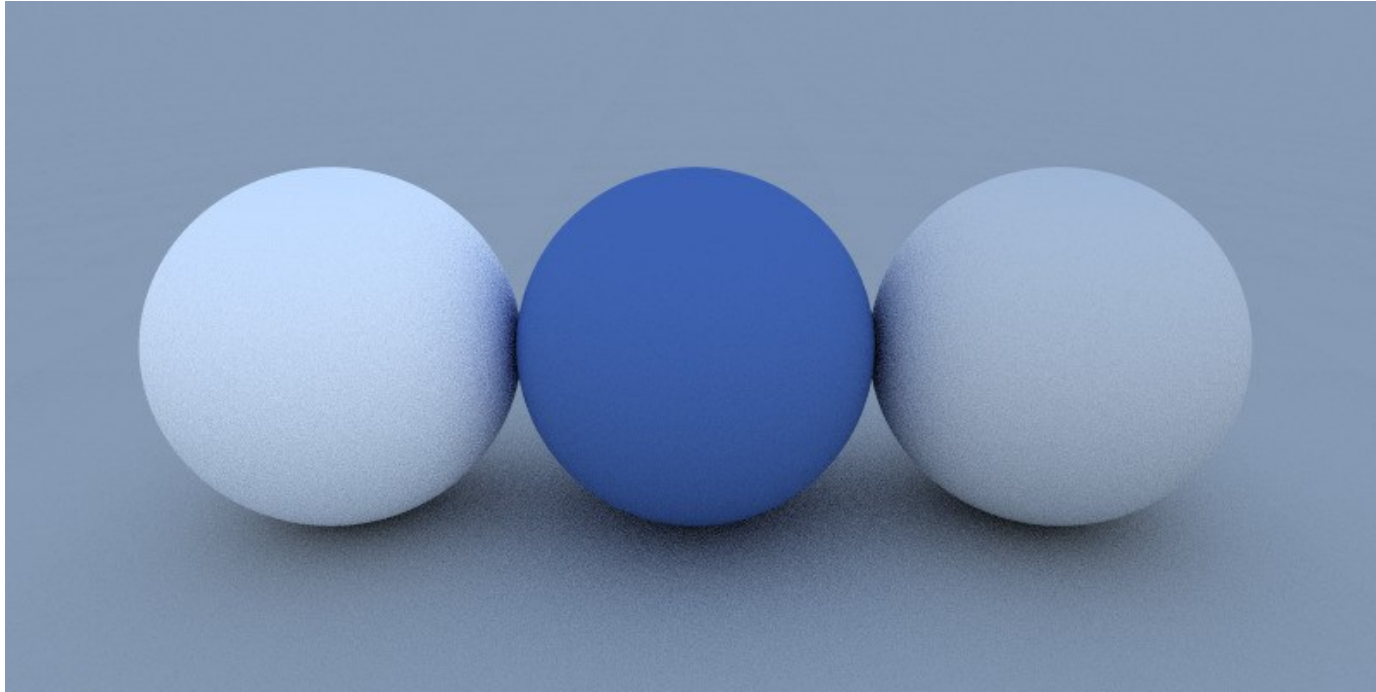
Objects materials

- What is the difference between those materials?

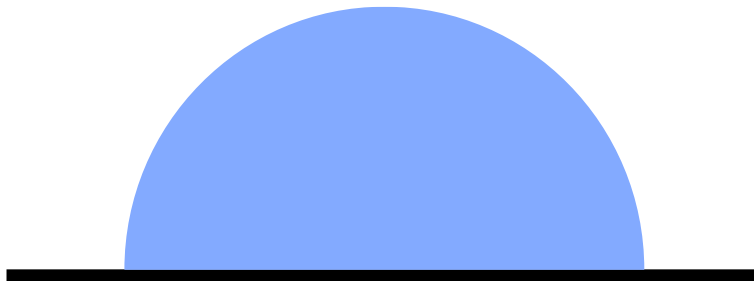
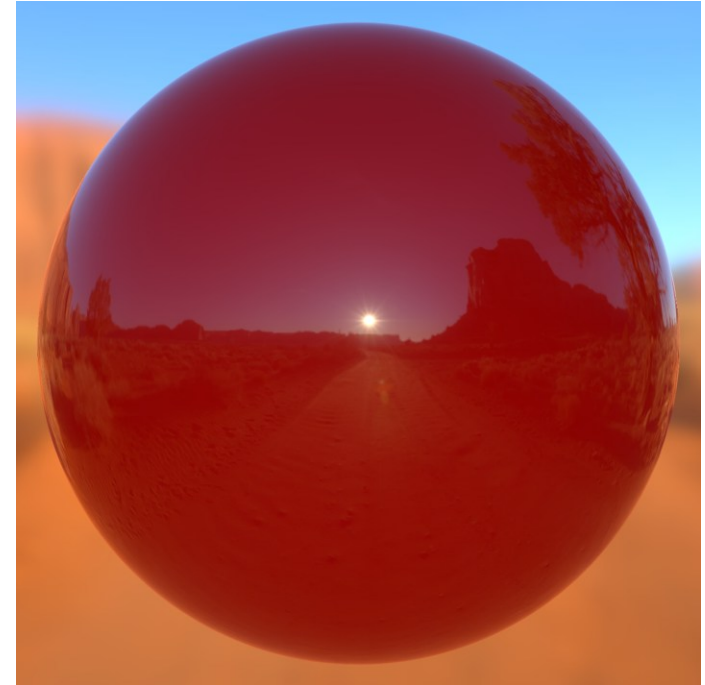


Objects materials

- What is the difference between those materials?

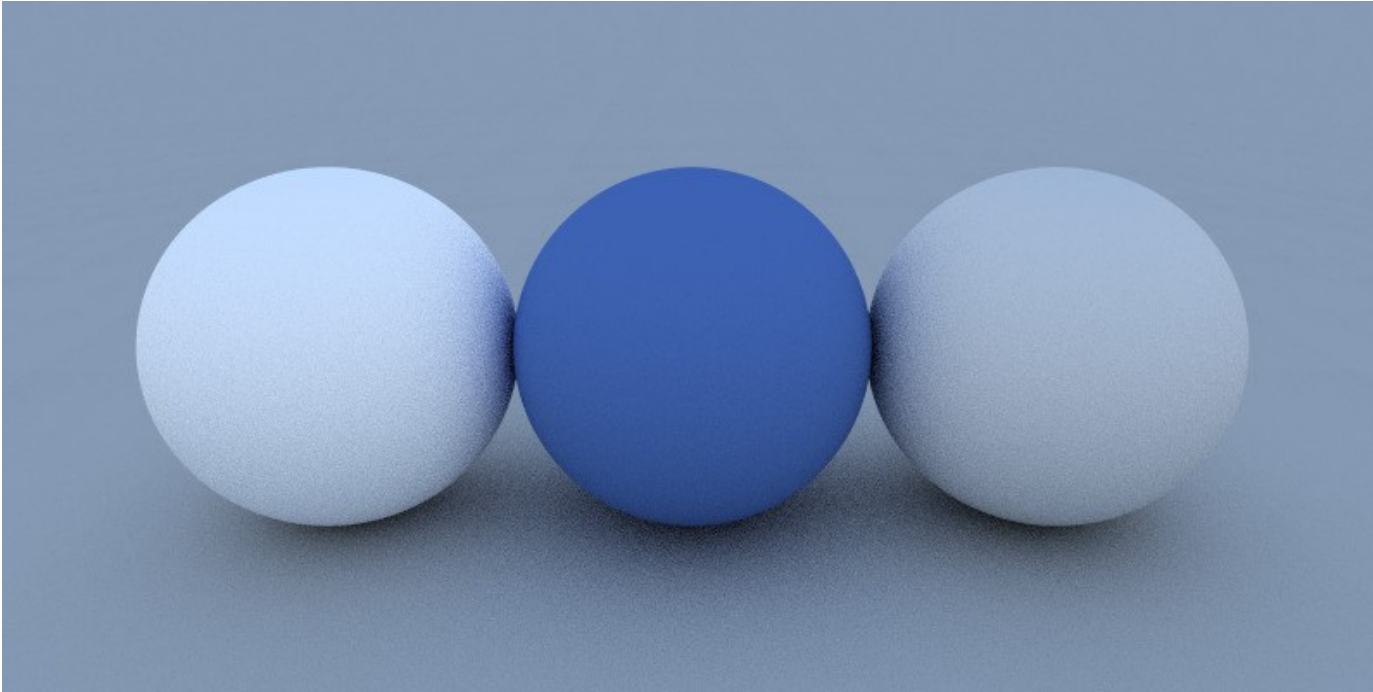


Diffuse material – BRDF symmetrical

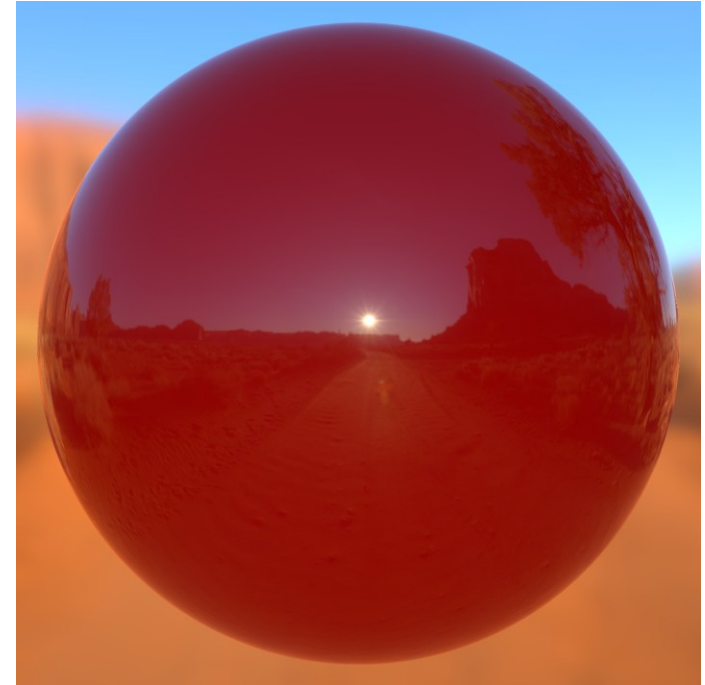
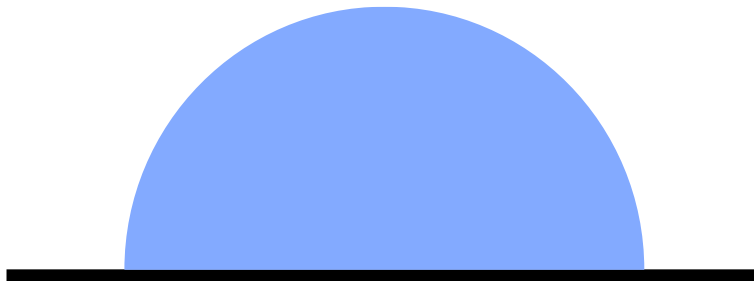


Objects materials

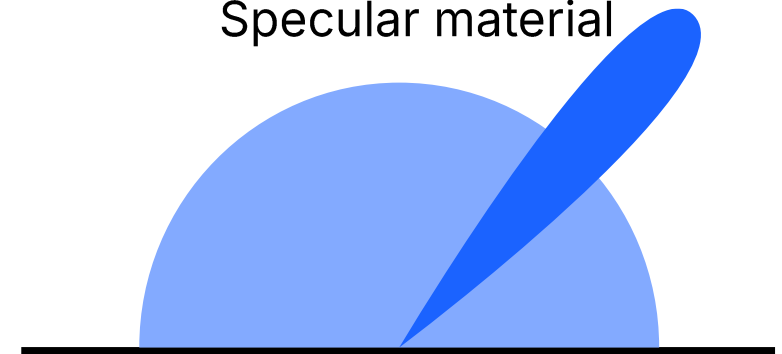
- What is the difference between those materials?



Diffuse material – BRDF symmetrical



Specular material



Objects materials

- What about this material? (plastic effect)

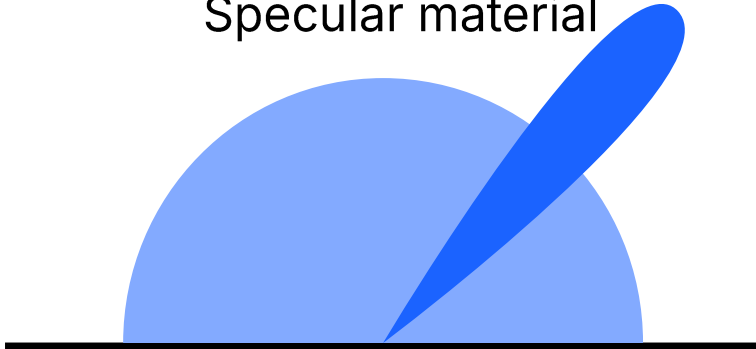


Objects materials

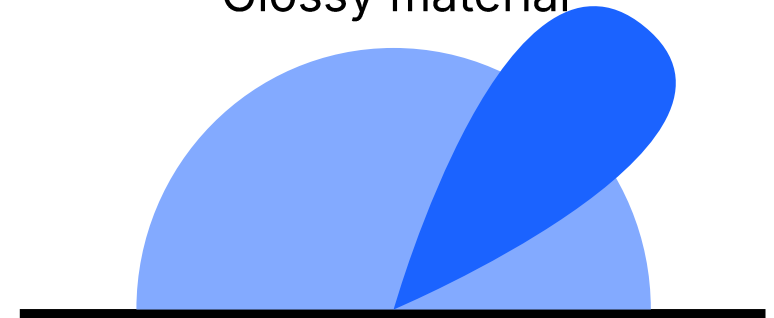
- What about this material? (plastic effect)



Specular material



Glossy material



Why materials are different ?

- Why do some materials are diffuse, glossy or specular ?
 - And often a mix of all of them

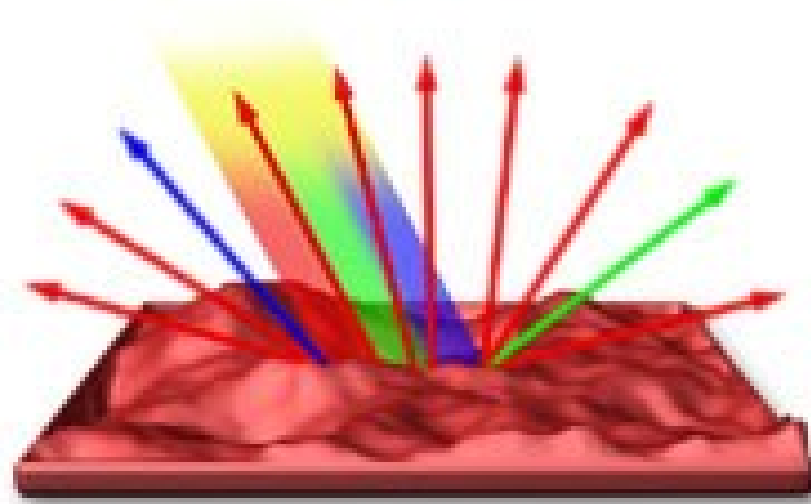
Why materials are different ?

- Why do some materials are diffuse, glossy or specular ?
 - And often a mix of all of them
- It depends on the roughness of the surface

Specular and Diffuse Reflection



**Specular
Reflection**



**Diffuse
Reflection**

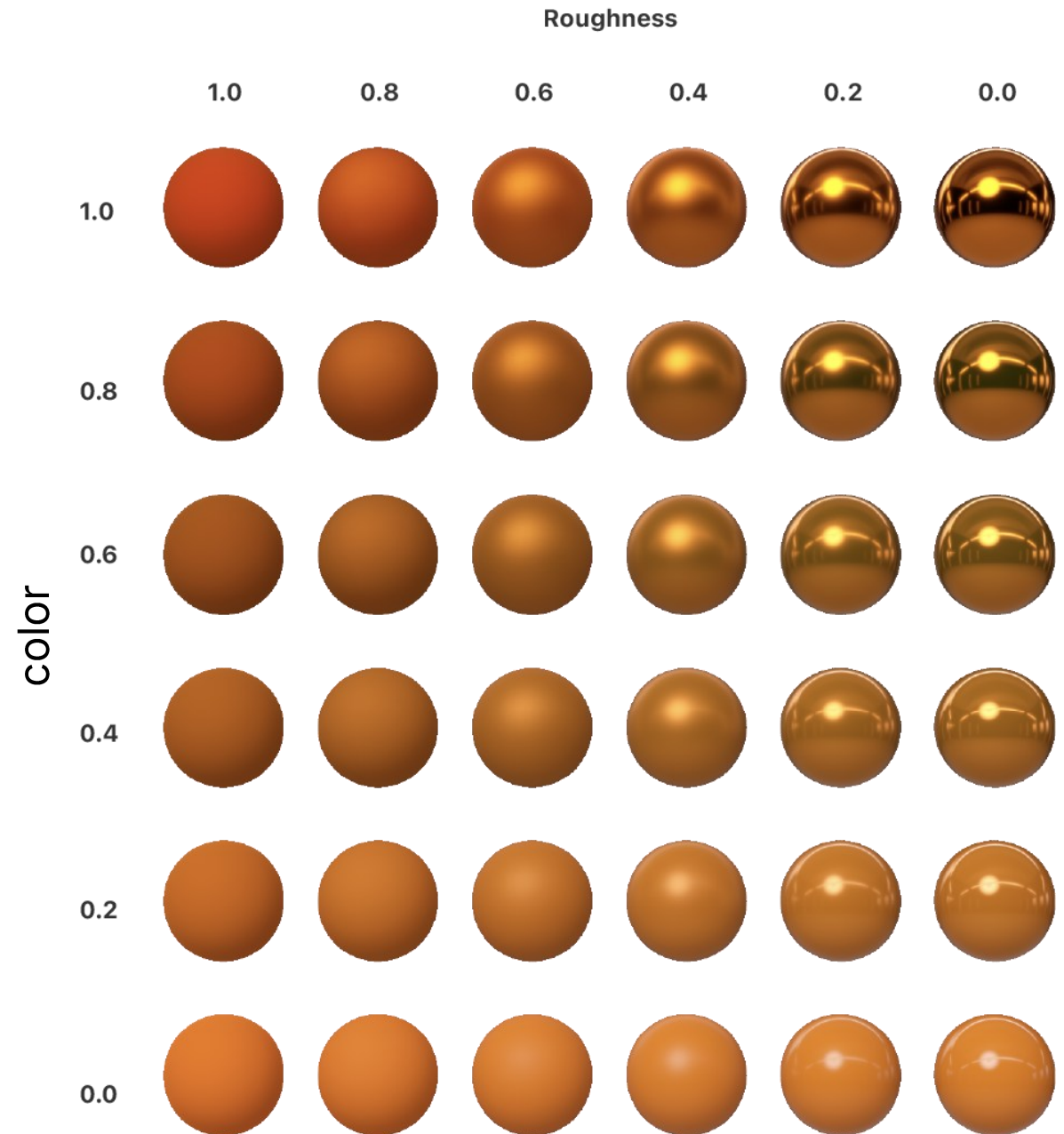
Metal material

- How to create a metal material?



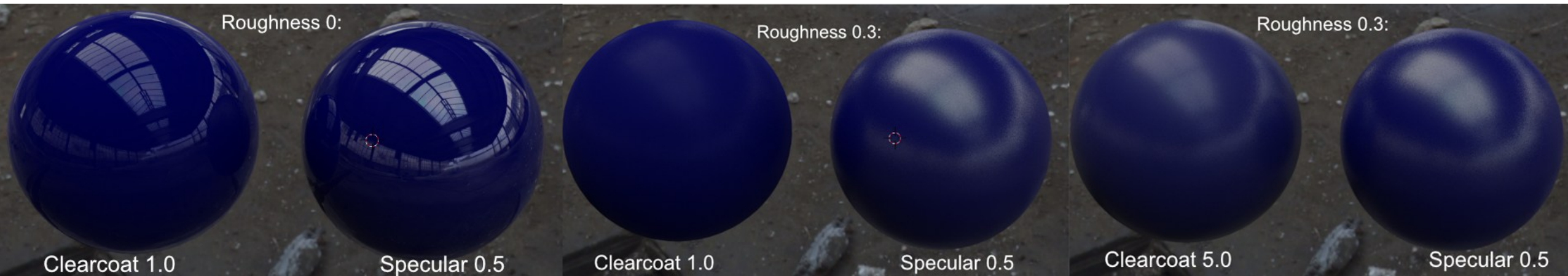
Metal material

- How to create a metal material?
- We just set:
 - a base color
 - And the roughness !



Bidirectional Reflectance Distribution Function (BRDF)

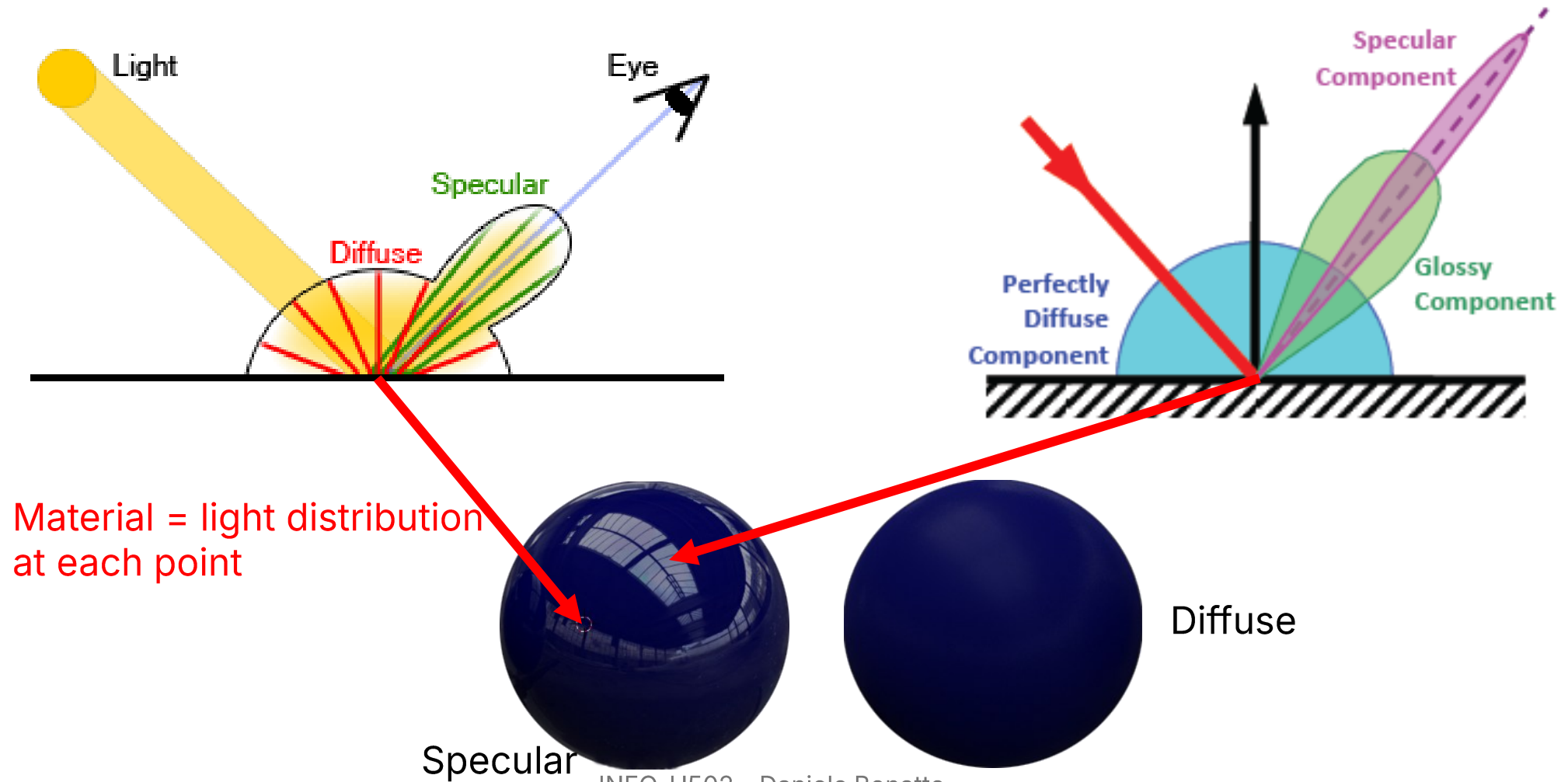
- More examples of diffuse and specular materials



- The BRDF function allows us to specify what a vertex will look like when there is light !

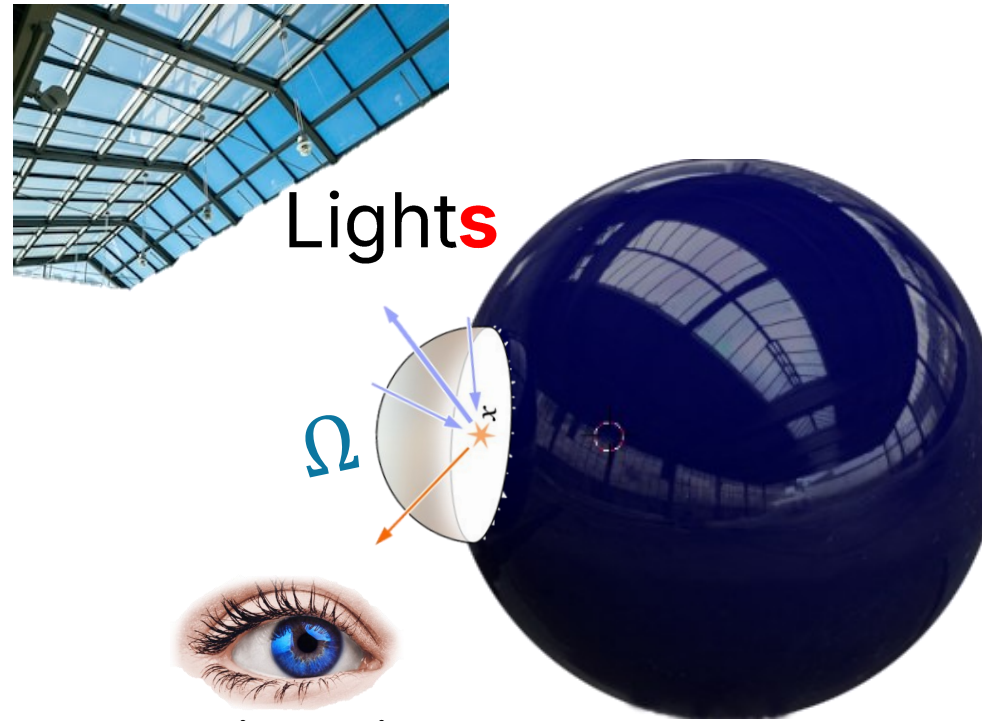
Bidirectional Reflectance Distribution Function (BRDF)

- Technically, we can have a different BRDF per vertex
- And the BRDF can be split in its components (diffuse, glossy, specular)



Bidirectional Reflectance Distribution Function (BRDF)

- Ok.. We have a way to describe how the light is distributed over a vertex.
- But..
 - It is not a description of the light (only the distribution of output given the input)
 - And several light_s bounce between objects, we need to sum all of them !



- Let's find a first model for the emitted light L_o

The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) =$$

- What could it be ?

• Let's find a first model for the emitted light L_o
$$L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

• What could it be ?

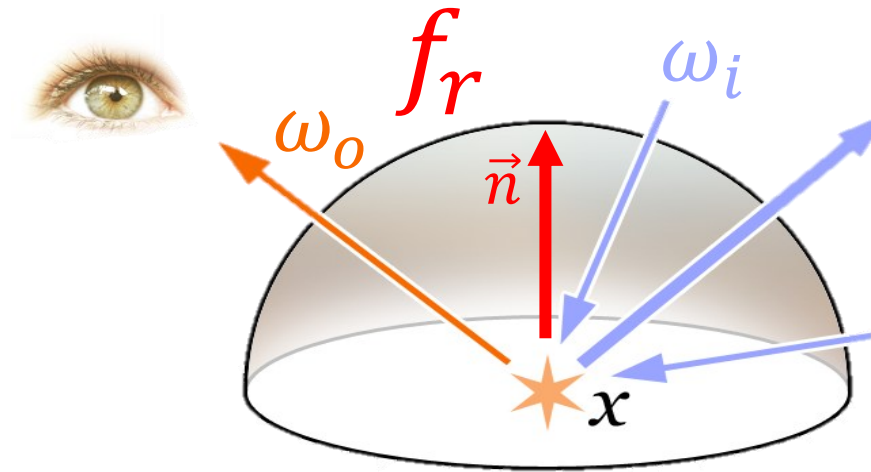


The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = f_r(\vec{x}, \omega_i, \omega_o, \lambda, t)$$

- What could it be ?
- There is at least the BRDF somewhere

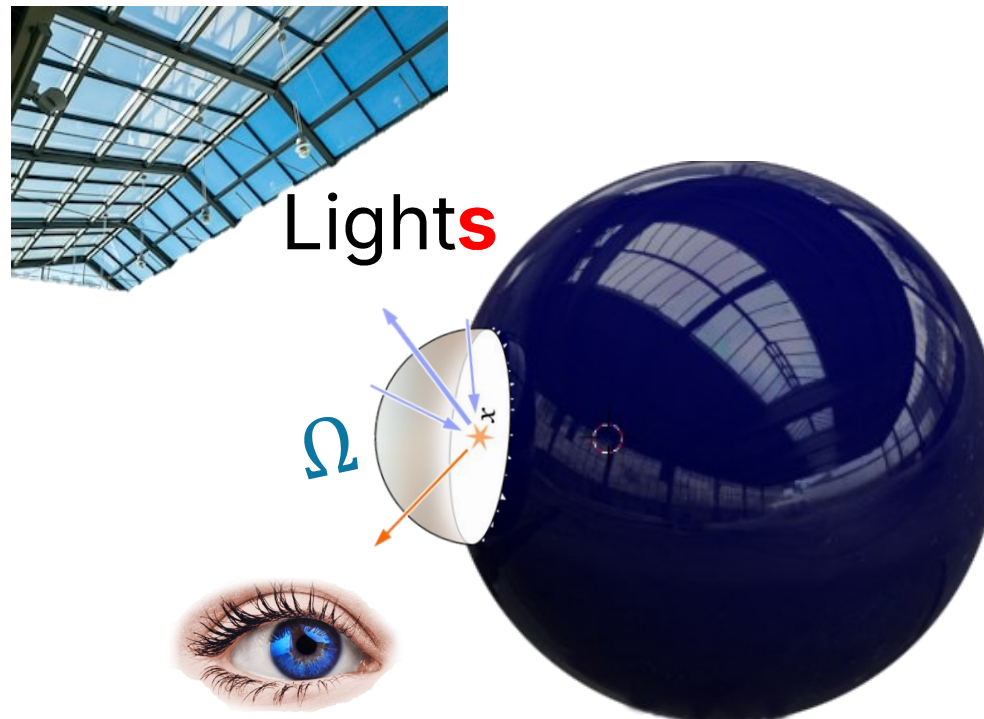


The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t)$$

- What could it be ?
- The BRDF is only a distribution, the light must come from somewhere !

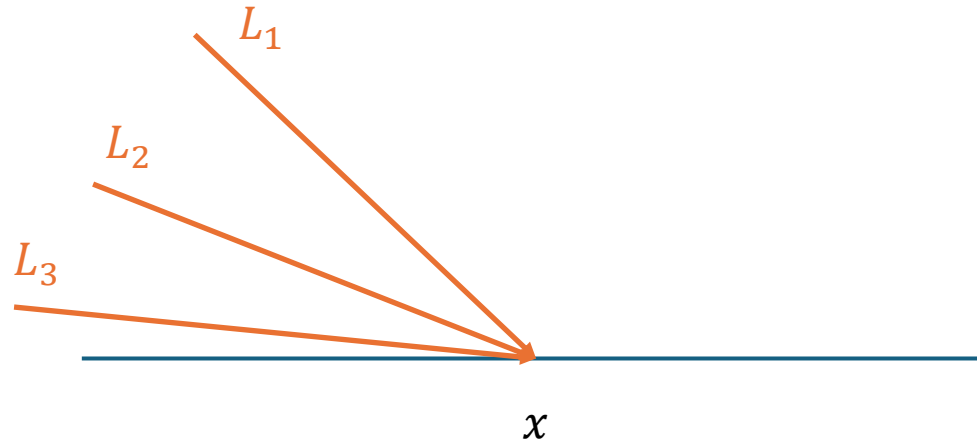


The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t)$$

- What could it be ?
- But light is directional, does it affect in the same way if it comes perpendicularly or parallelly to the point x ?

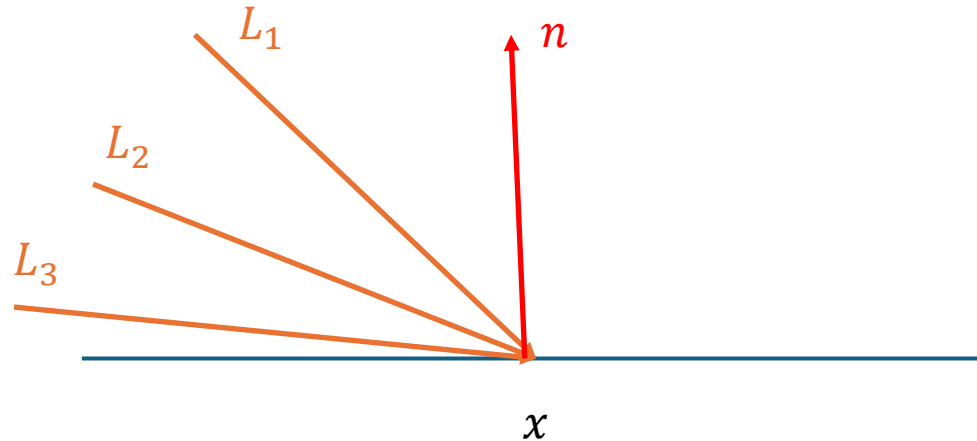


The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n})$$

- What could it be ?
- But light is directional, does it affect in the same way if it comes perpendicularly or parallelly to the point x ?

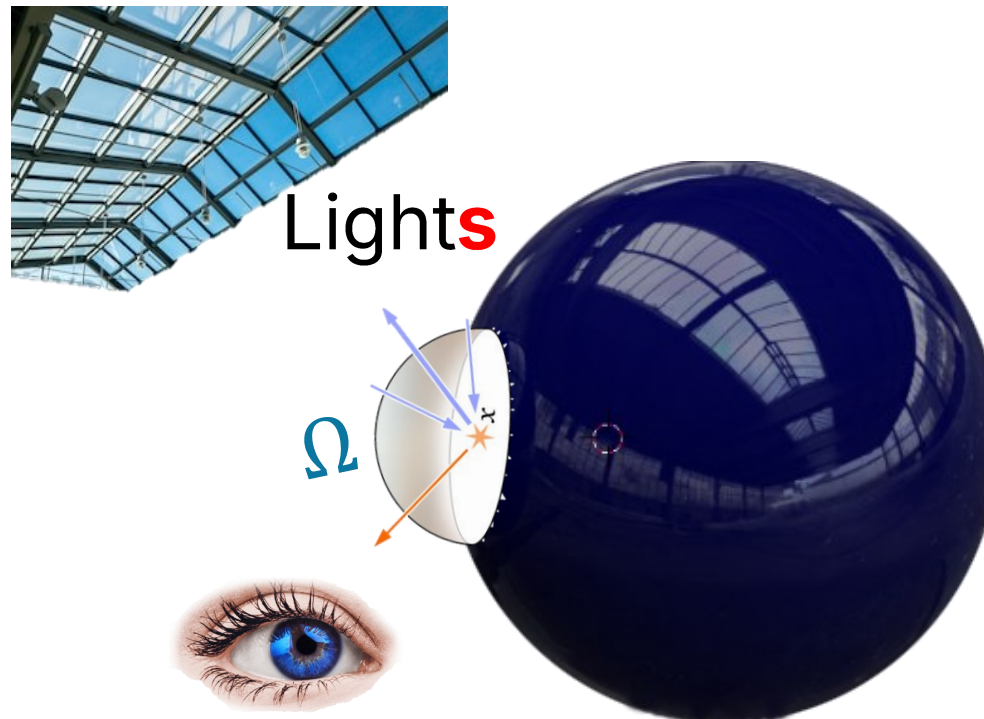


The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n})$$

- What could it be ?
- There are actually multiple lights, how to take them into account ?

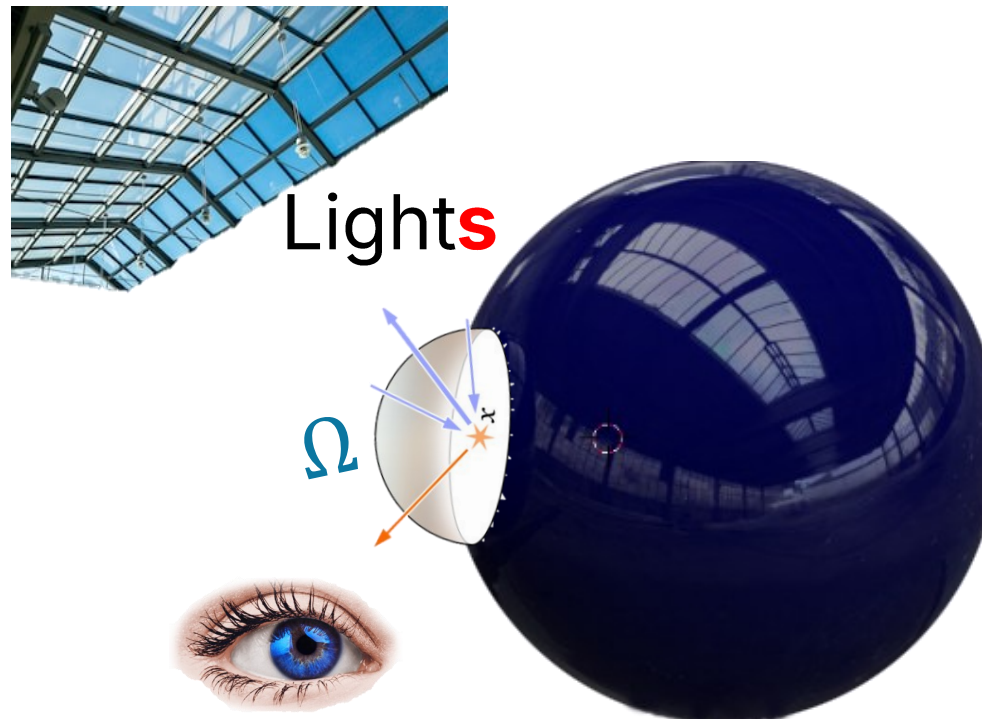


The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

- What could it be ?
- There are actually multiple lights, how to take them into account ?



The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

- What could it be ?
- What is the missing piece ?

The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

- What could it be ?
- What is the missing piece ?

hint



The Rendering Equation

- Let's find a first model for the emitted light L_o

$$L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

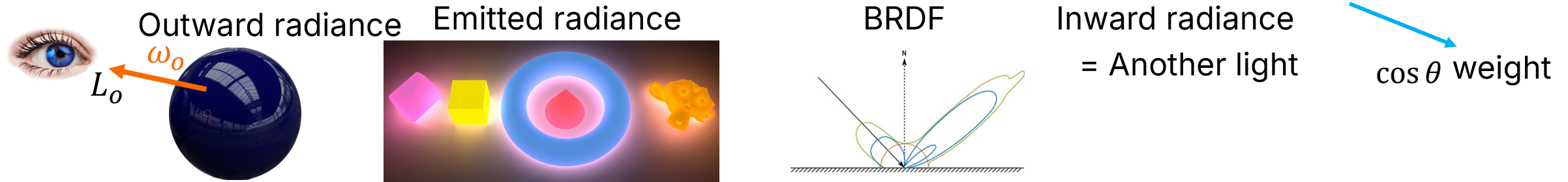
- What could it be ?
- What is the missing piece ? Yes! Some objects emit light !



The Rendering Equation

- In one slide

$$L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$



- We have a complete model of the observed light from a point x
- But wait..
- The light L_i in the integral.. Should come from somewhere..
- Where exactly ?

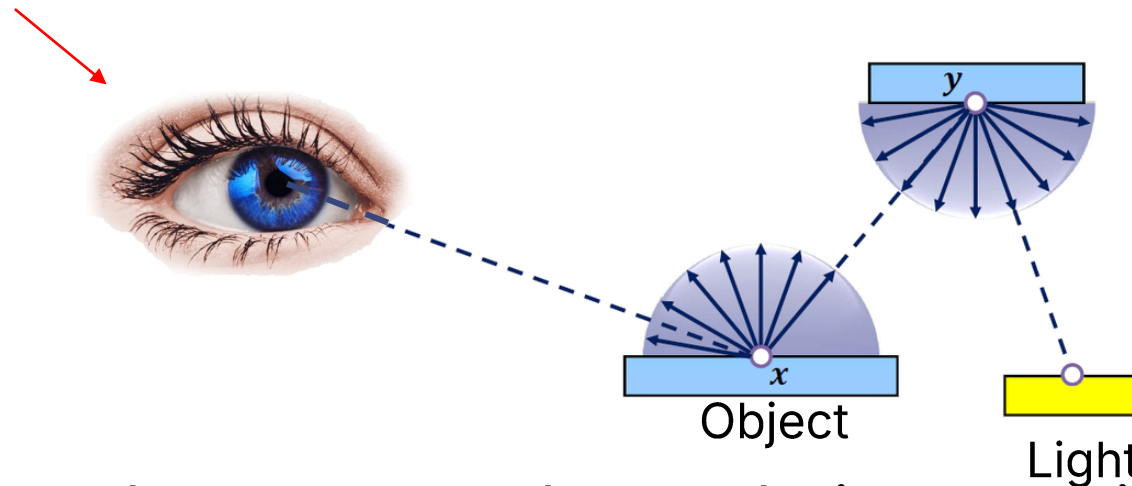
The Rendering Equation

- In one slide

$$L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

- The light L_i in the integral.. Should come from somewhere..
- Where exactly ?

This is a recursive call..



- At each step we need to compute the rendering equation for that point !
- But in OpenGL we do not have recursions.. How to do ?

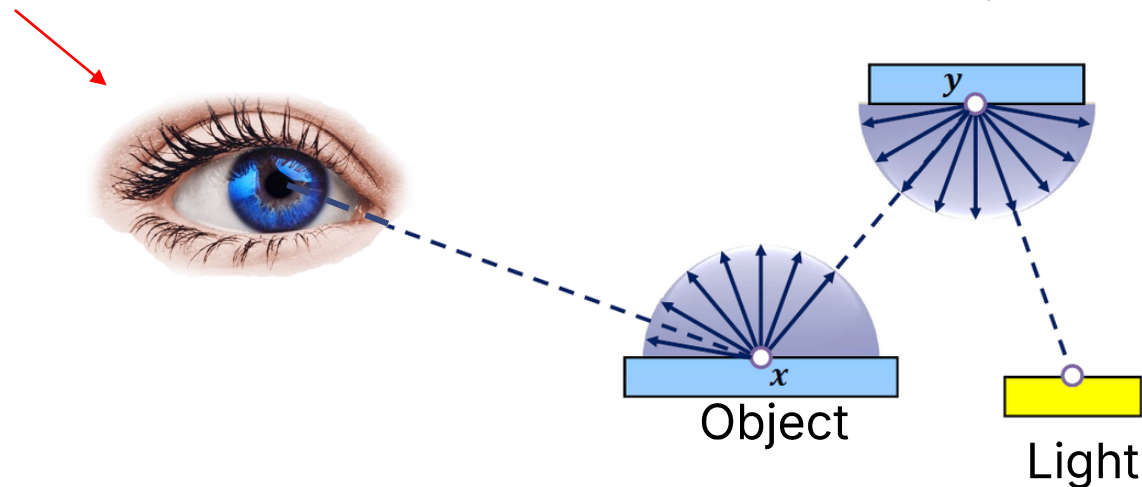
The Rendering Equation

- In one slide

$$L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

- At each step we need to compute the rendering equation for that point !
- But in OpenGL we do not have recursions.. How to do ?

This is a recursive call..



- Yes, we do **simplifications** that will look good, but are not physical ! Which?

The Rendering Equation – Simplifications

$$\begin{array}{ccccccc} L_o(\vec{x}, \omega_o, \lambda, t) & = & L_e(\vec{x}, \omega_o, \lambda, t) & + & \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i \\ \text{Outward radiance} & & \text{Emitted radiance} & & \text{BRDF} & & \text{Inward radiance} \end{array}$$

- The first simplification consists in removing the time and wavelength
 - Who cares.

$$L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$$

$$L_o(\vec{x}, \omega_o) = L_e(\vec{x}, \omega_o) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) (\omega_i \cdot \vec{n}) d\omega_i$$

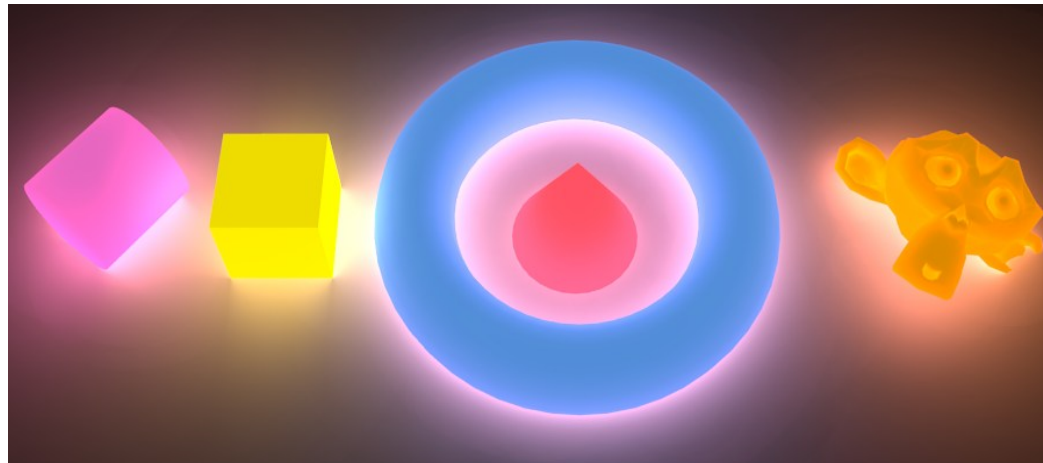
- The lambda is actually split in (R,G,B) colors only when you implement it.
- If you do a rendering engine that uses them, you need to add them back !

The Rendering Equation – Simplifications

$$\underbrace{L_o(\vec{x}, \omega_o)}_{\text{Outward radiance}} = \underbrace{L_e(\vec{x}, \omega_o)}_{\text{Emitted radiance}} + \underbrace{\int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o)}_{\text{BRDF}} \underbrace{L_i(\vec{x}, \omega_i)}_{\text{Inward radiance}} (\omega_i \cdot \vec{n}) d\omega_i$$

- The emitted light $L_e(\vec{x}, \omega_o)$ can be considered isotropic
- We set it as a constant

$$L_o(\vec{x}, \omega_o) = \textcolor{red}{L_e(\vec{x}, \omega_o)} + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) (\omega_i \cdot \vec{n}) d\omega_i$$
$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) (\omega_i \cdot \vec{n}) d\omega_i$$



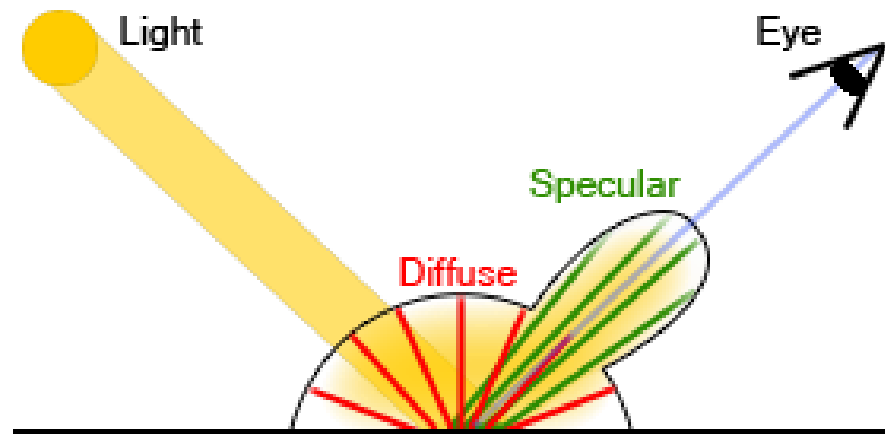
The Rendering Equation – Simplifications

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) (\omega_i \cdot \vec{n}) d\omega_i$$

Outward radiance Emitted radiance BRDF Inward radiance

- We saw previously, the BRDF can be decomposed into diffuse and specular!

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) (\omega_i \cdot \vec{n}) d\omega_i$$
$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (diff + spec) L_i(\vec{x}, \omega_i) (\omega_i \cdot \vec{n}) d\omega_i$$



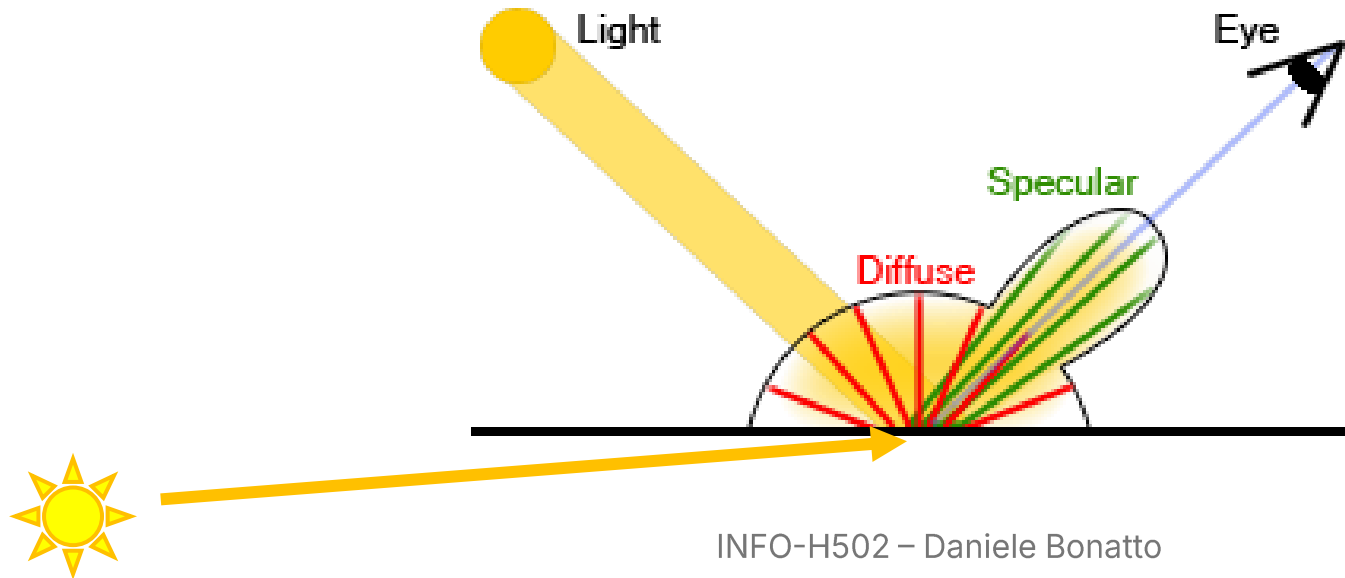
The Rendering Equation – Simplifications

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) (\omega_i \cdot \vec{n}) d\omega_i$$

Outward radiance Emitted radiance BRDF Inward radiance

- What happens if the light comes from below the surface ?
- We need to discard those values!

$$(\omega_i \cdot \vec{n}) \mapsto \max(\omega_i \cdot \vec{n}, 0)$$



The Rendering Equation – Simplifications

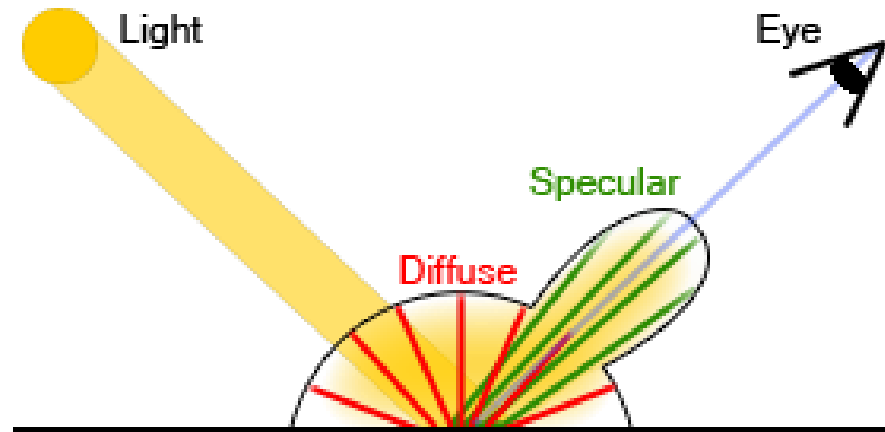
$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) \max(\omega_i \cdot \vec{n}, 0) d\omega_i$$

Outward radiance Emitted radiance BRDF Inward radiance

- What happens if the light comes from below the surface ?

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o) L_i(\vec{x}, \omega_i) \max(\omega_i \cdot \vec{n}, 0) d\omega_i$$

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (diff + spec) L_i(\vec{x}, \omega_i) \max(\omega_i \cdot \vec{n}, 0) d\omega_i$$

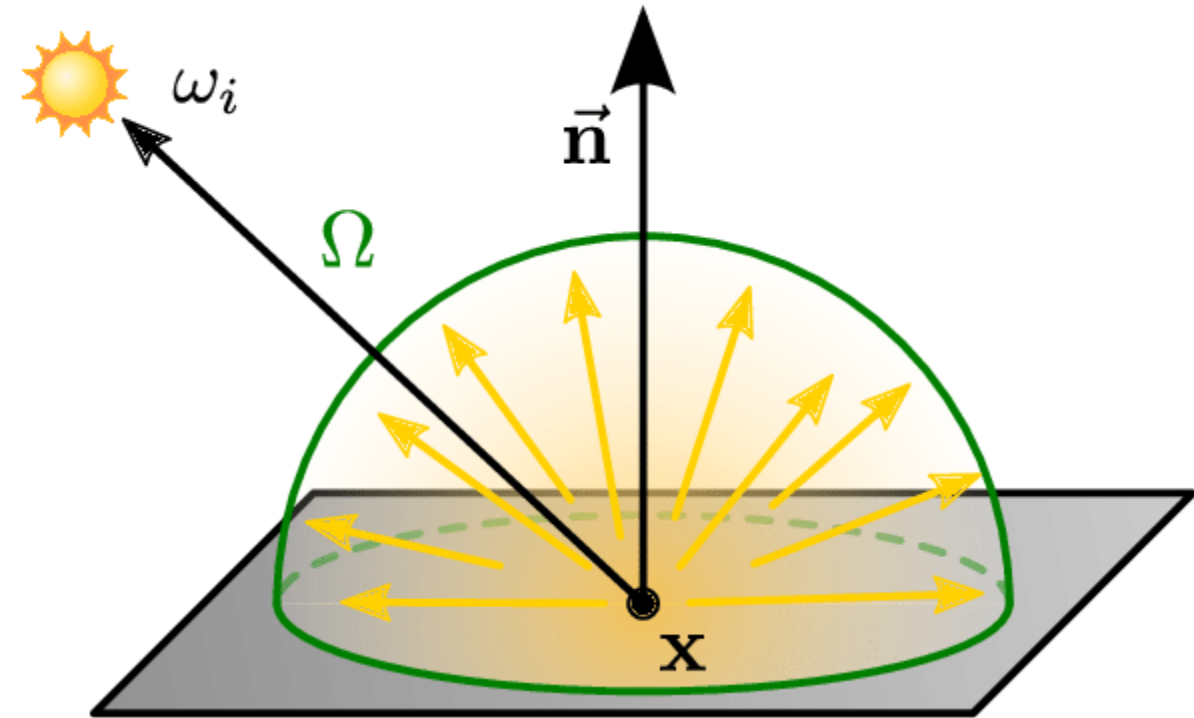


The Rendering Equation – Simplifications – Diffuse

- Let's focus on the diffuse light in the BRDF *diff*
- What is the shape of a diffuse light ?

The Rendering Equation – Simplifications – Diffuse

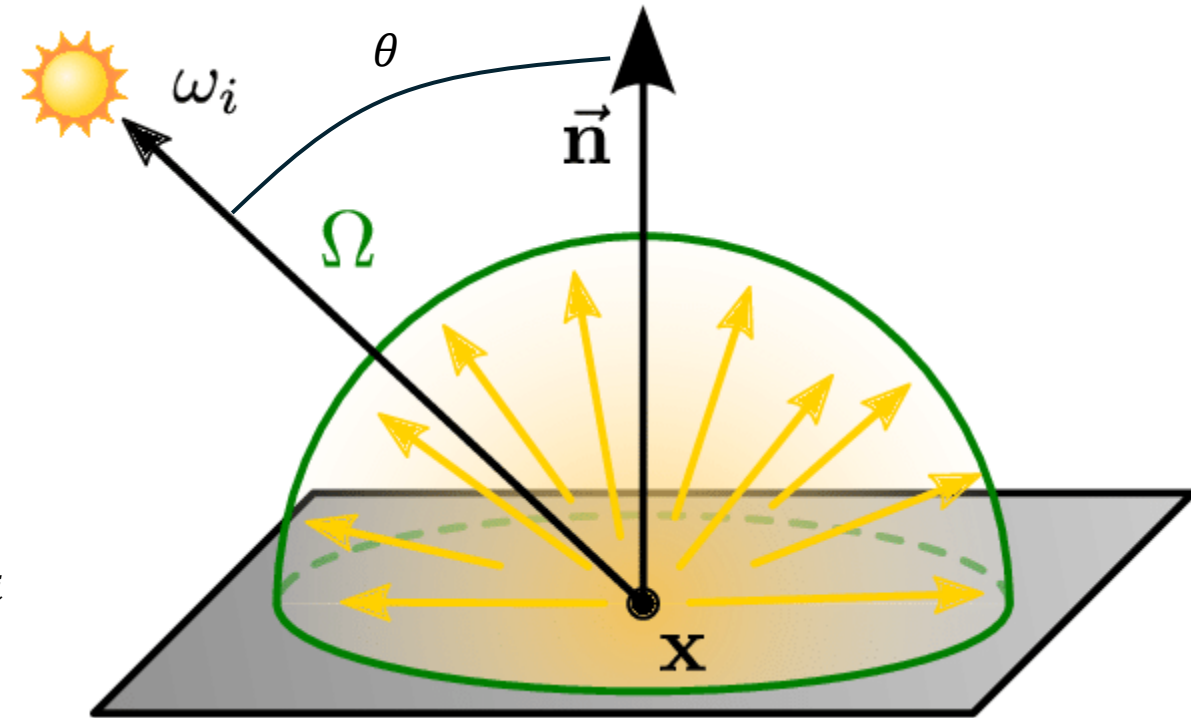
- Let's focus on the diffuse light in the BRDF *diff*
- What is the shape of a diffuse light ?
- Yes, it is symmetrical, how to simplify ?



The Rendering Equation – Simplifications – Diffuse

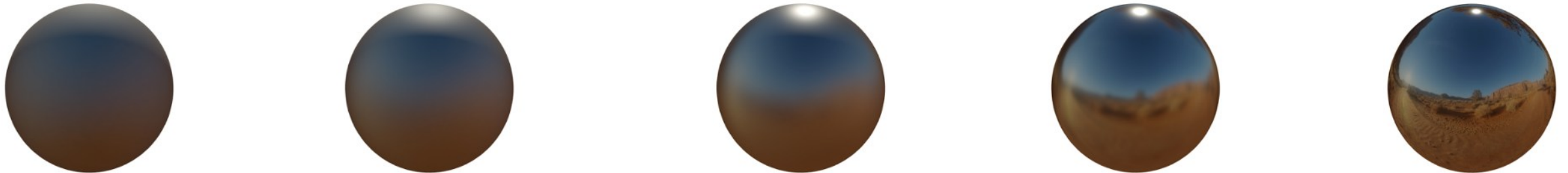
- Let's focus on the diffuse light in the BRDF *diff*
- What is the shape of a diffuse light ?
- Yes, it is symmetrical, how to symplify ?
- Just put $diff = 1$! Or some color !
- So the diffuse distribution is modeled by $(\omega_i \cdot \vec{n}) = \cos \theta$
- But.. We alredy have this component !

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (\textcolor{blue}{diff} + spec) L_i(\vec{x}, \omega_i) \textcolor{brown}{max}(\omega_i \cdot \vec{n}, 0) d\omega_i$$



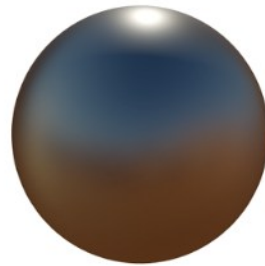
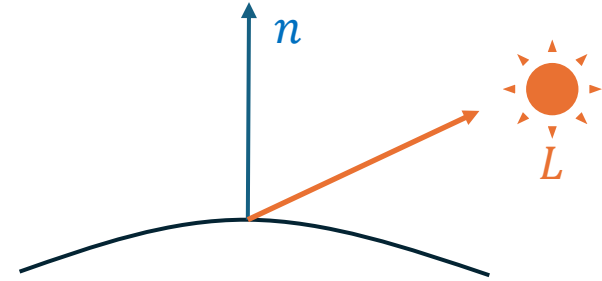
The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Look at a specular object near you, try to understand how specularities work !



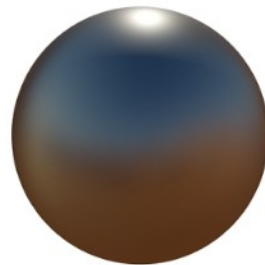
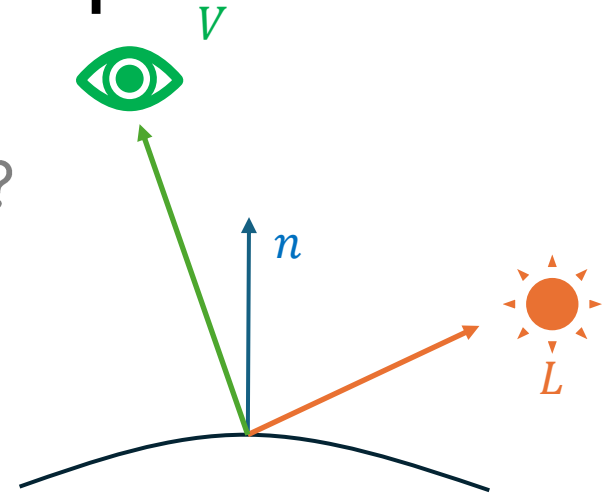
The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Light vector L



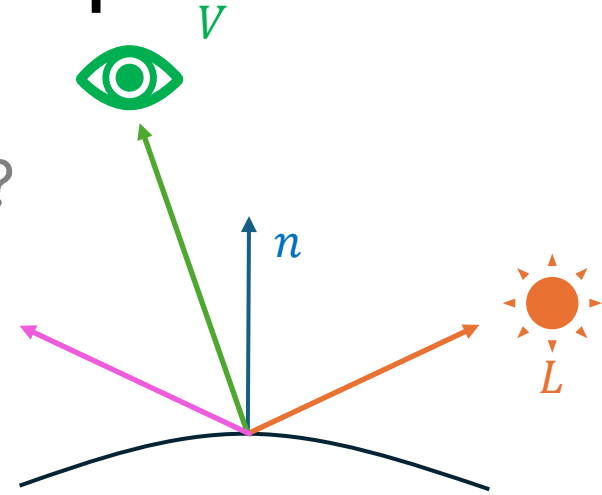
The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Light vector L , **Eye vector v ! (= the camera position)**
- But now, how to compute the light perceived by the eye ?

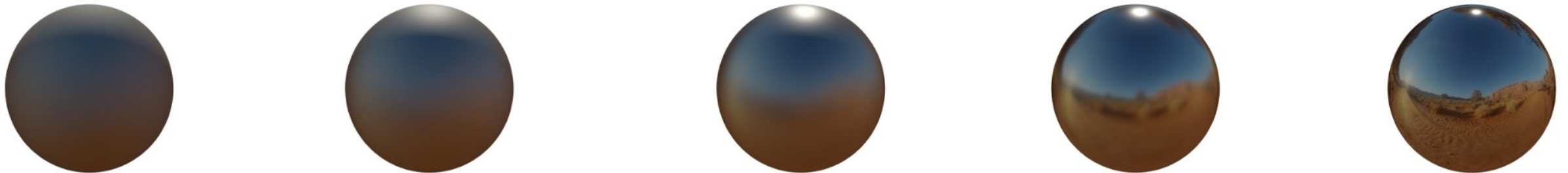


The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Light vector L , Eye vector V ! (= the camera position)
- But now, how to compute the light perceived by the eye ? R

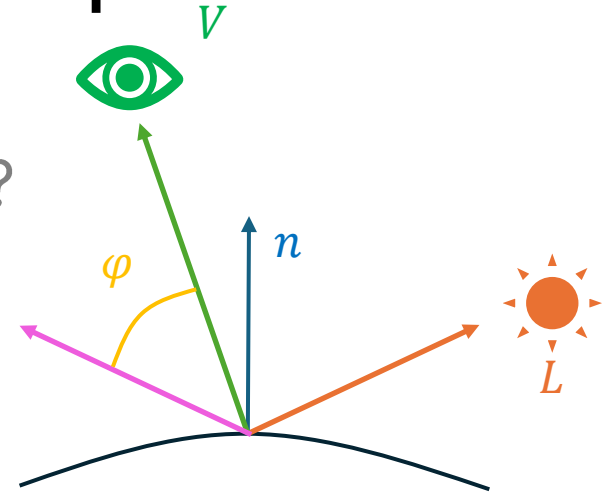


- The light we perceive is on the opposite side of the light with the normal
- Trick: Introduce the reflection vector of the light R
- What now ?

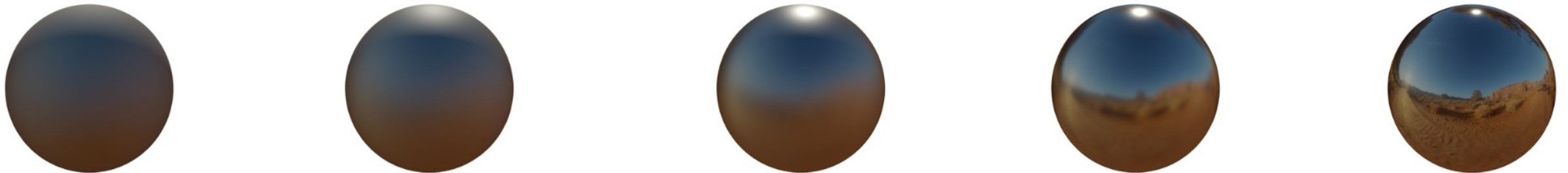


The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Light vector L , Eye vector V ! (= the camera position)
- But now, how to compute the light perceived by the eye ?

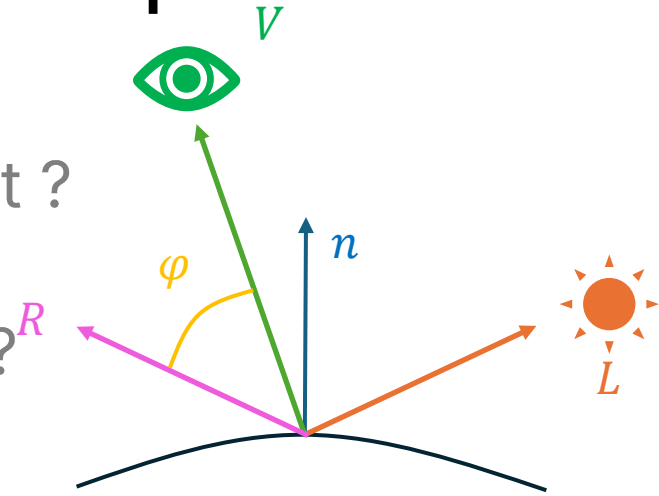


- R defines an angle φ with the eye, can we do something with it ?

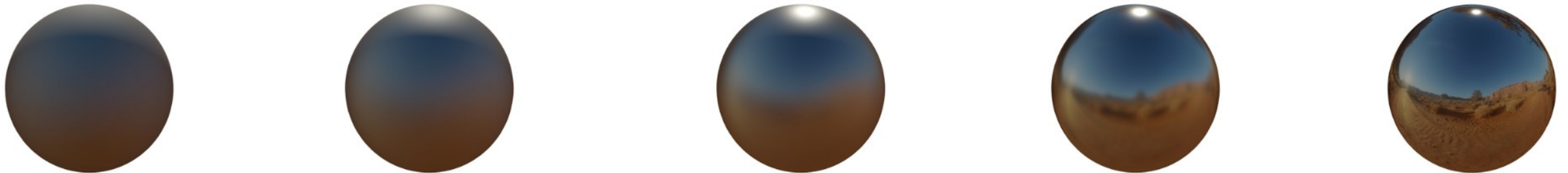


The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Light vector L , Eye vector V ! (= the camera position)
- But now, how to compute the light perceived by the eye ?

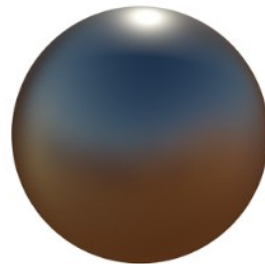
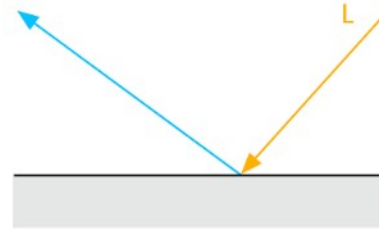
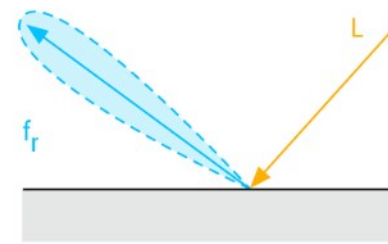
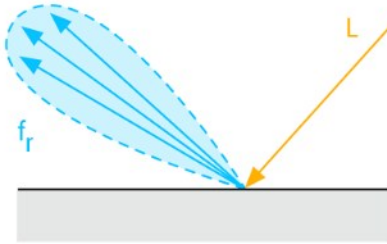
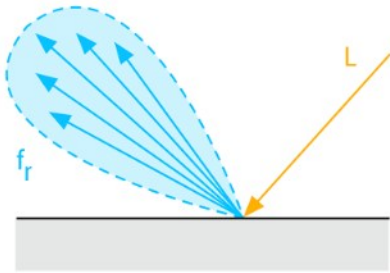
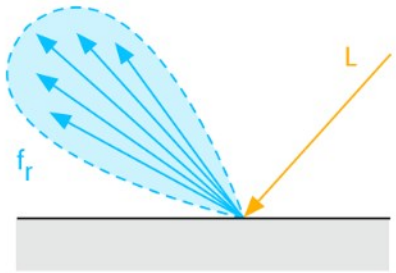
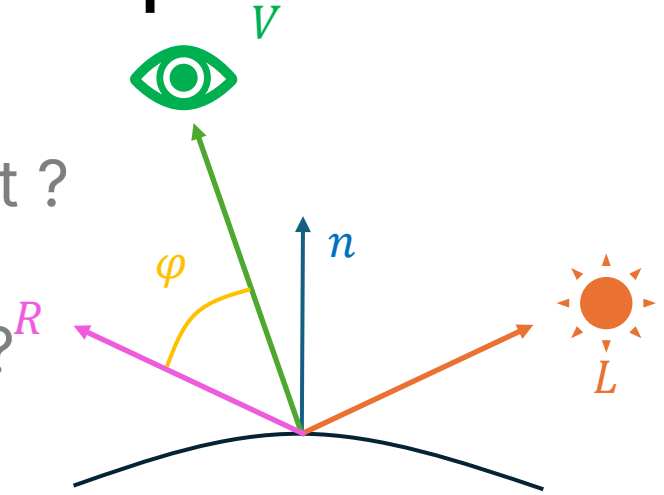


- R defines an angle φ with the eye, can we do something with it ?
- Yes, we can use it within a dot product $R \cdot V$
- But...



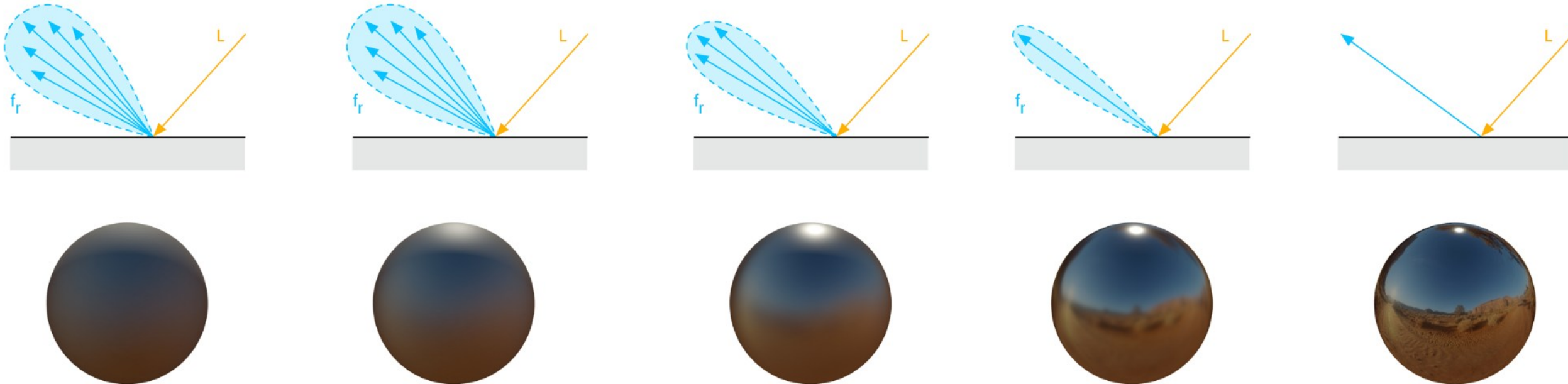
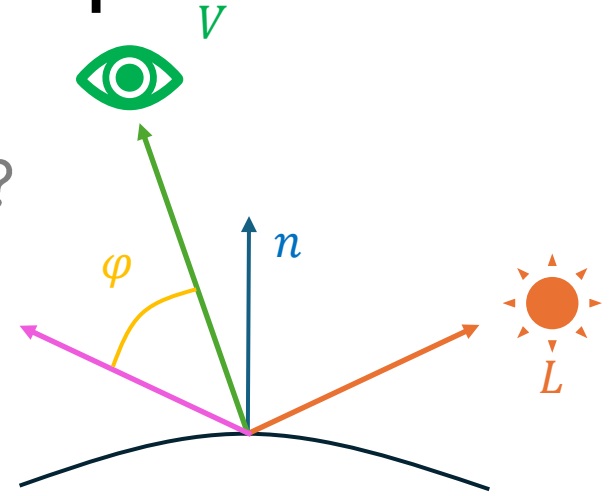
The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Light vector L , Eye vector V ! (= the camera position)
- But now, how to compute the light perceived by the eye ?
- Yes, we can use it within a dot product $R \cdot V$, But...



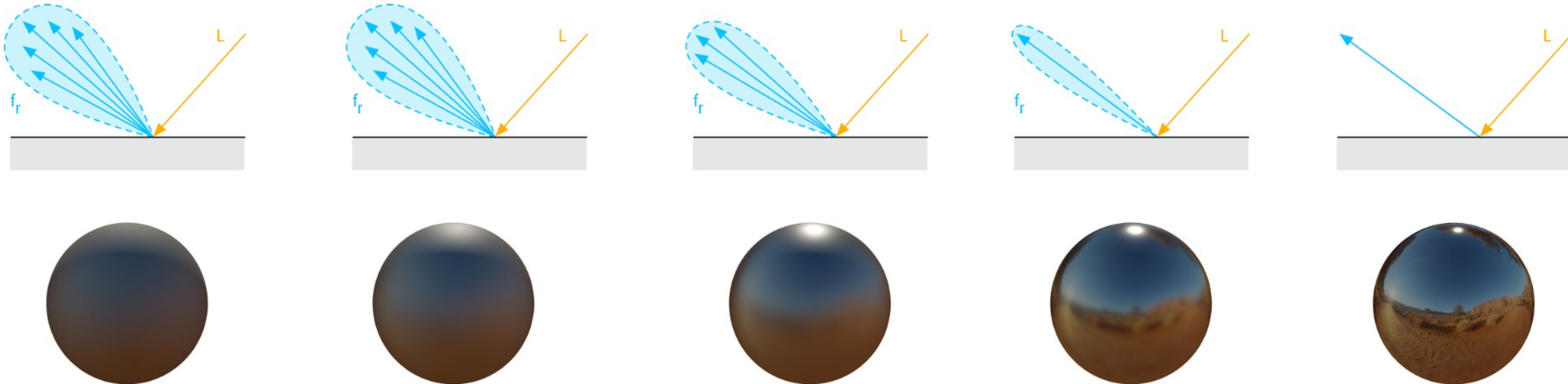
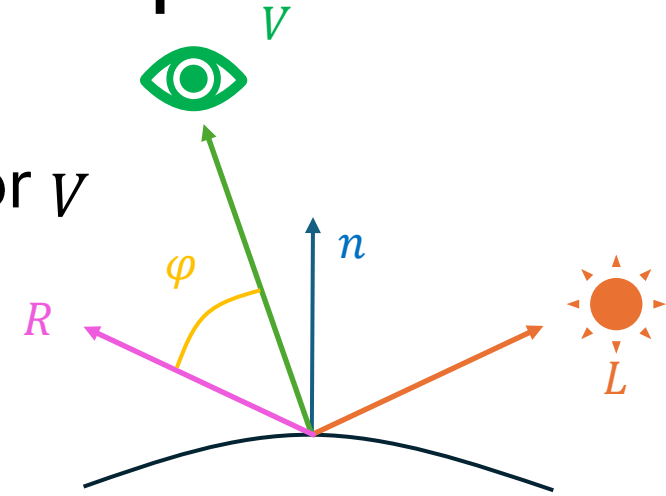
The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- What elements should we take into account to compute it ?
 - Light vector L , Eye vector V ! (= the camera position)
- But now, how to compute the light perceived by the eye ? R
- Modulate the **strength** of the dot product $spec = \cos^{\gamma} \varphi$



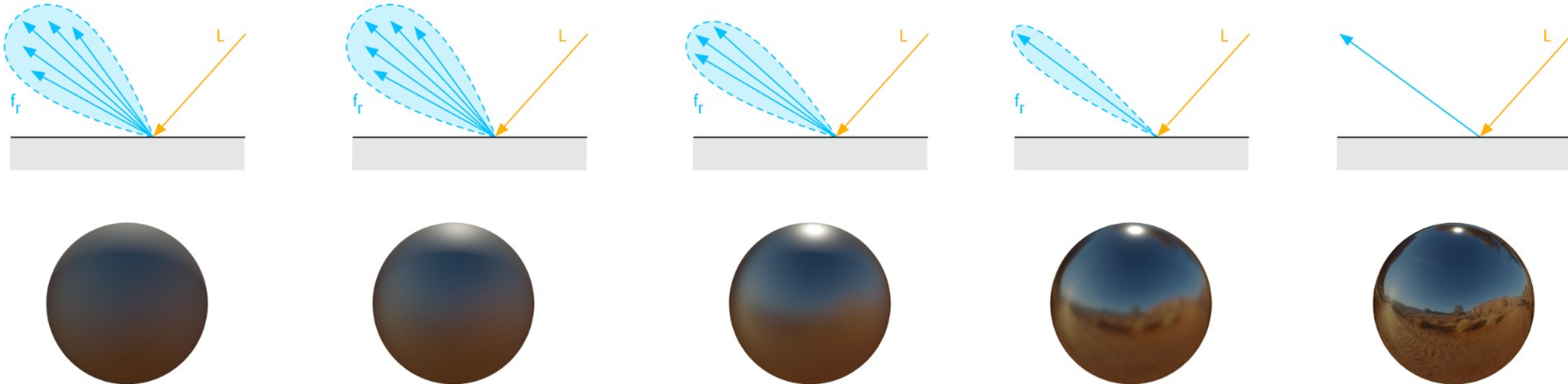
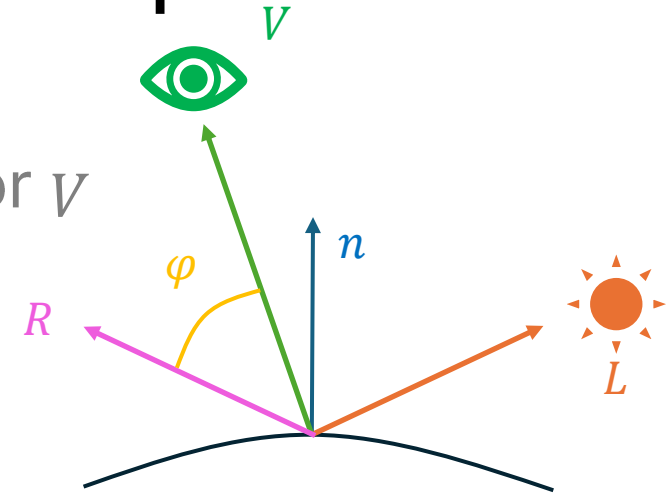
The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- In summary: spec depends on : Light vector L , Eye vector V
- We model it as $spec = \cos^r \varphi = (R \cdot V)^r$
- Notice something missing?



The Rendering Equation – Simplifications – Specular

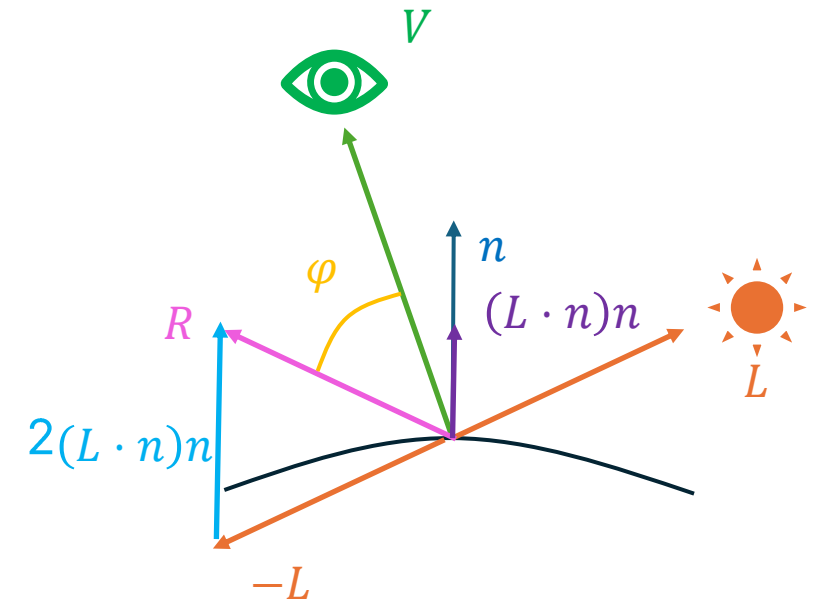
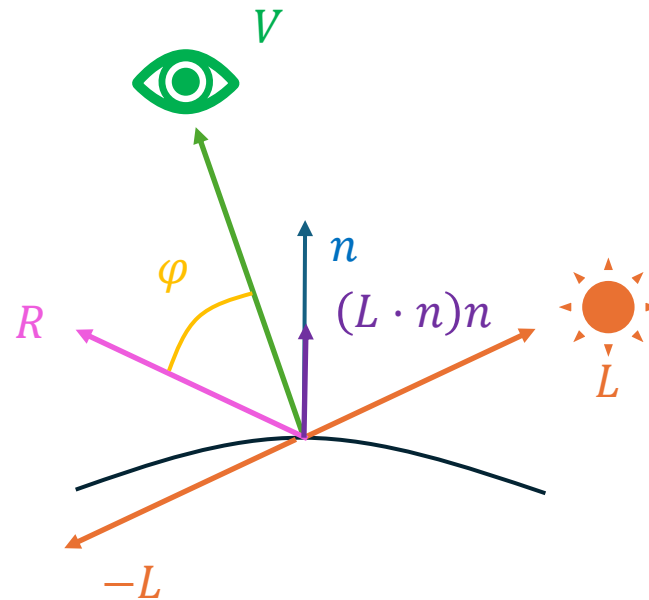
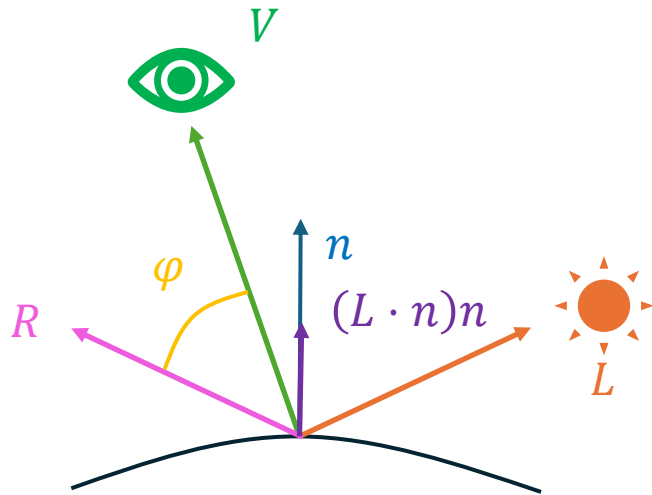
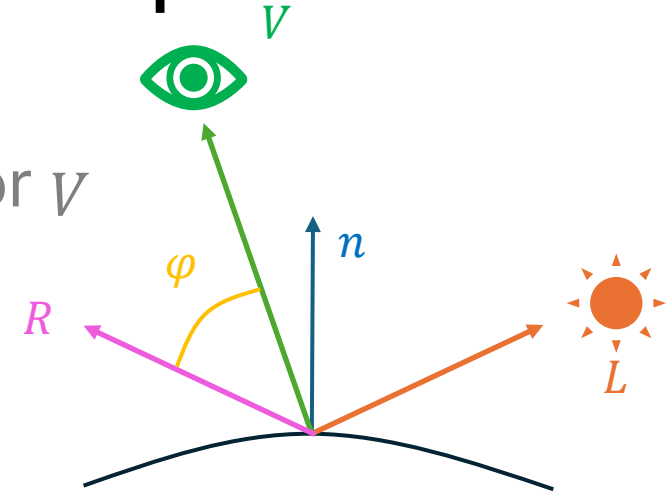
- What about the specular light ? *spec*
- In summary: spec depends on : Light vector L , Eye vector V
- We model it as $spec = \cos^r \varphi = (R \cdot V)^r$
- Notice something missing?
 - We do not know R ... Back to geometry !



The Rendering Equation – Simplifications – Specular

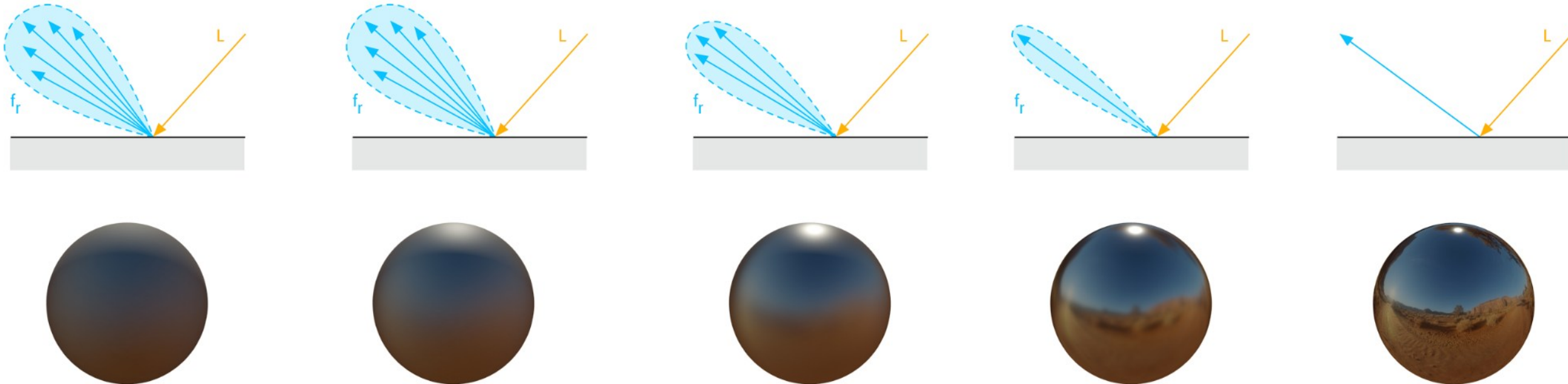
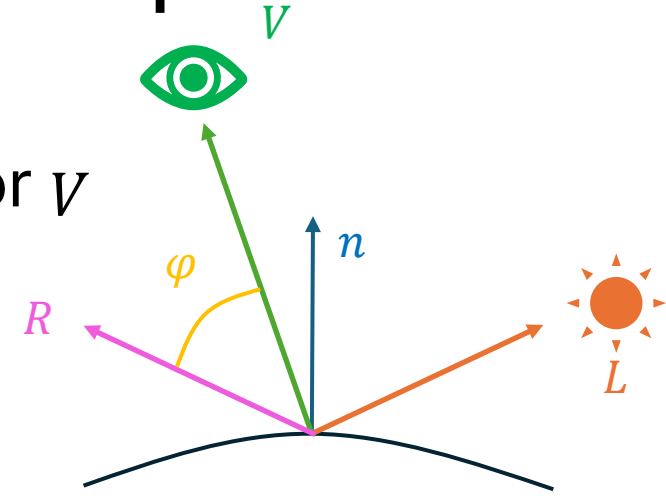
- What about the specular light ? *spec*
- In summary: spec depends on : Light vector L , Eye vector V
- How to find R ?

$$R = 2(L \cdot n)n - L$$



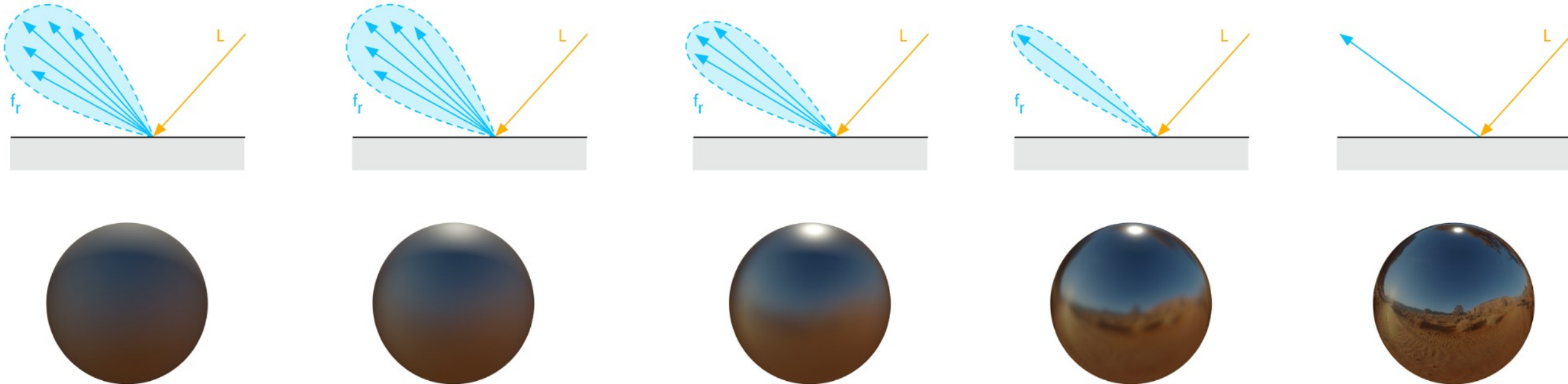
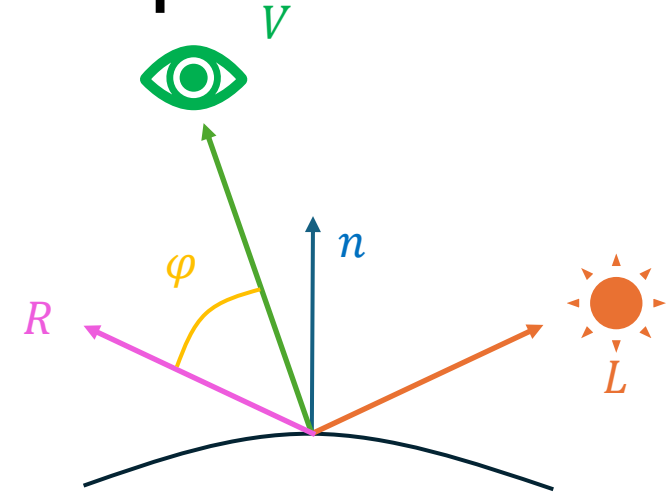
The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- In summary: spec depends on : Light vector L , Eye vector V
- We model it as $spec = \cos^r \varphi = (R \cdot V)^r$
- R is given by $R = 2(L \cdot n)n - L$
- Is there still an issue with $spec_{\max}(\omega_i \cdot \vec{n}, 0)$?



The Rendering Equation – Simplifications – Specular

- What about the specular light ? *spec*
- Is there still an issue with $spec \max(\omega_i \cdot \vec{n}, 0)$?
 - We don't really care about $\max(\omega_i \cdot \vec{n}, 0)$
 - It is kinda included in our model, we remove it !
- This is only a model, called the **Blinn-Phong** model



The Rendering Equation – Simplifications – Specular

- In summary, the BRDF was split into

$$f_r(\vec{x}, \omega_i, \omega_o) \max(\omega_i \cdot \vec{n}, 0) = (diff + spec) \max(\omega_i \cdot \vec{n}, 0)$$

- Then we defined the diffuse component as

$$diff = 1$$

- And the specular component as (Blinn-Phong model):

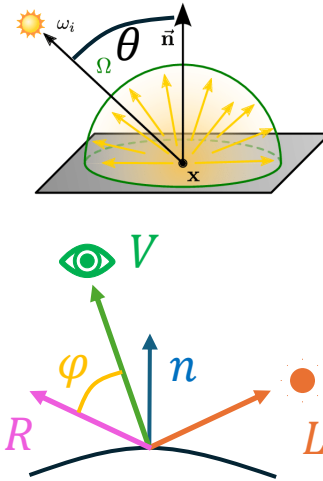
$$spec = (R \cdot V)^{\gamma}$$

- We have:

$$f_r(\vec{x}, \omega_i, \omega_o) \max(\omega_i \cdot \vec{n}, 0) = (\max(\omega_i \cdot \vec{n}, 0) + (R \cdot V)^{\gamma})$$

The $\omega_i \cdot \vec{n}$ disappeared here !

- Let's get back to our simplifications.

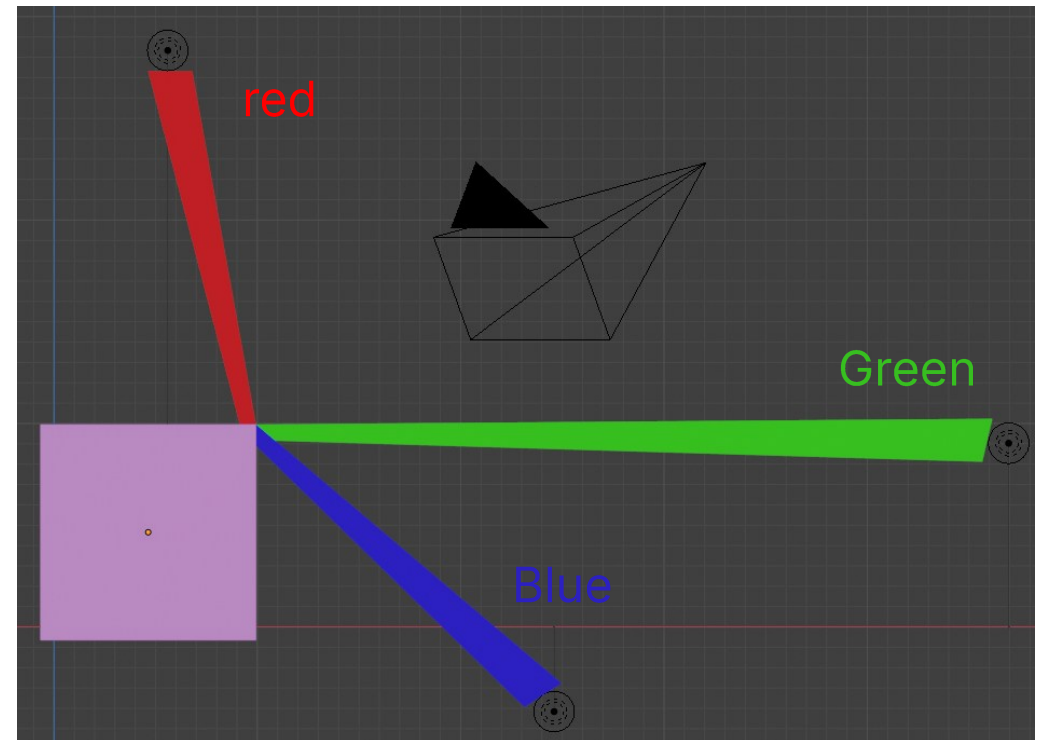


The Rendering Equation – Simplifications

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (diff + spec) L_i(\vec{x}, \omega_i) d\omega_i$$

Outward radiance Emitted radiance BRDF Inward radiance

- Now we cannot intersect with the bounces of light on several objects..
- But we still can take into account the different light sources L in our scene !
- What can we do ?



The Rendering Equation – Simplifications

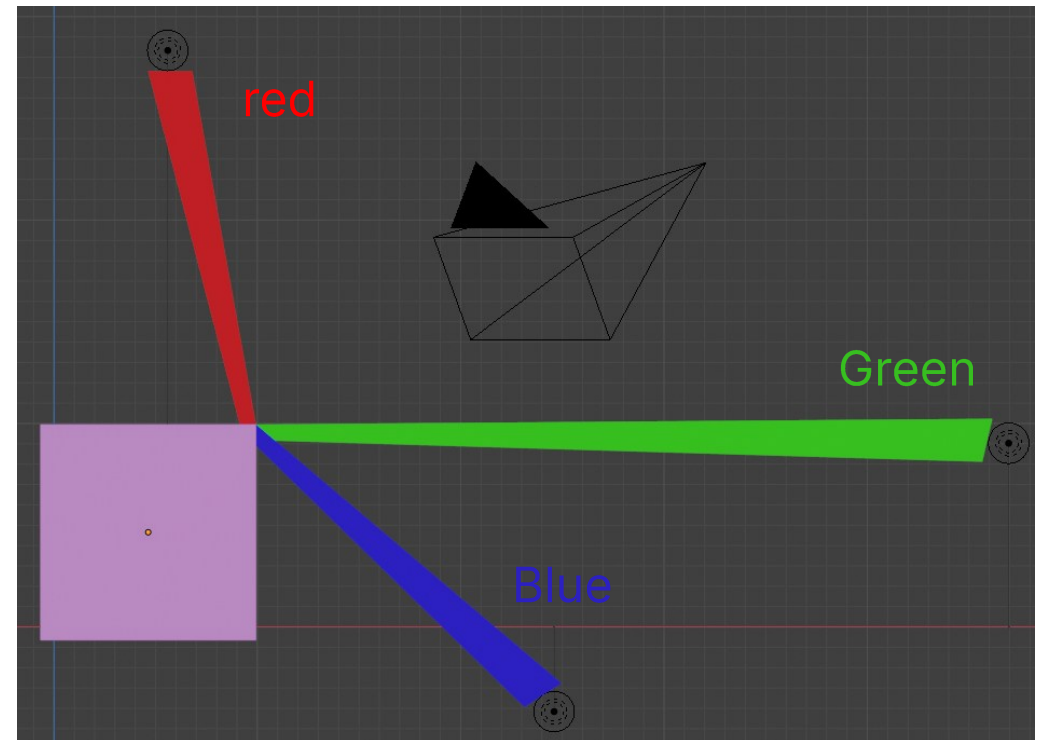
$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (diff + spec) L_i(\vec{x}, \omega_i) d\omega_i$$

Outward radiance Emitted radiance $\int_{\Omega}$ BRDF Inward radiance

- Now we cannot interact with the bounces of light on several objects..
- But we still can take into account the different light sources L in our scene !

$$L_i(\vec{x}, \omega_i) = L_i$$

- We set them as constants
- One for each light !



The Rendering Equation – Simplifications

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (diff + spec) L_i d\omega_i$$

Outward radiance Emitted radiance BRDF Inward radiance

- Light can be attenuated by another way than the cosine
- Do you see which one ?

The Rendering Equation – Simplifications

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (diff + spec)L_i d\omega_i$$

Outward radiance

Emitted radiance

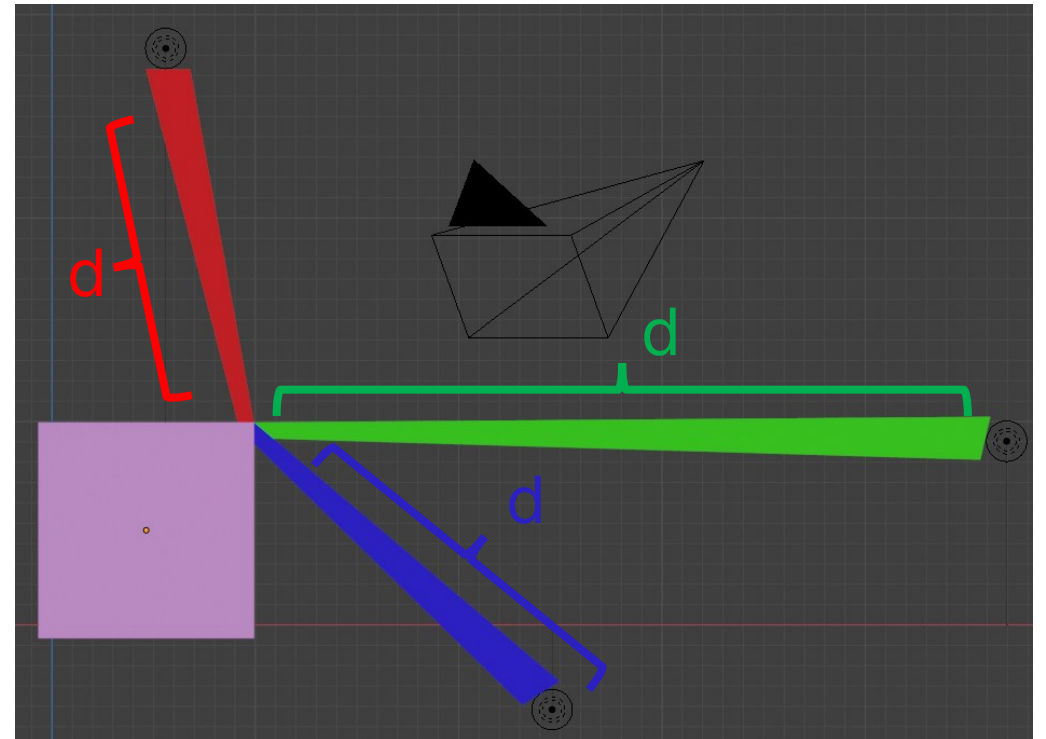
BRDF

Inward radiance

- Light can be attenuated by another way than the cosine
- Do you see which one ?
- Light are attenuated with the distance !
- We change again L_i

$$L_i \mapsto L_i \cdot att_i$$

- But, how is it computed ?



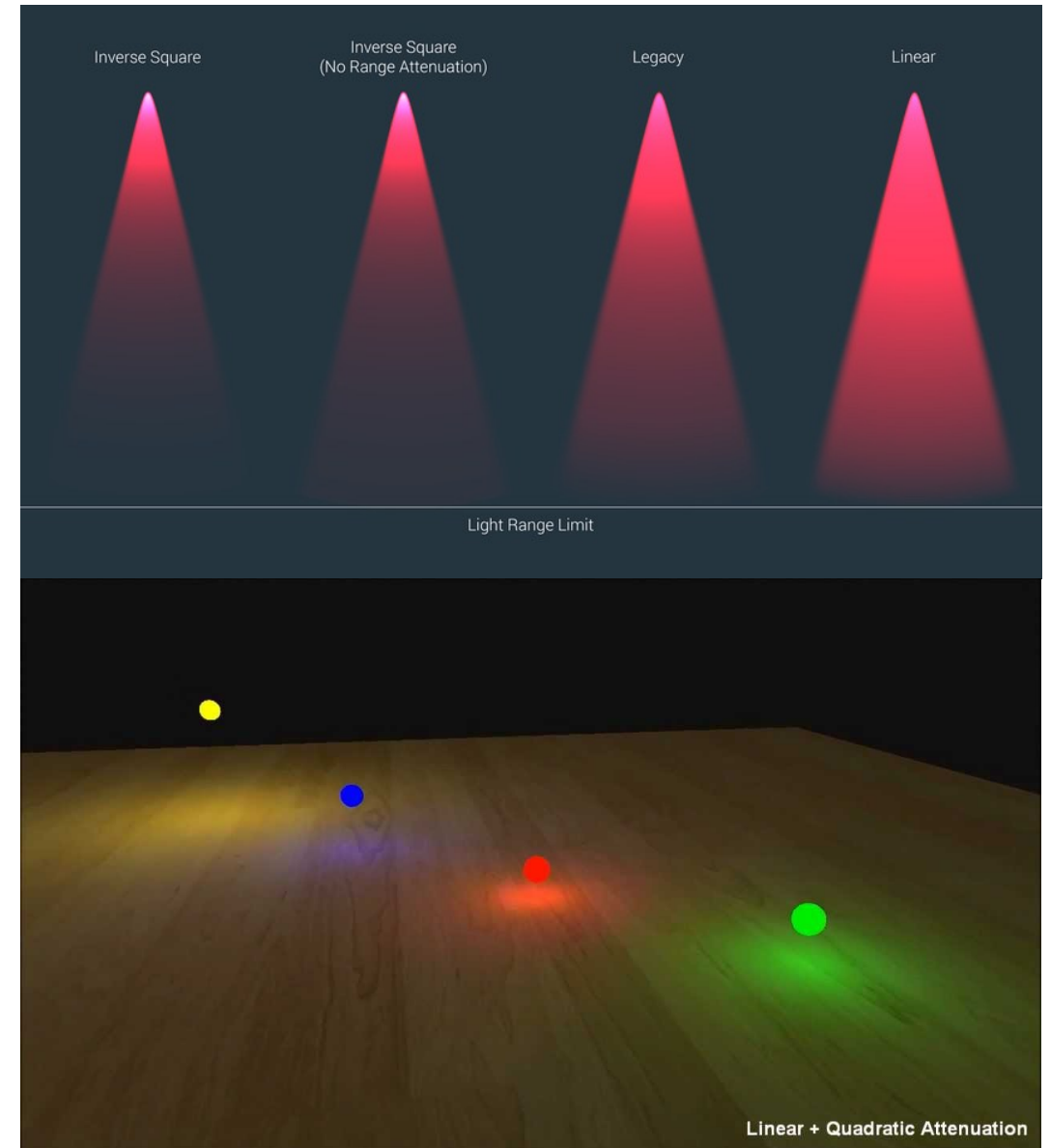
The Rendering Equation – Simplifications

How is the attenuation factor computed?

- The attenuation factor uses a simple polynomial model

$$att_i = \frac{1}{k_c^i + k_l^i d^i + k_q^i (d^i)^2}$$

- k_c = Constant attenuation
- k_l = Linear attenuation
- k_q = Quadratic attenuation
- d = Distance between the light source and the vertex



The Rendering Equation – Simplifications

$$L_o(\vec{x}, \omega_o) = E + \int_{\Omega} (diff + spec) L_i att_i d\omega_i$$

Outward radiance

Emitted radiance

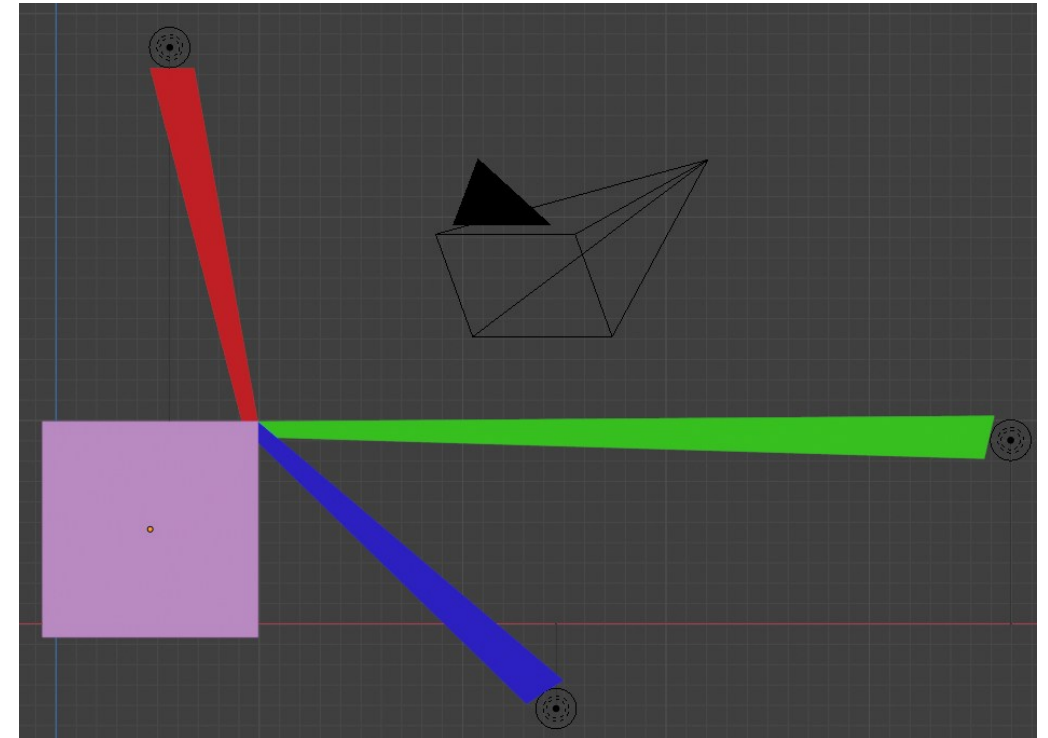
BRDF

Inward radiance

- Finally, we can remove the integral, as we have a finite number of lights.

$$\int_{\Omega} d\omega_i \mapsto \sum_i^n$$

Only 3 lights here!

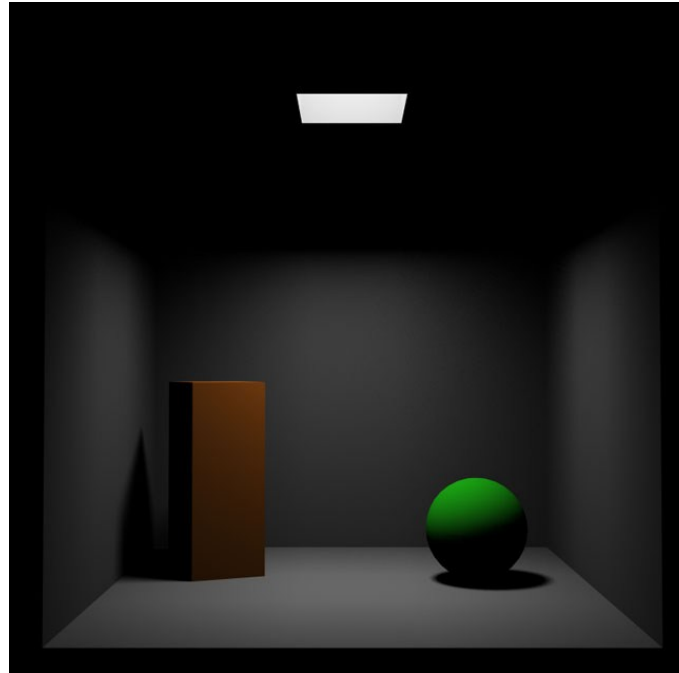


The Rendering Equation – Simplifications

$$L_o(\vec{x}, \omega_o) = E + \sum_i^N (diff + spec)L_i att_i$$

Outward radiance Emitted radiance i BRDF Inward radiance

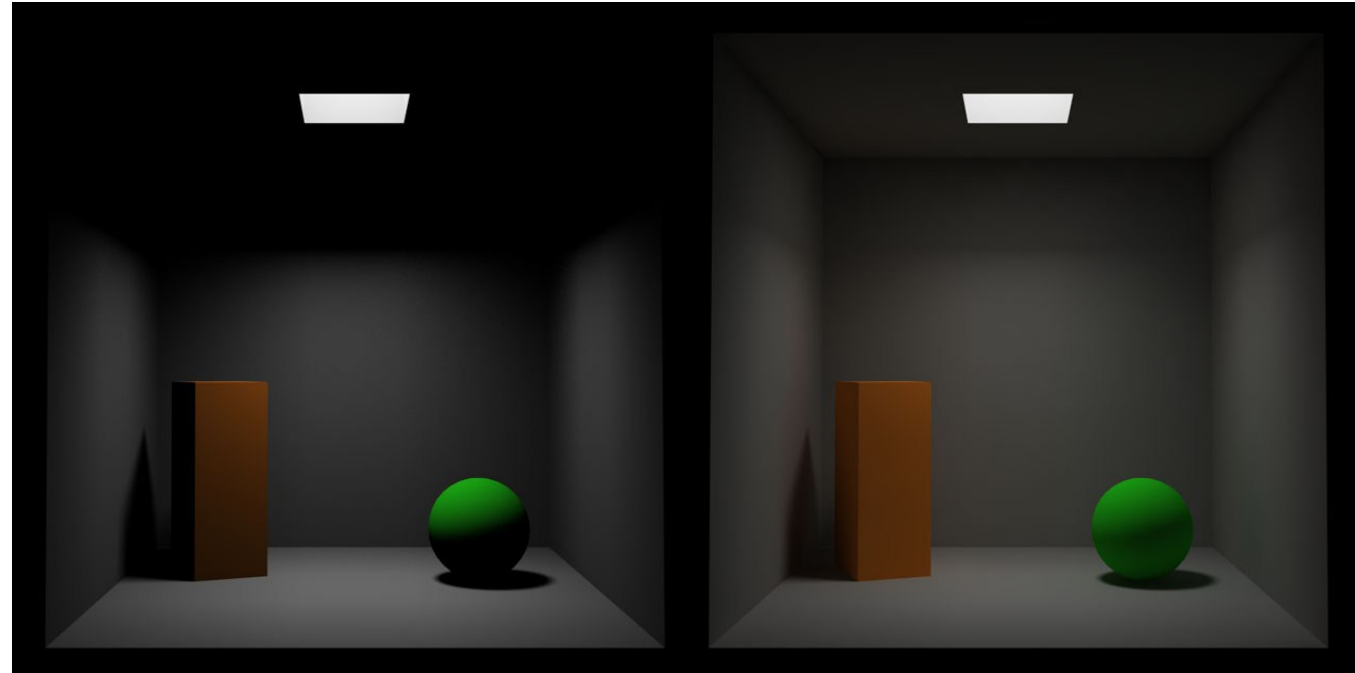
- Hum.. Almost there.. But maybe we removed too much light
- The scene is pretty dark
- What can we do ?



The Rendering Equation – Simplifications

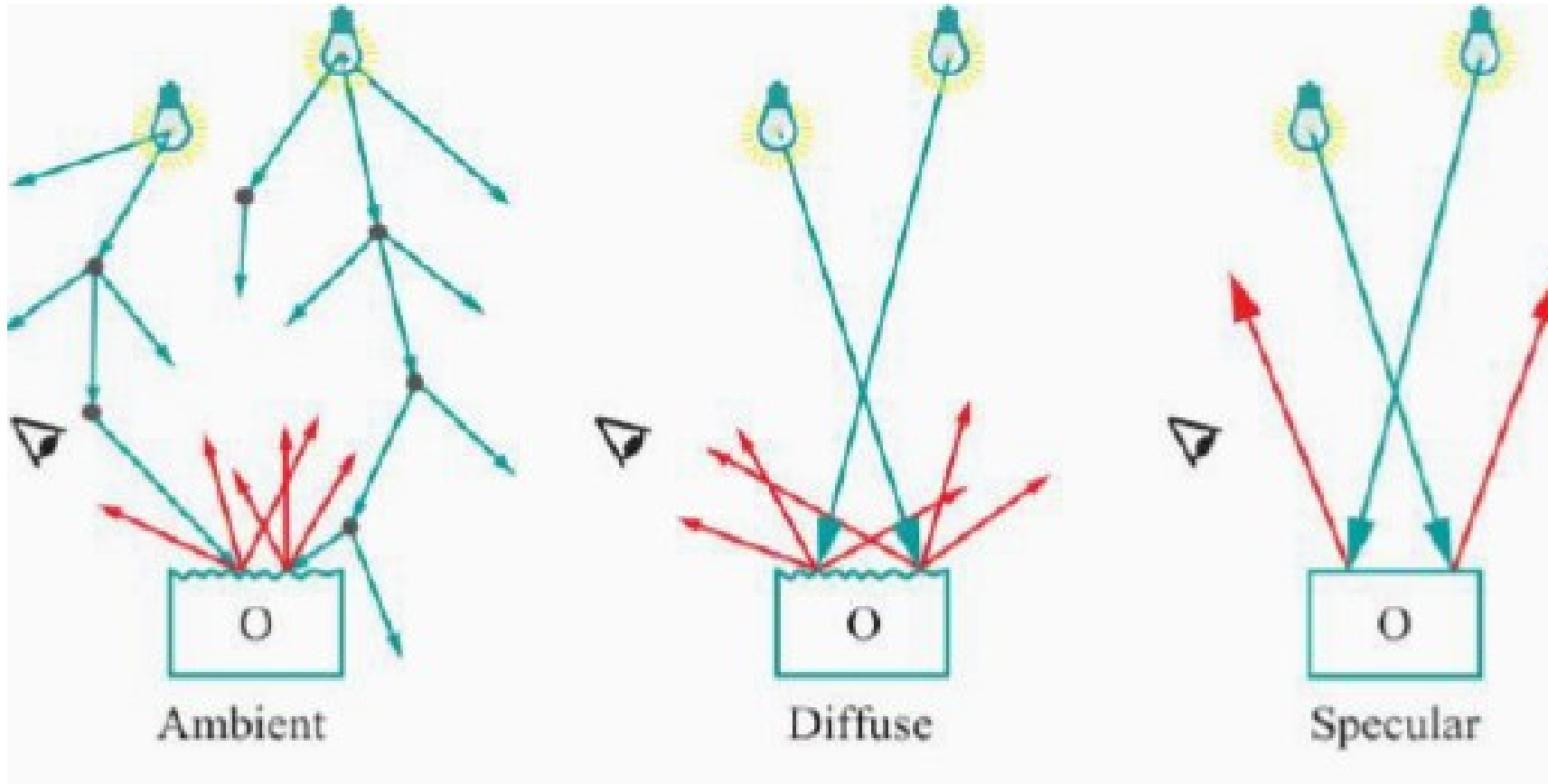
$$\underset{\text{Outward radiance}}{L_o(\vec{x}, \omega_o)} = \underset{\text{Emitted radiance}}{E} + \sum_i^N (\underset{\text{BRDF}}{diff} + \underset{\text{Inward radiance}}{spec}) L_i att_i$$

- Hum.. Almost there.. But maybe we removed too much light
- The scene is pretty dark
- Let's add some ambient light A !
- It represent all the light that bounces on other objects and that we didn't capture.



The Rendering Equation – Ambient light

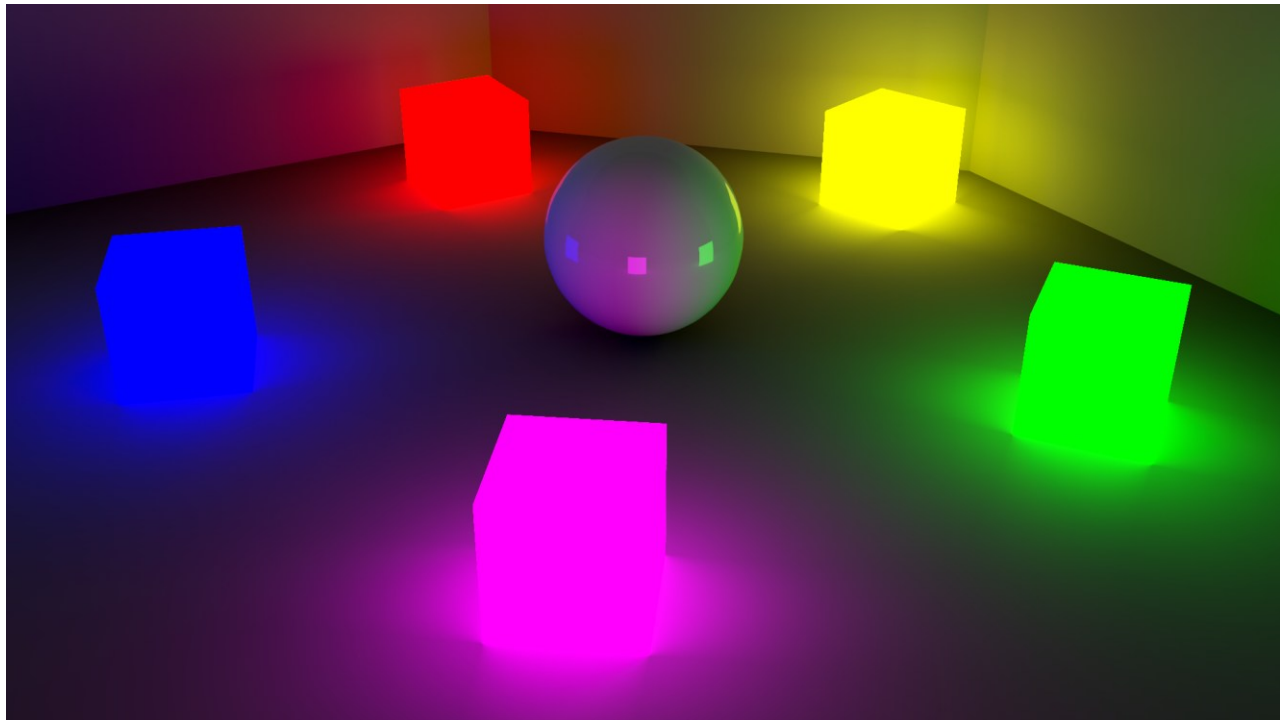
- The ambient light illustrated:



The Rendering Equation – Blinn-Phong Model

$$L_o(\vec{x}, \omega_o) = \overset{\text{Ambient light}}{A} + \overset{\text{Emitted radiance}}{E} + \sum_i^N \underbrace{(diff + spec)}_{\text{BRDF}} \underbrace{L_i att_i}_{\text{Inward radiance}}$$

Outward radiance



$L_o(\vec{x})$ = Output light color at vertex x in direction ω_o

A = Ambient light to compensate lack of luminosity

E = Emitted light from the object (can vary with time)

$diff$ = Diffuse factor

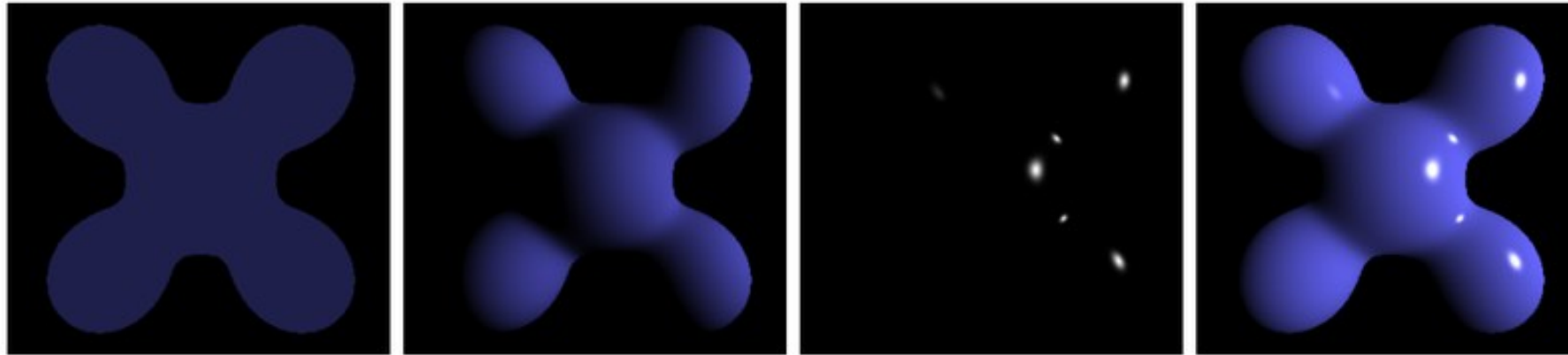
$spec$ = Specular factor

L_i = Constant color for each light t_i

att_i = Attenuation factor for the light t_i

The Rendering Equation – Components

- Illustration of the effect of each components of the BRDF



Ambient
Constant

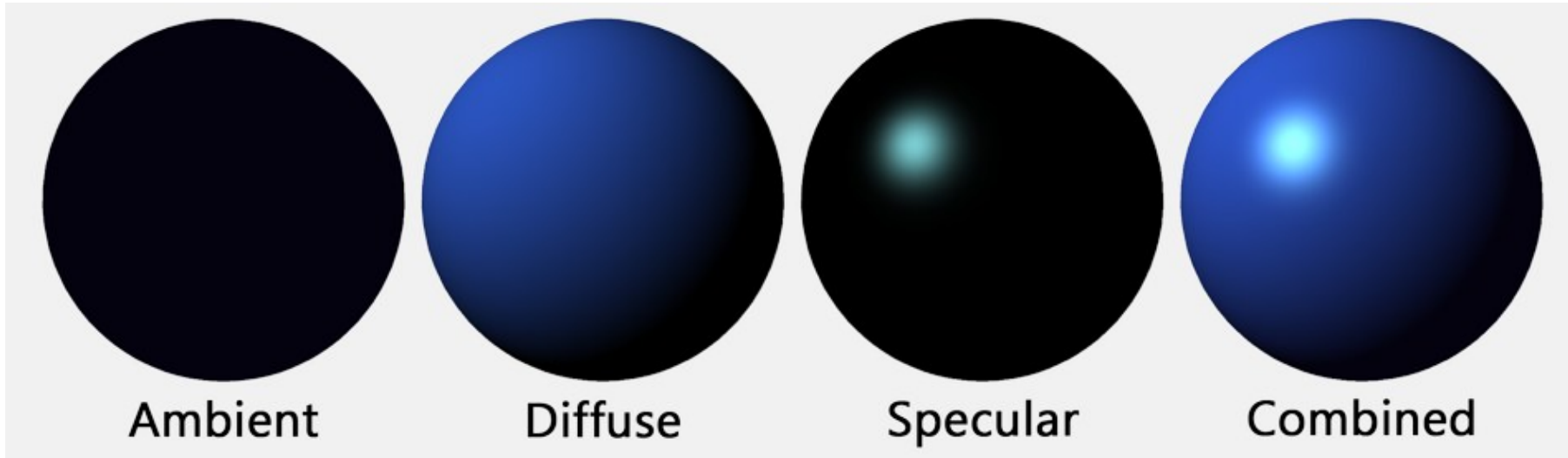
+

Diffuse
Angle
light-normal

+

Specular
Reflection
light-eye

= Phong Reflection



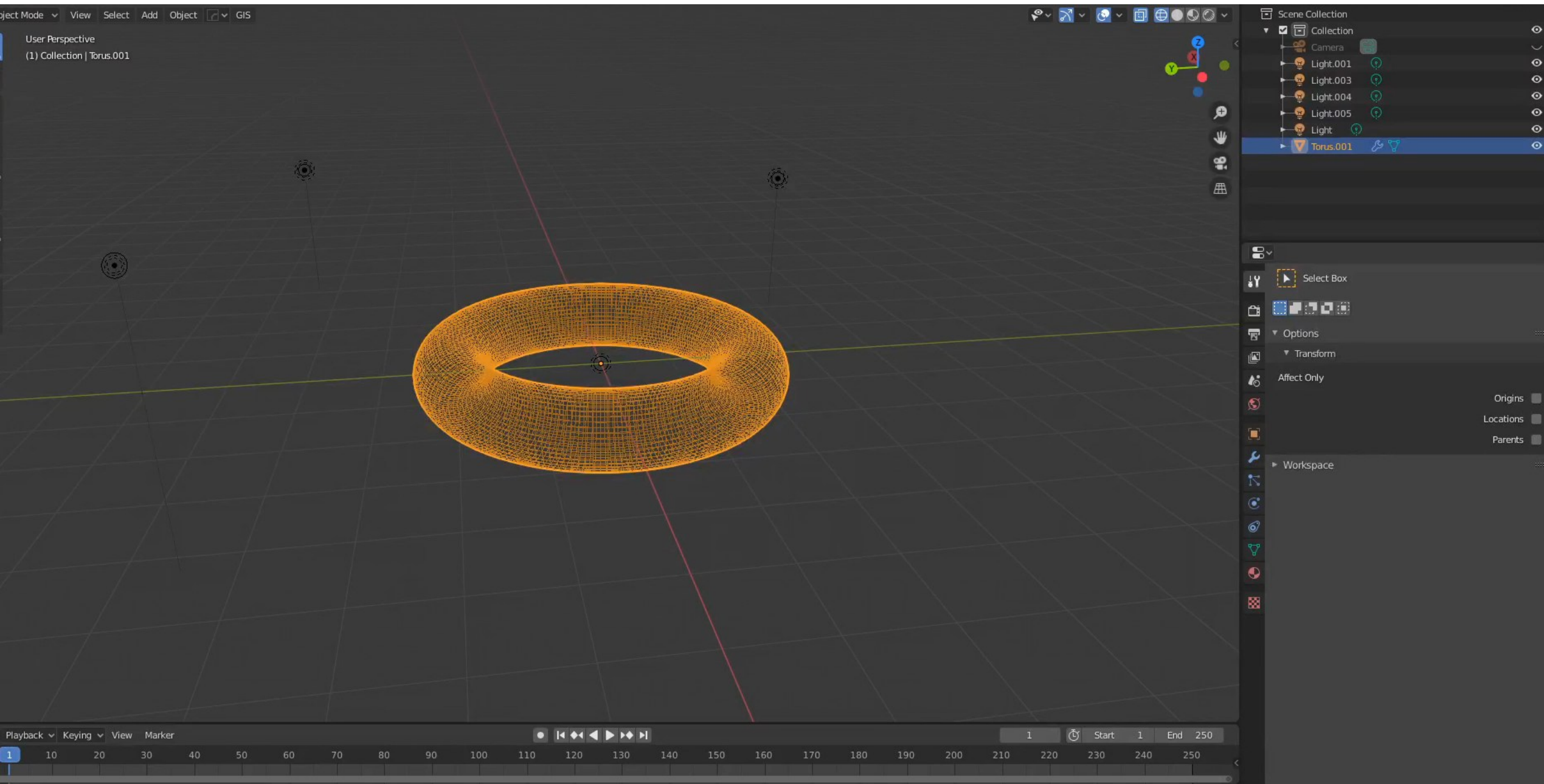
Ambient

Diffuse

Specular

Combined

The Rendering Equation – Blender Example



Where Is Light Computed ?

- Okay.. We derived the light equation and its simplifications

$$L_o(\vec{x}, \omega_o) = A + E + \sum_i^N (diff + spec)L_i att_i$$

- But.. Where is it computed exactly ?

Where Is Light Computed ?

- Okay.. We derived the light equation and its simplifications

$$L_o(\vec{x}, \omega_o) = A + E + \sum_i^N (diff + spec)L_i att_i$$

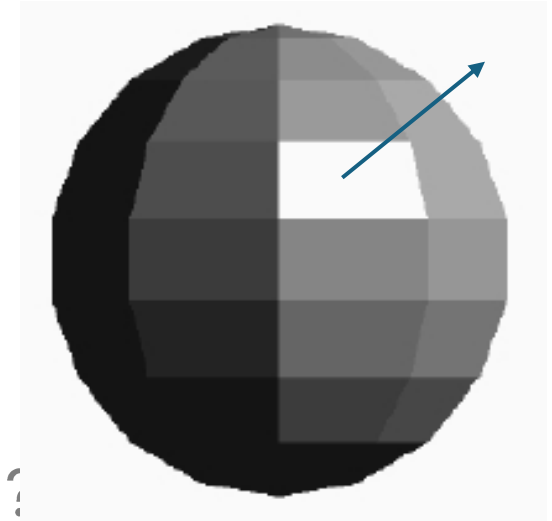
- But.. Where is it computed exactly ?
- The simplest model:
 - We know that 3 vertices form a triangle
 - This triangle has a normal (easy to find, why?)
- What if we compute the light equation using that normal ?

Where Is Light Computed ?

- Okay.. We derived the light equation and its simplifications

$$L_o(\vec{x}, \omega_o) = A + E + \sum_i^N (diff + spec)L_i att_i$$

- But.. Where is it computed exactly ?
- The simplest model:
 - We know that 3 vertices form a triangle
 - This triangle has a normal (easy to find, why?)
- What if we compute the light equation using that normal ?
- This is called: **Flat shading**
- One computation per triangle ! But not very realistic ☹️

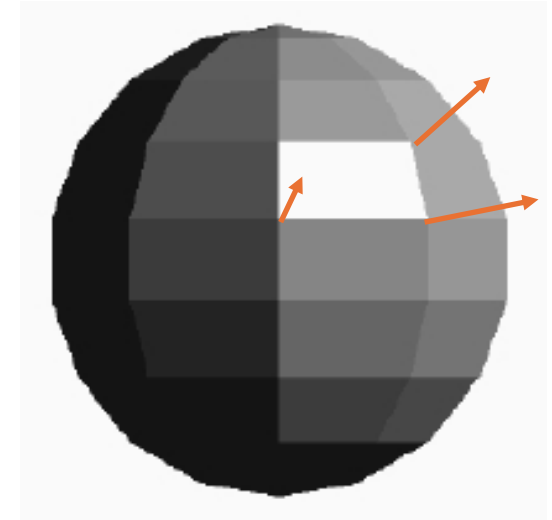


Where Is Light Computed ?

- Okay.. We derived the light equation and its simplifications

$$L_o(\vec{x}, \omega_o) = A + E + \sum_i^N (diff + spec)L_i att_i$$

- But.. Where is it computed exactly ?
- Another option is to add the normal data per vertex in our object
 - Then we have 3 normals per triangle, one on each vertex!
- What can we do with that ?
- (normal data comes from Blender)

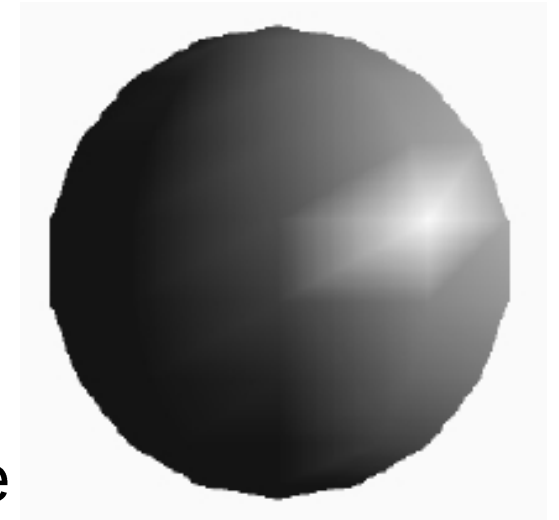


Where Is Light Computed ?

- Okay.. We derived the light equation and its simplifications

$$L_o(\vec{x}, \omega_o) = A + E + \sum_i^N (diff + spec)L_i att_i$$

- But.. Where is it computed exactly ?
- Another option is to add the normal data per vertex in our object
 - Then we have 3 normals per triangle, one on each vertex!
- We compute the light at each vertex and.. Interpolate the triangle!
- This is called: **Gouraud shading**
- 3 computations per triangle + interpolation, better but still not very good ☹️
- What can we do instead ?

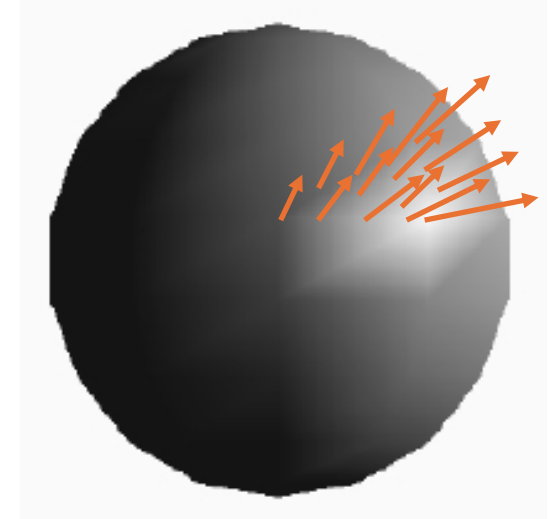


Where Is Light Computed ?

- Okay.. We derived the light equation and its simplifications

$$L_o(\vec{x}, \omega_o) = A + E + \sum_i^N (diff + spec)L_i att_i$$

- But.. Where is it computed exactly ?
- Finally, we can interpolate the **normals** on the triangle !
- Then compute the light at every fragment !

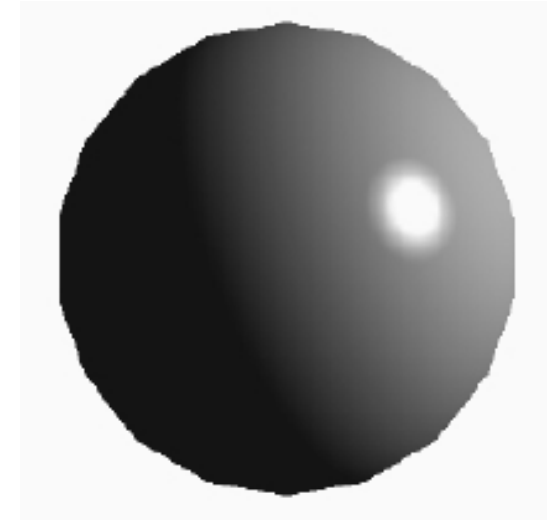


Where Is Light Computed ?

- Okay.. We derived the light equation and its simplifications

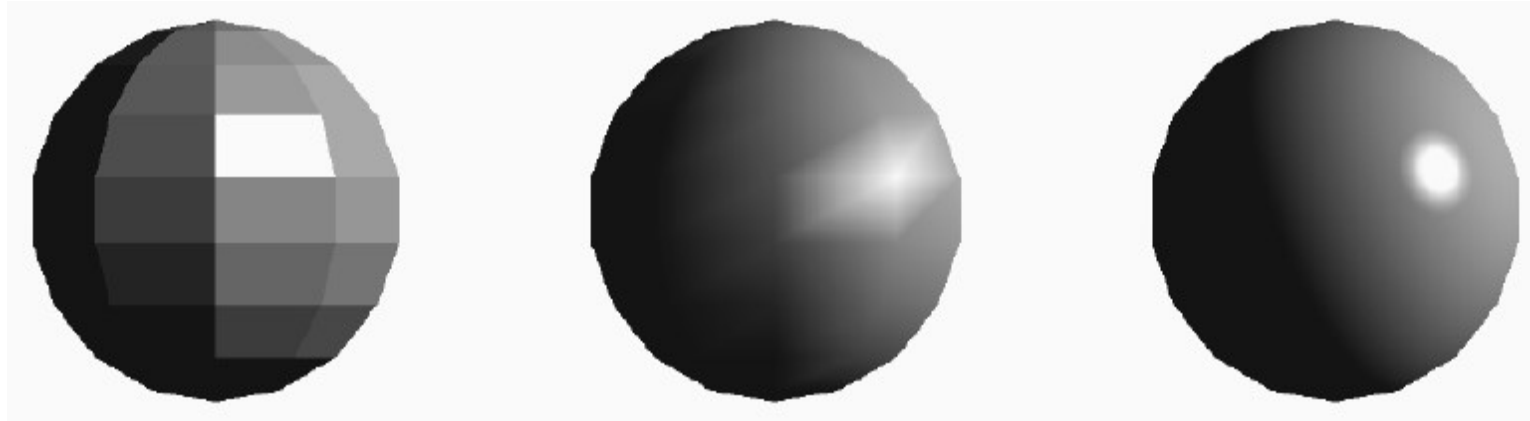
$$L_o(\vec{x}, \omega_o) = A + E + \sum_i^N (diff + spec)L_i att_i$$

- But.. Where is it computed exactly ?
- Finally, we can interpolate the **normals** on the triangle !
- Then compute the light at every fragment !



- This is called: **Phong shading**
- Interpolation of normals + number of fragments computations per triangle, Very good results 😊, but expensive !

Where Is Light Computed ?



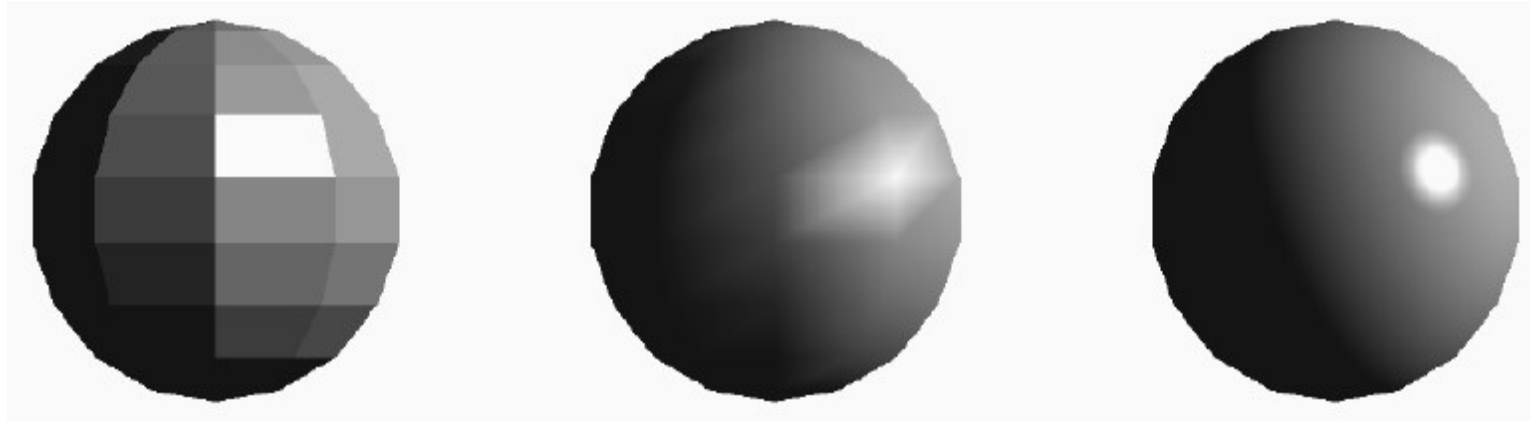
Flat

Gouraud

Phong

- **Flat** Shading
 - One normal per triangle
- **Gouraud** shading
 - One normal per vertex per triangle
 - Light computed per vertex
 - Light values interpolated in fragments
- **Phong** shading
 - One normal per vertex per triangle
 - Normals are interpolated between vertices
 - Light is computed for every fragment using its normal

Where Is Light Computed ?

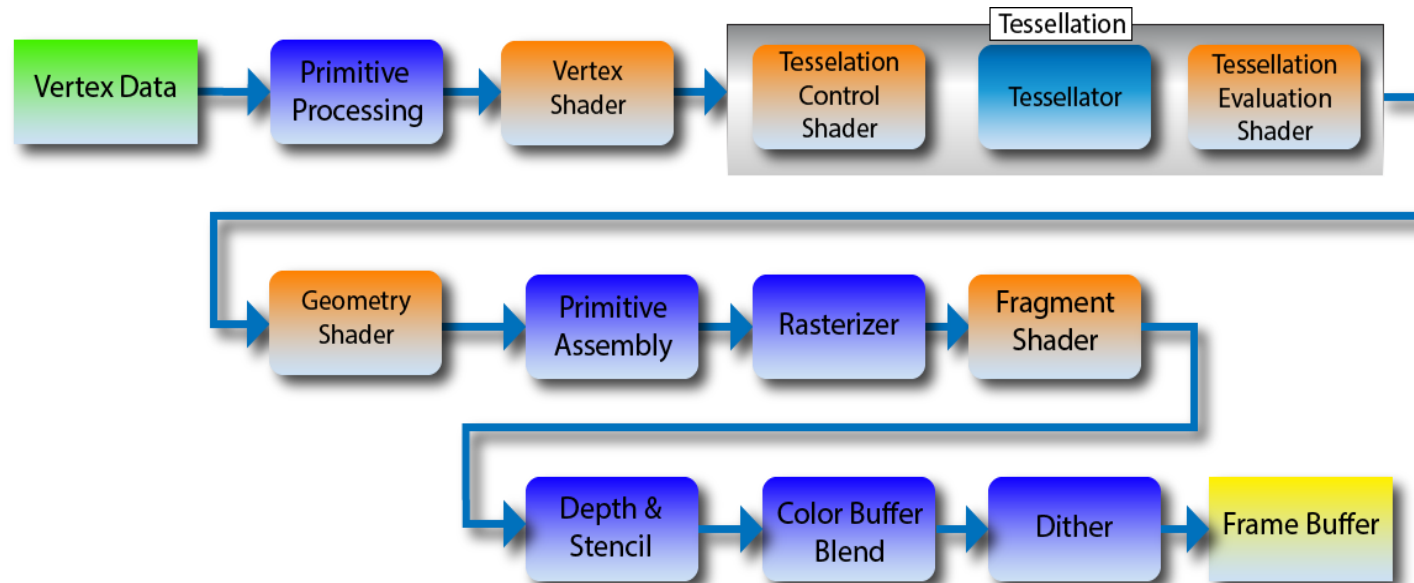


Flat

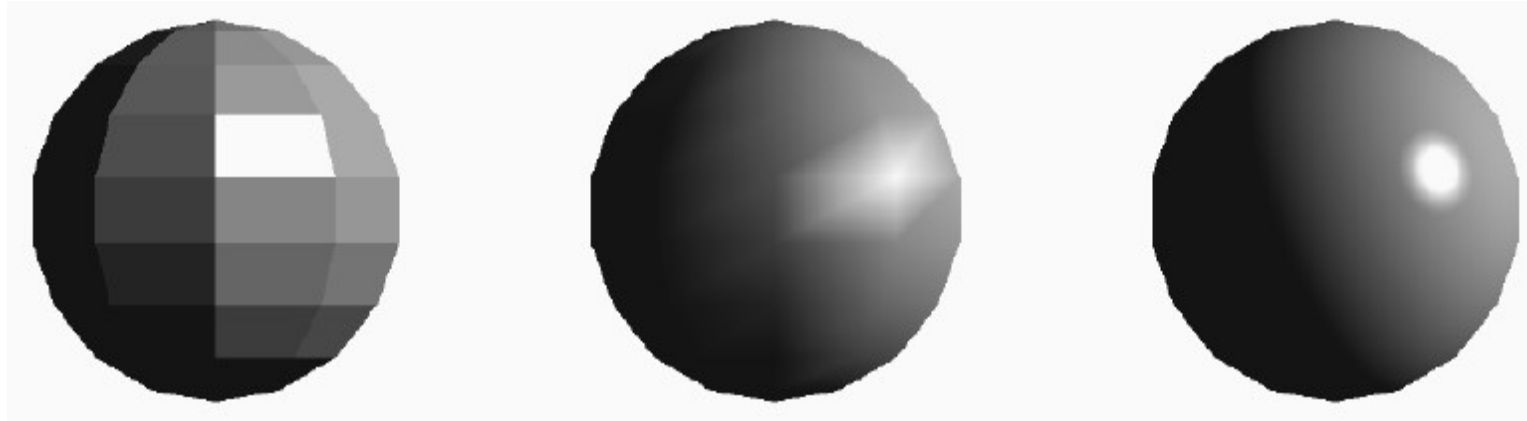
Gouraud

Phong

- Where are they implemented ?



Where Is Light Computed ?



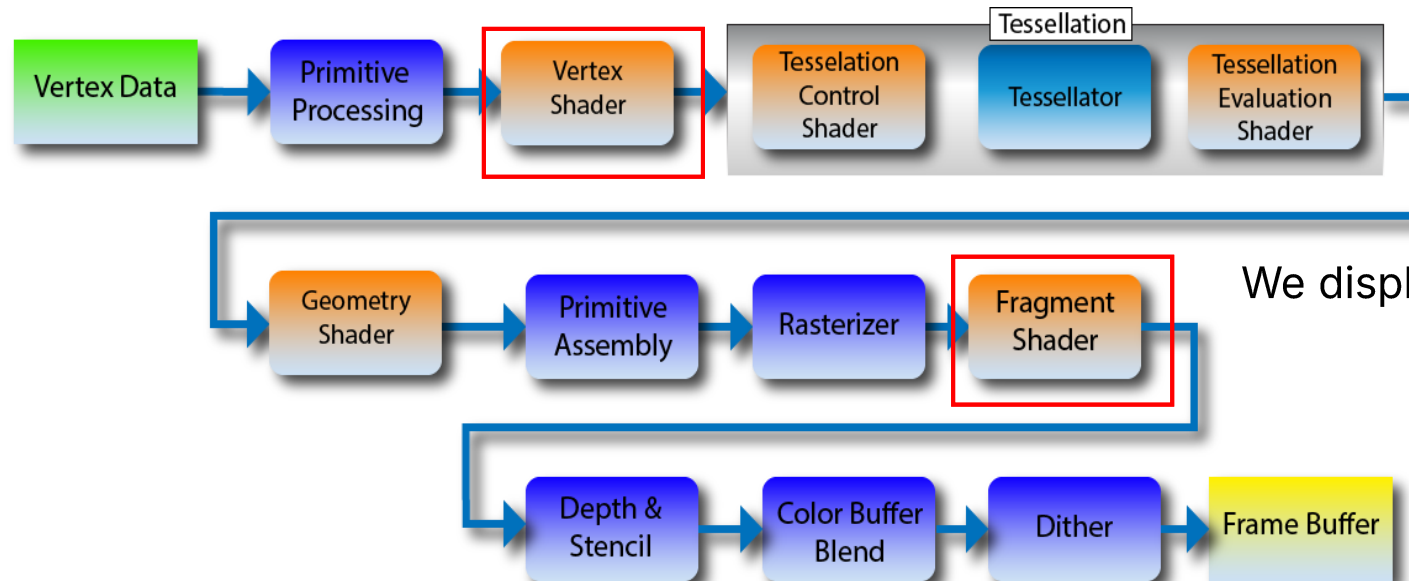
Flat

Gouraud

Phong

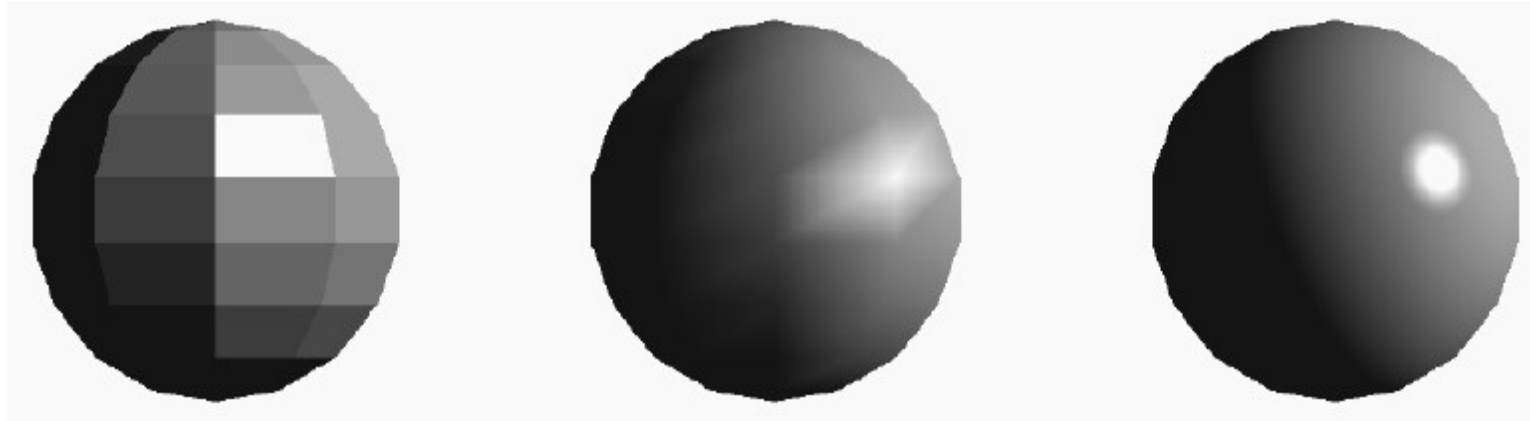
- Where are they implemented ?

We compute one light for each triangle



We display the received value

Where Is Light Computed ?



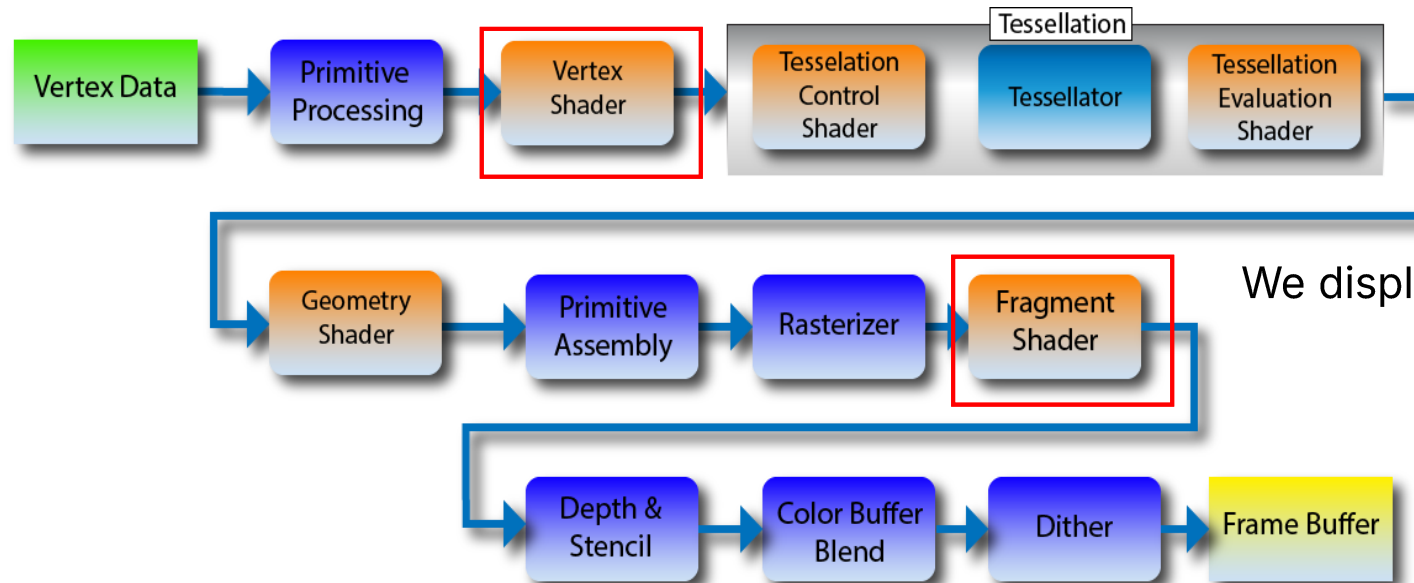
Flat

Gouraud

Phong

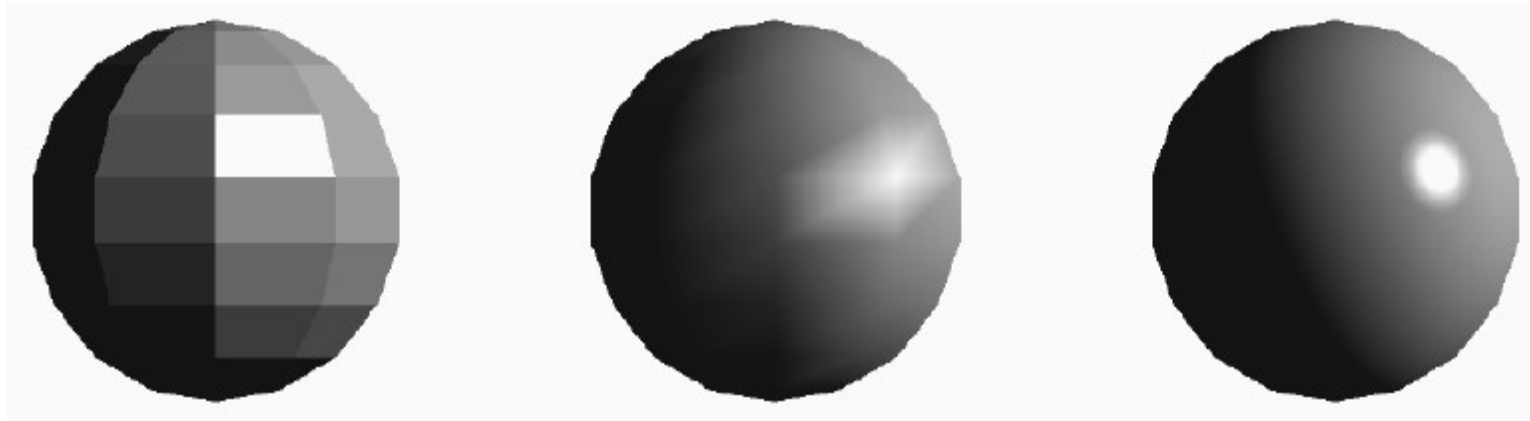
- Where are they implemented ?

We compute the light for each vertex, this value is interpolated by the pipeline



We display the received value

Where Is Light Computed ?



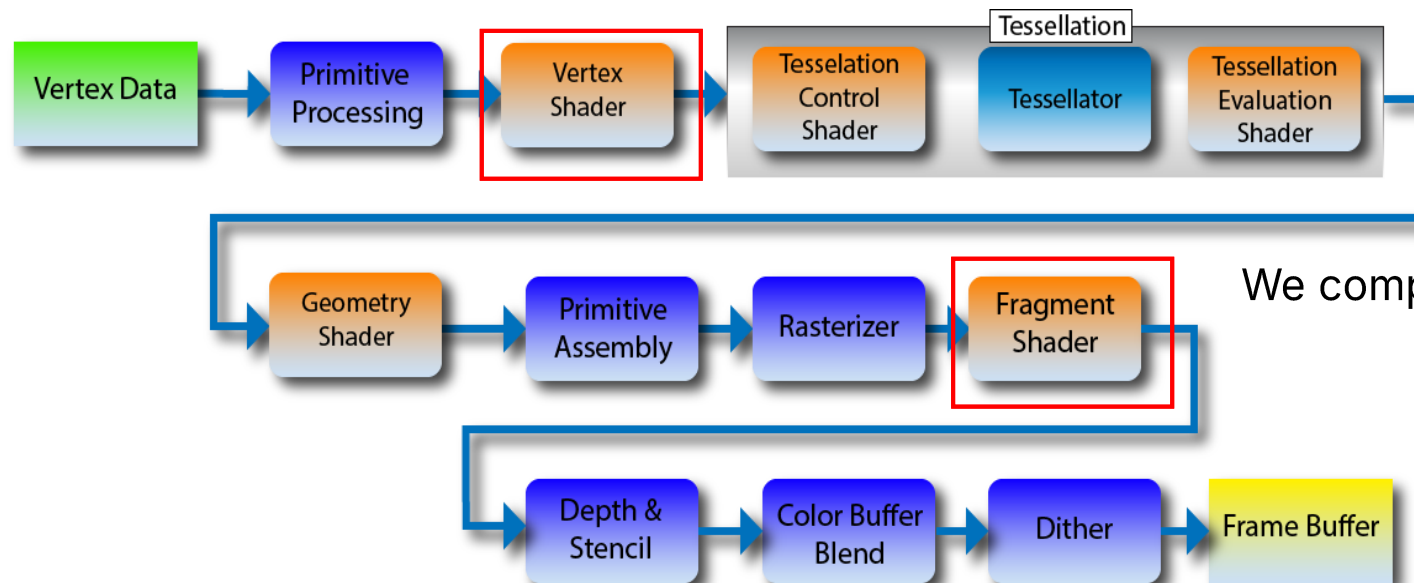
Flat

Gouraud

Phong

- Where are they implemented ?

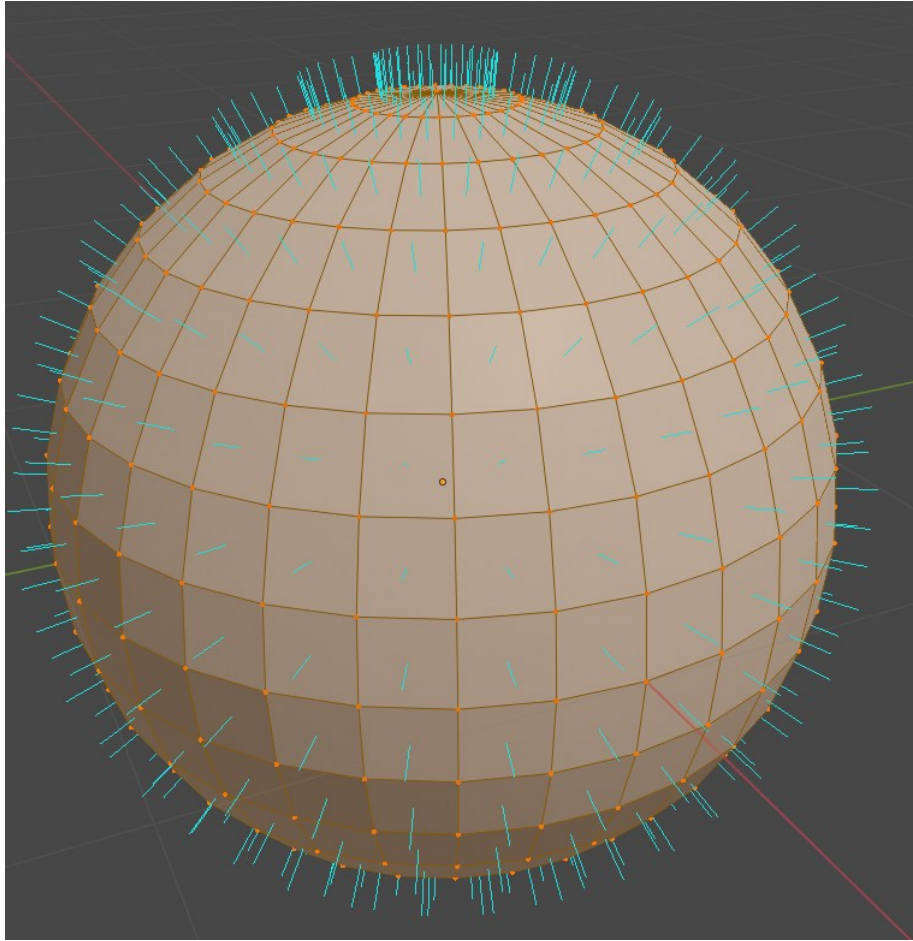
We export the normals that are interpolated by the pipeline



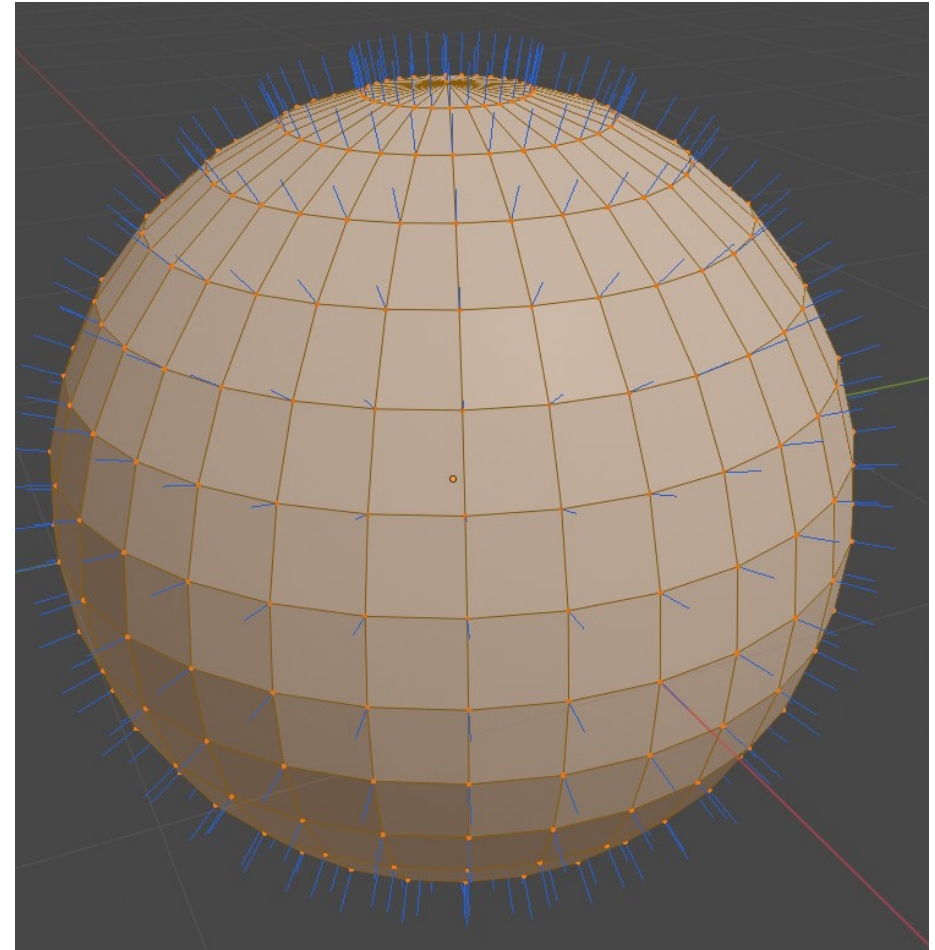
We compute the light at every normal

Where Is Light Computed ?

- Example of normals on a sphere object



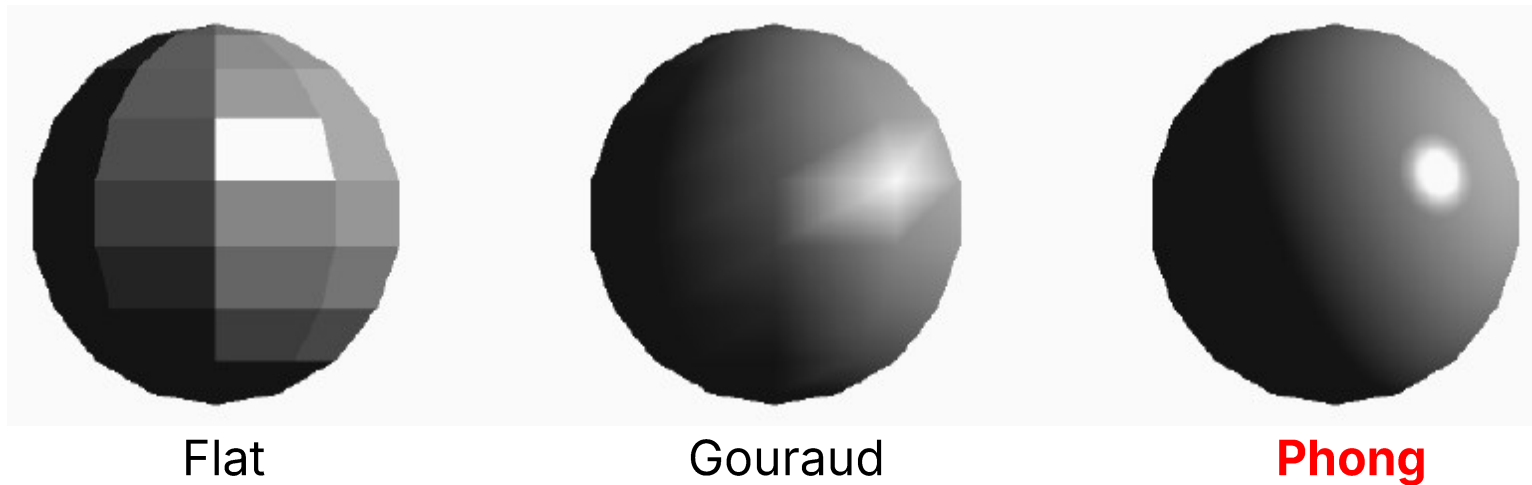
Normal per face



Normal per vertex

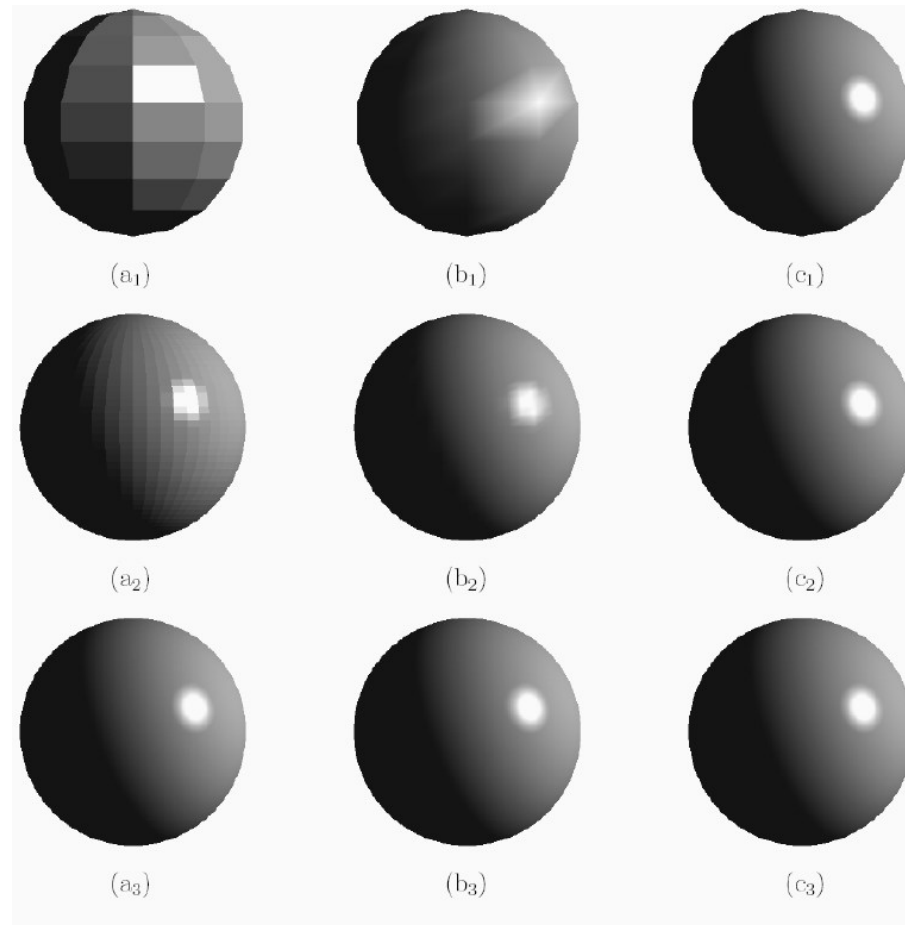
Where Is Light Computed ?

- There is one variable that may make it difficult to differentiate between the three models.
- Do you see which one ?



Where Is Light Computed ?

- There is one variable that may make it difficult to differentiate between the three models.



Flat

Gouraud

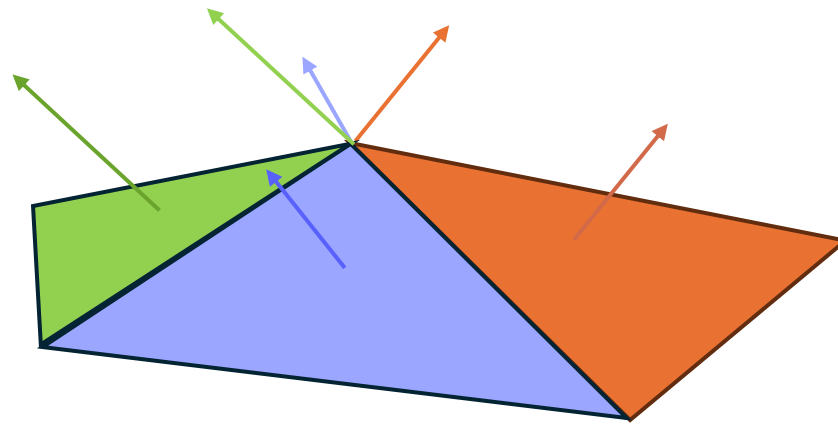
Phong

Less triangles

More triangles

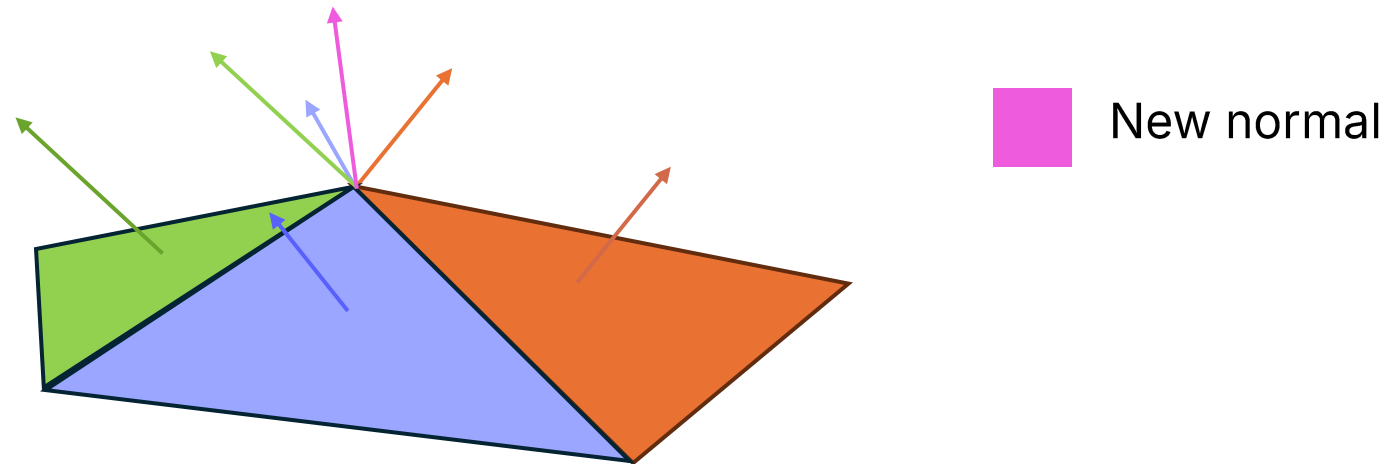
Where Is Light Computed ?

- There is one last improvement we can do with our normals.
- By default each triangle has its own set of normals
- It means that triangles that share the same vertex do not have the same normal.
- Creating a non-smooth point at the vertex
- What can we do to improve that ?



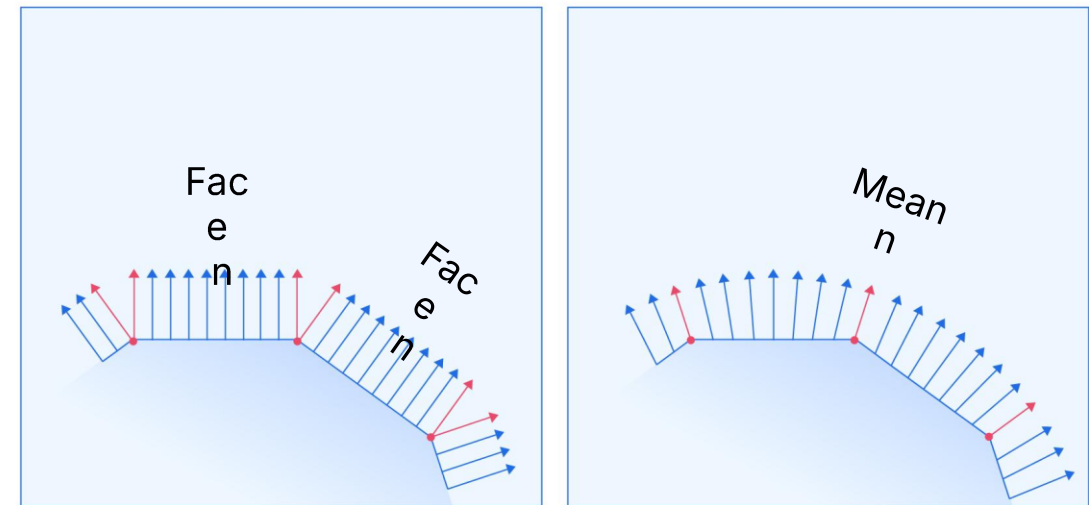
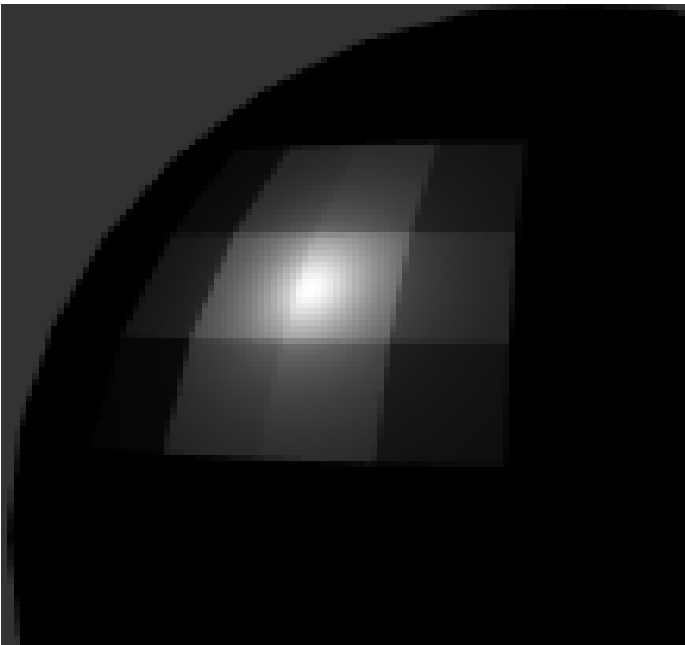
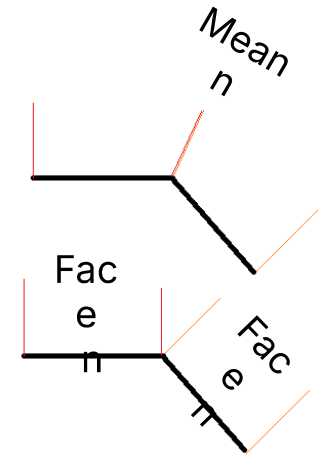
Where Is Light Computed ?

- There is one last improvement we can do with our normals.
- By default each triangle has its own set of normals
- It means that triangles that share the same vertex do not have the same normal.
- Creating a non-smooth point at the vertex
- What can we do to improve that ?
- We can average the normals between the three triangles at that vertex !
- Called **Smoothing**

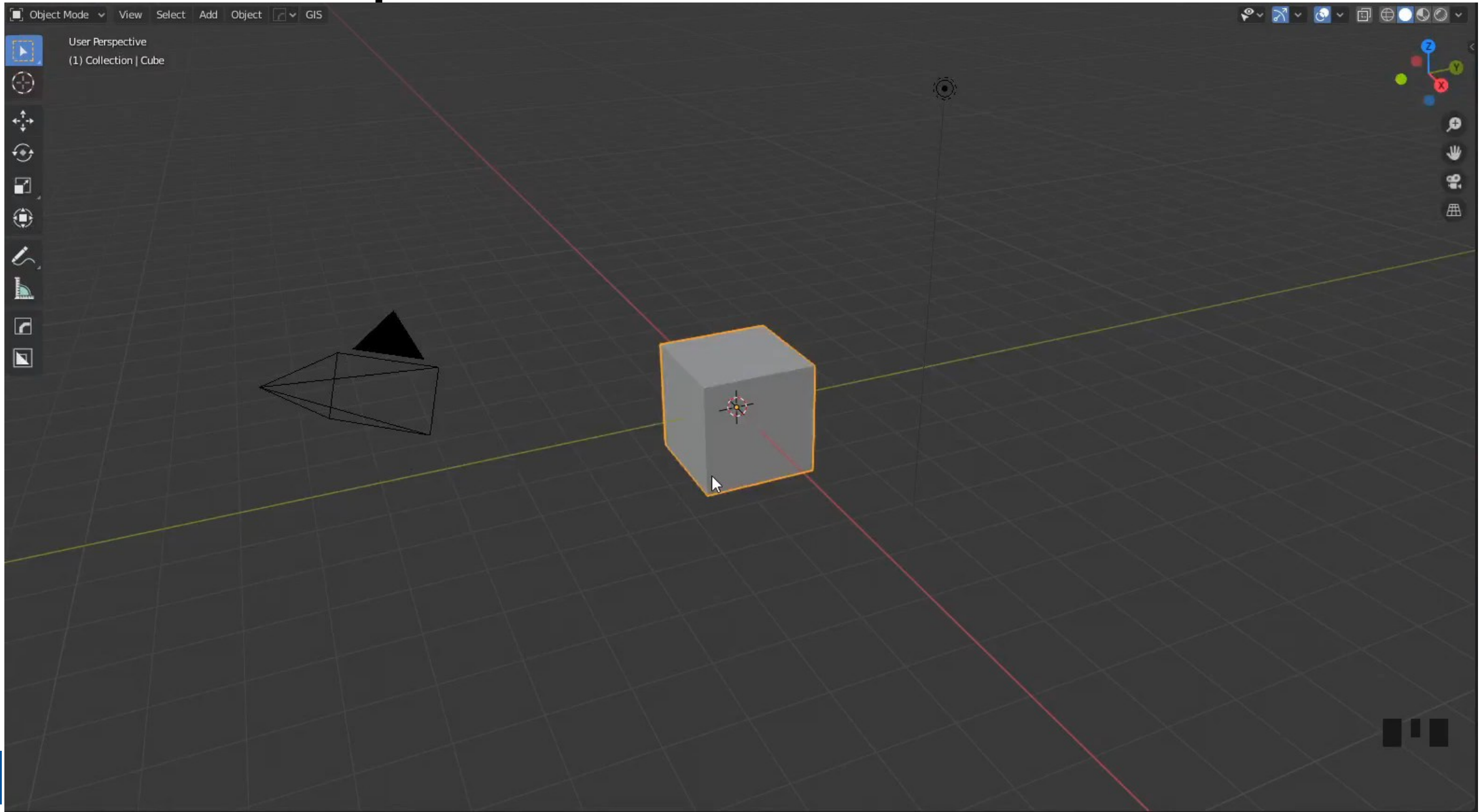


Difference Smooth and non-Smooth Objects

- Even with Phong shading, in "non smooth" sphere you see the triangles!
- This happens because the normal on the sphere are not averaged
- Each vertex uses the same normal as the face's normal



Blender – Sphere Coarse vs Smooth Normals



Can we have reflections in OpenGL ?

- Now..
- In classical games, we observe **reflections** and **refractions** on objects
- But.. We did not take into account the bouncing of other light rays from other objects

When the material is transparent

$$L_i(\vec{x}, \omega_i) = L_i$$

- Is this possible in OpenGL ?



Can we have reflections in OpenGL ?

- Now..
- In classical games, we observe **reflections** and **refractions** on objects
- But.. We did not take into account the bouncing of other light rays from other objects

When the material is transparent

$$L_i(\vec{x}, \omega_i) = L_i$$

- Is this possible in OpenGL ?
- **NO !**
- But.. How can we make it happens anyways ?



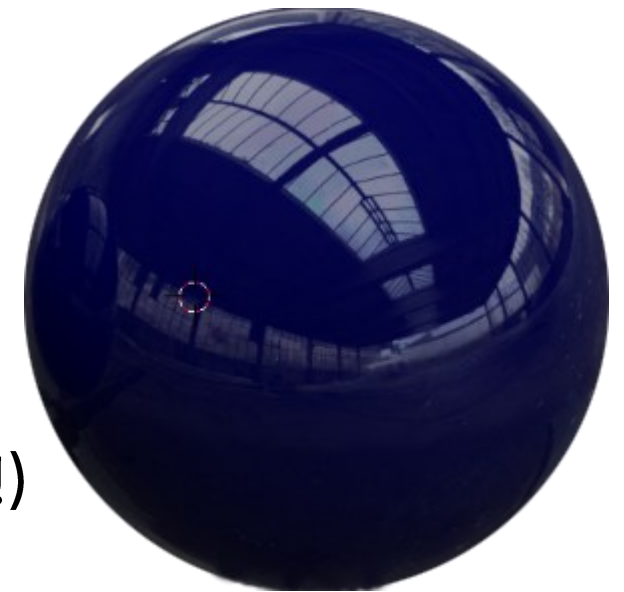
Can we have reflections in OpenGL ?

- Now..
- In classical games, we observe **reflections** and **refractions** on objects
- But.. We did not take into account the bouncing of other light rays from other objects

When the material is transparent

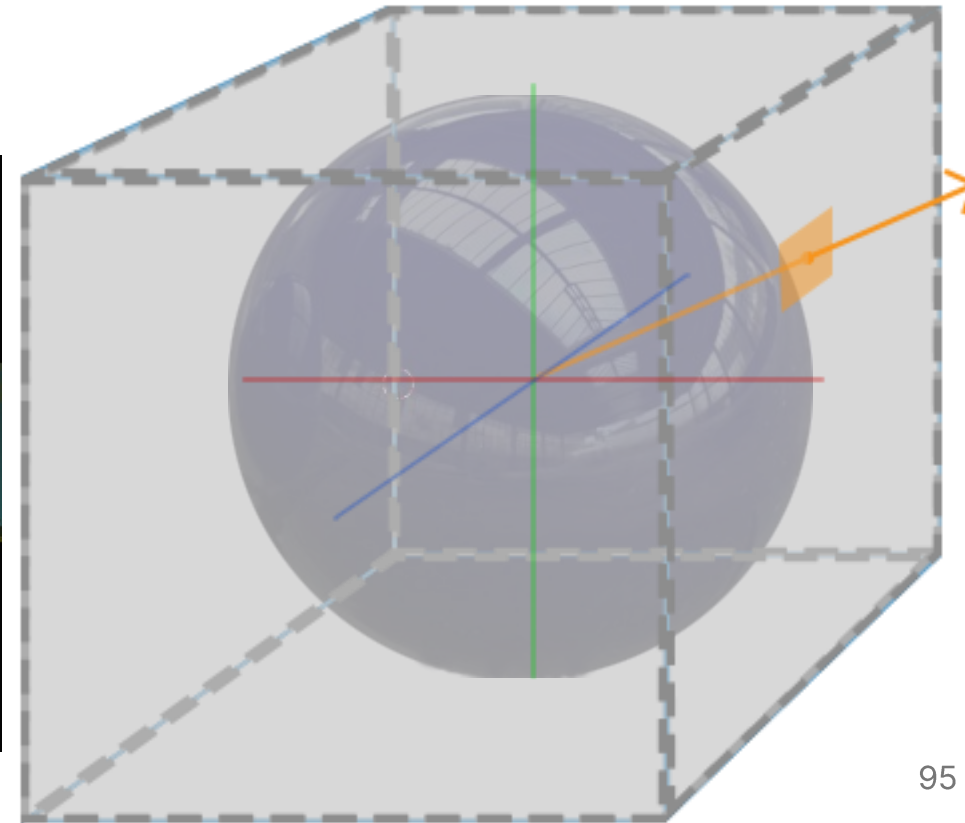
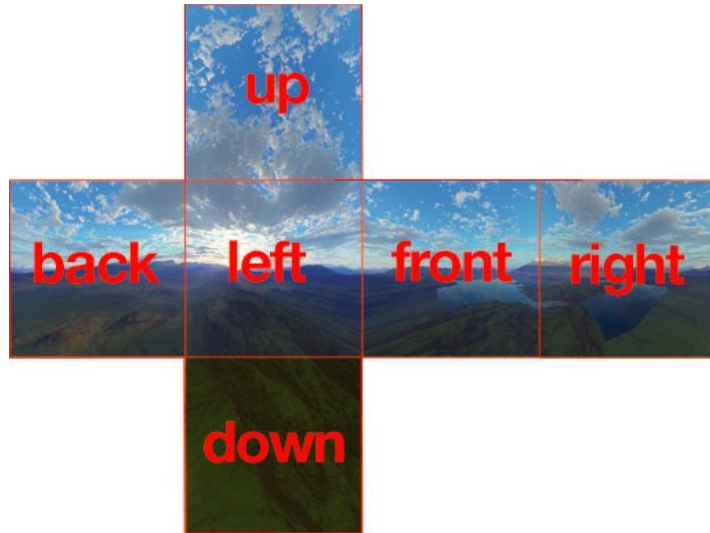
$$L_i(\vec{x}, \omega_i) = L_i$$

- Is this possible in OpenGL ?
- **NO !**
- But.. How can we make it happens anyways ?
- We cheat :O (after all most of computer graphics is cheating, to make a picture look nice!)



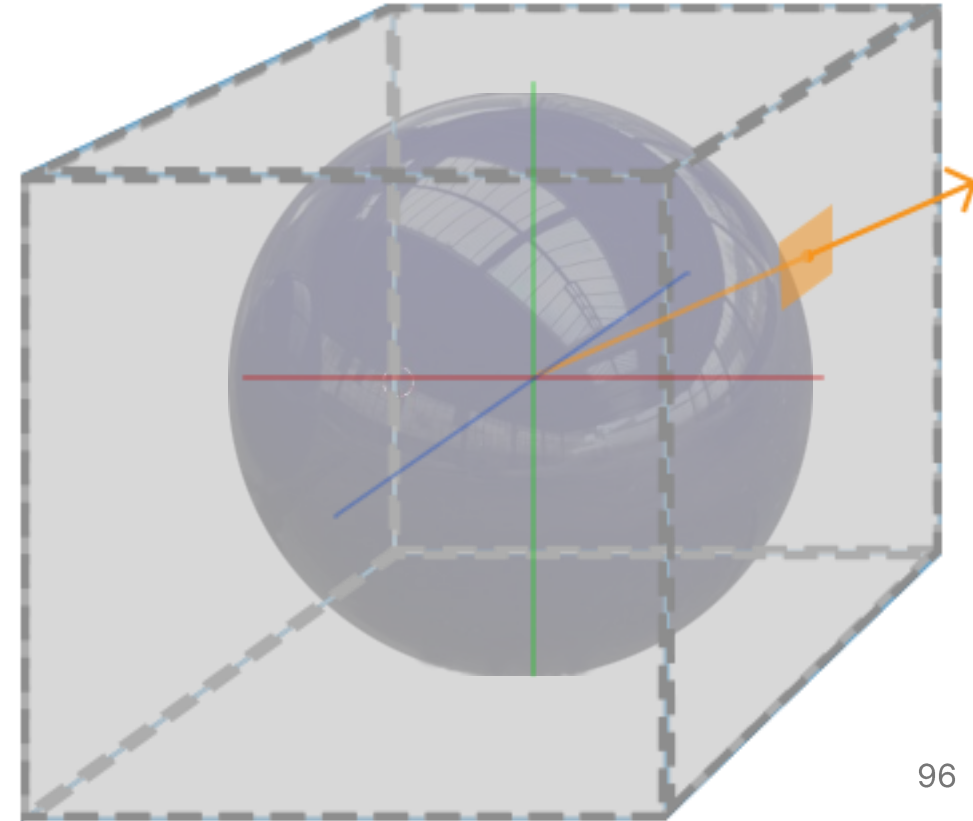
Can we have reflections in OpenGL ?

- To make **reflections** and **refractions** possible in OpenGL without computing recursively all the lights
- We englobe our scene in a cube.
- This cube will have a texture (image) called a **cubemap** ! (or environment map)
- This allows us to fetch the colors in the texture and display them on the objects !



Can we have reflections in OpenGL ?

- Let's study **reflections** with a cubemap.
- How can we do them ?
 - We want to reflect the environment map on our object

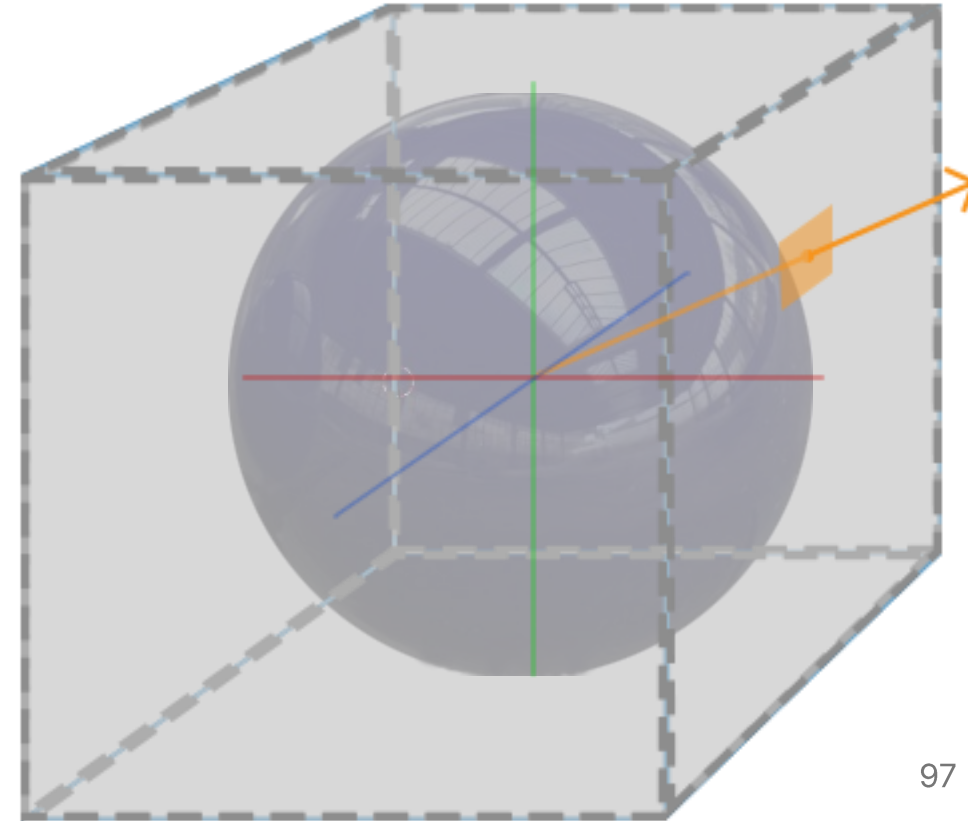


Can we have reflections in OpenGL ?

- Let's study **reflections** with a cubemap.
- Similarly to the specular light, for each vertex we can compute the reflection of the eye vector v

$$R = 2(L \cdot n)n - V$$

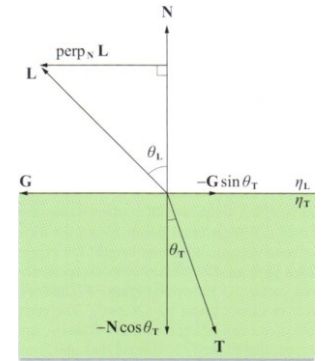
- This vector is then used to sample the cubemap !



Can we have reflections in OpenGL ?

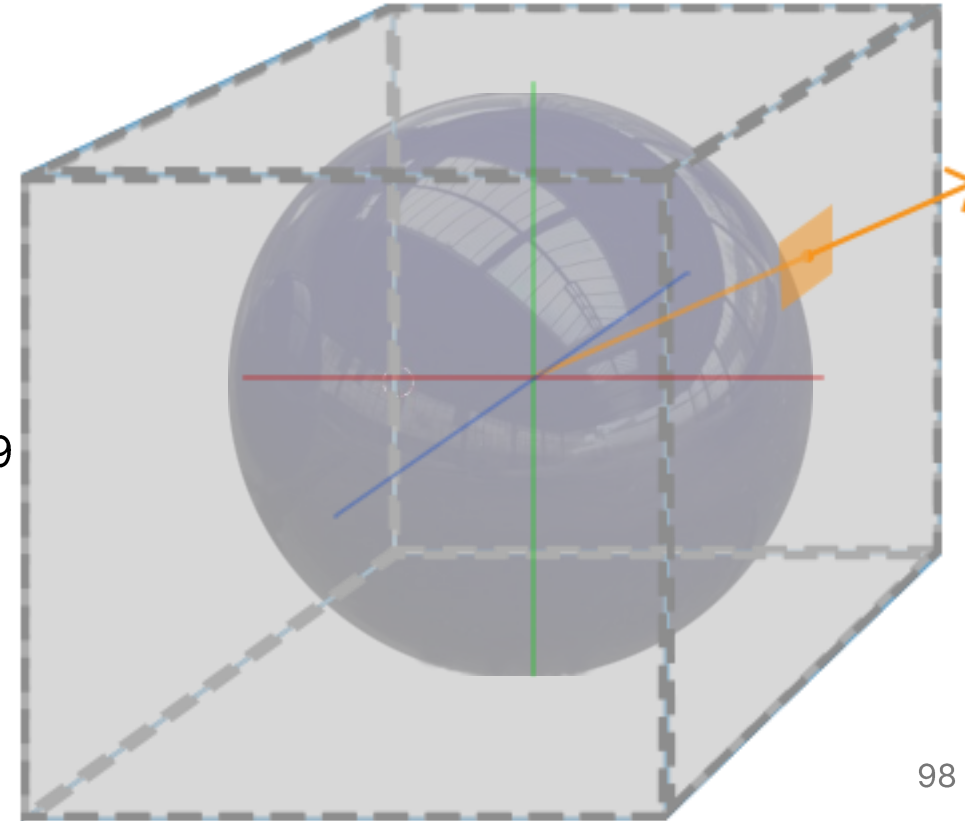
- Let's study **refraction** with a cubemap.
- Refraction has a similar formula, slightly more complicated

$$T = \left(\frac{\eta_L}{\eta_T} N \cdot L - \sqrt{1 - \frac{\eta_L^2}{\eta_T^2} [1 - (N \cdot L)^2]} \right) N - \frac{\eta_L}{\eta_T} L$$



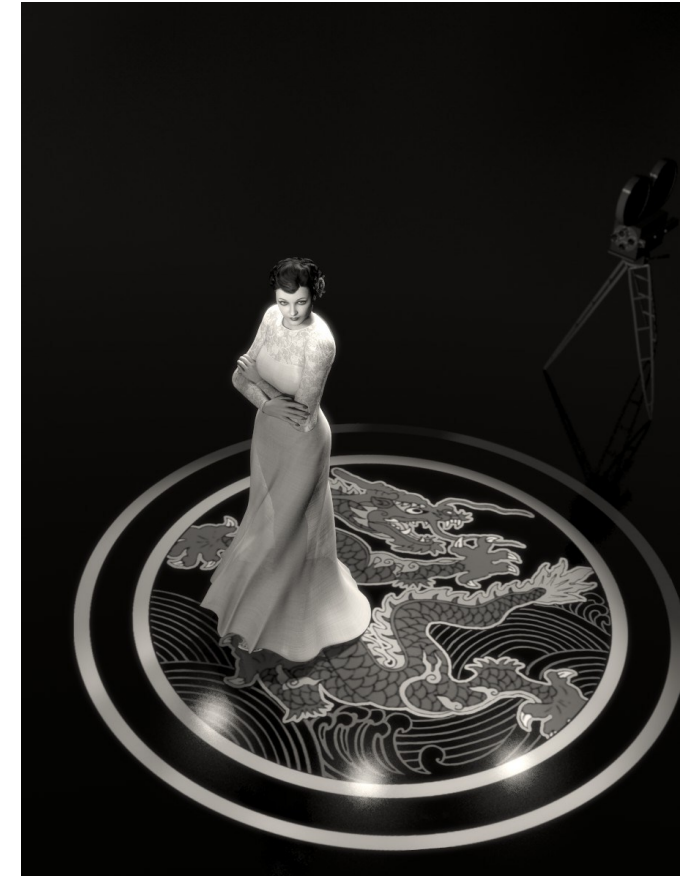
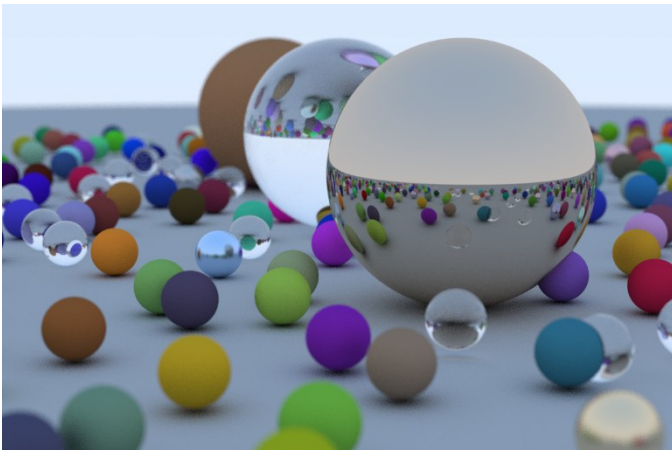
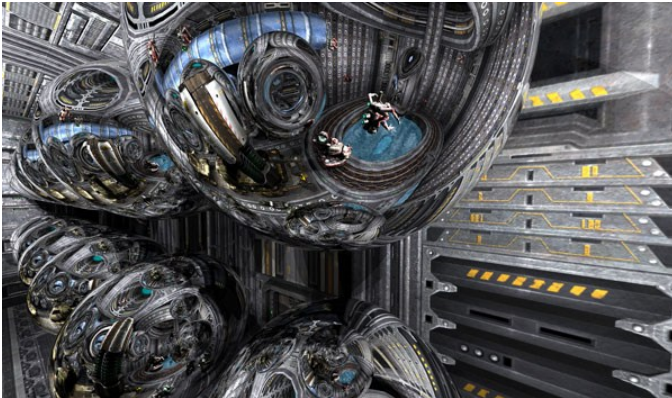
Some η_T :

Air	1.00
Water	1.33
Ice	1.309
Glass	1.52
Diamond	2.42

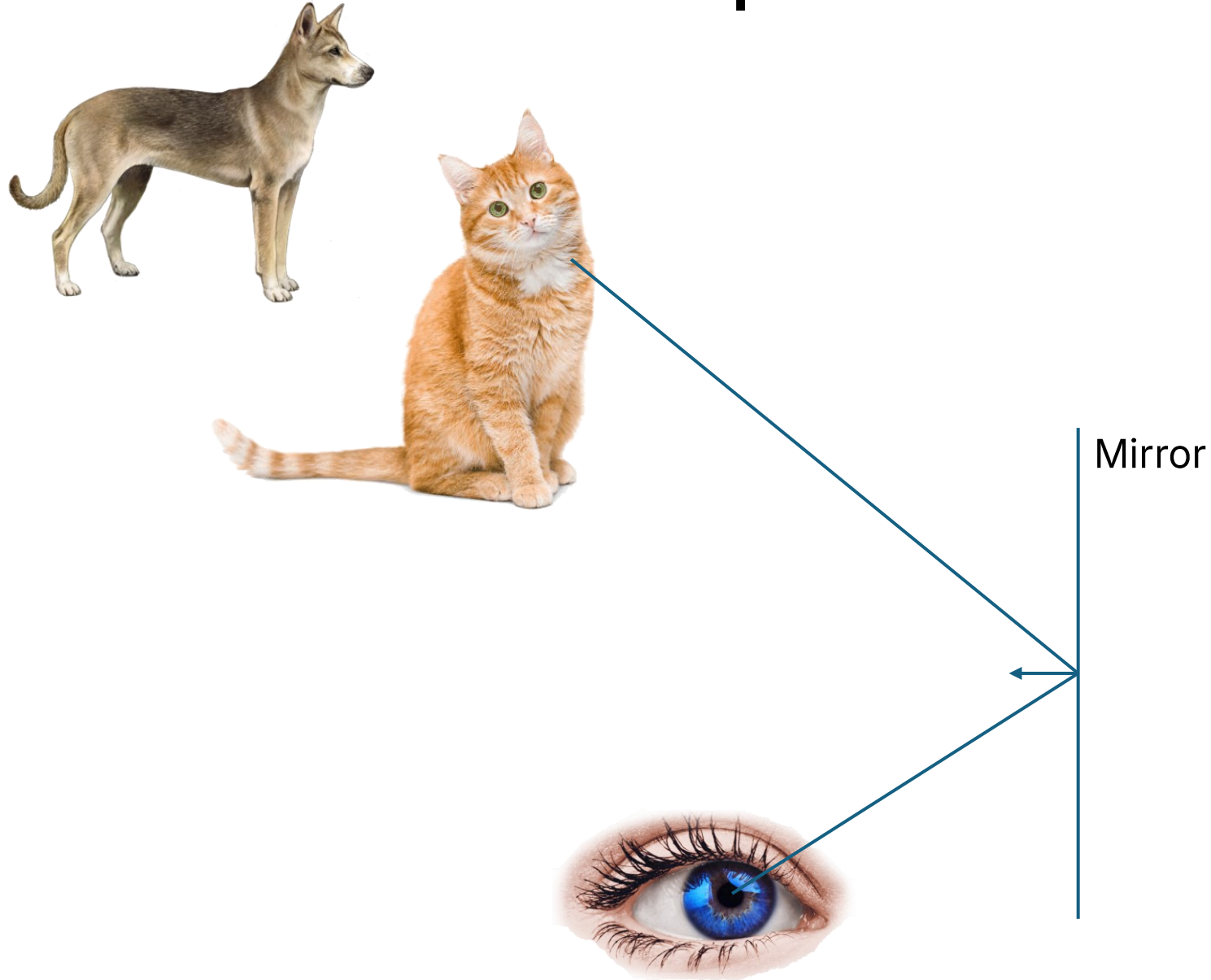


Raytracing

- Raytracing pipeline is an alternative to the OpenGL pipeline
- It leads to incredibly realistic results, however it is very expensive: <http://hof.povray.org/>
- Let's start with the problems in the current OpenGL Pipeline with reflective objects.

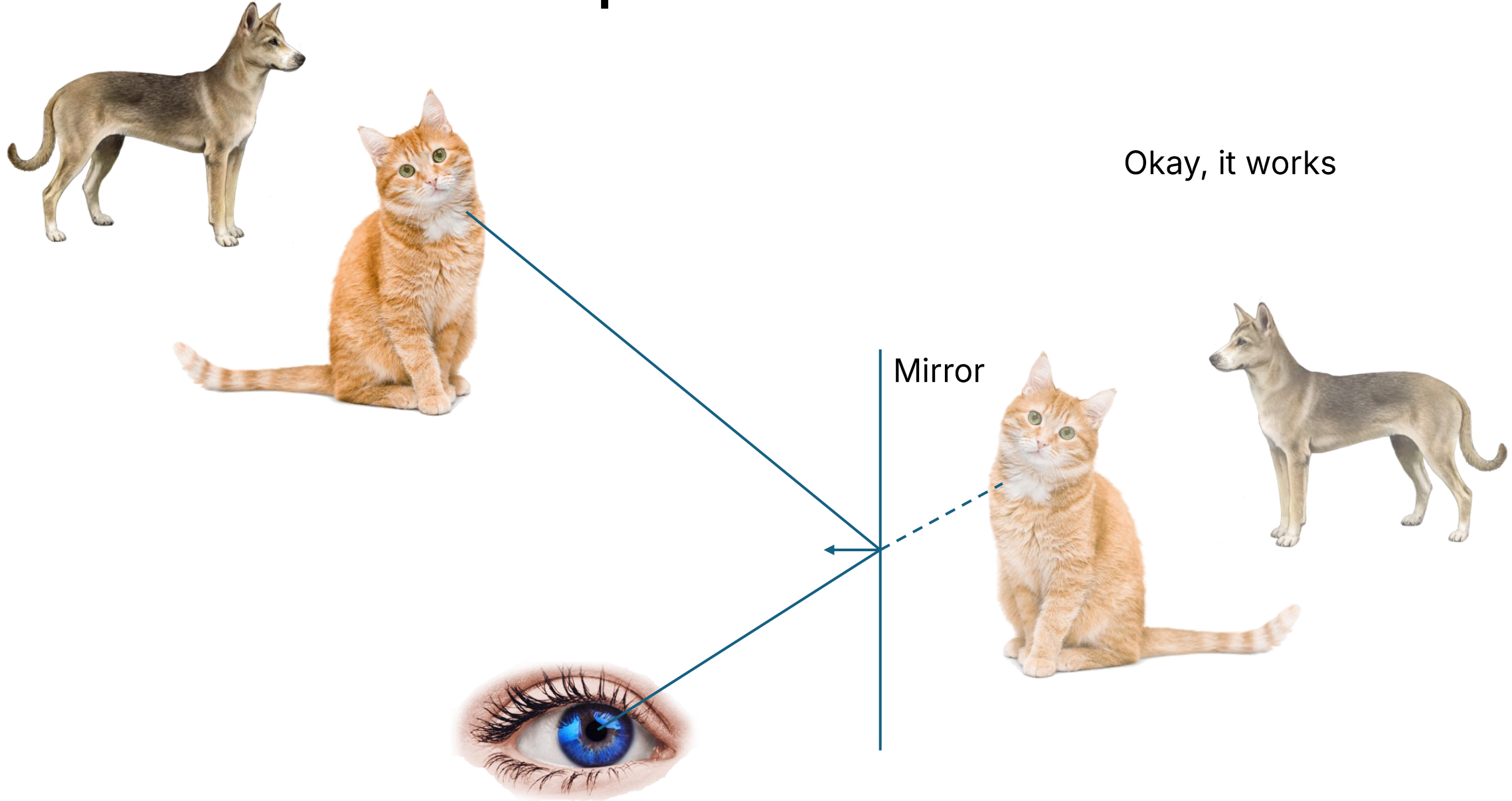


Reflections Multiple Mirrors



What do we see in the screen?

Reflections Multiple Mirrors



Okay, it works

Mirror

Reflections Multiple Mirrors



Mirror



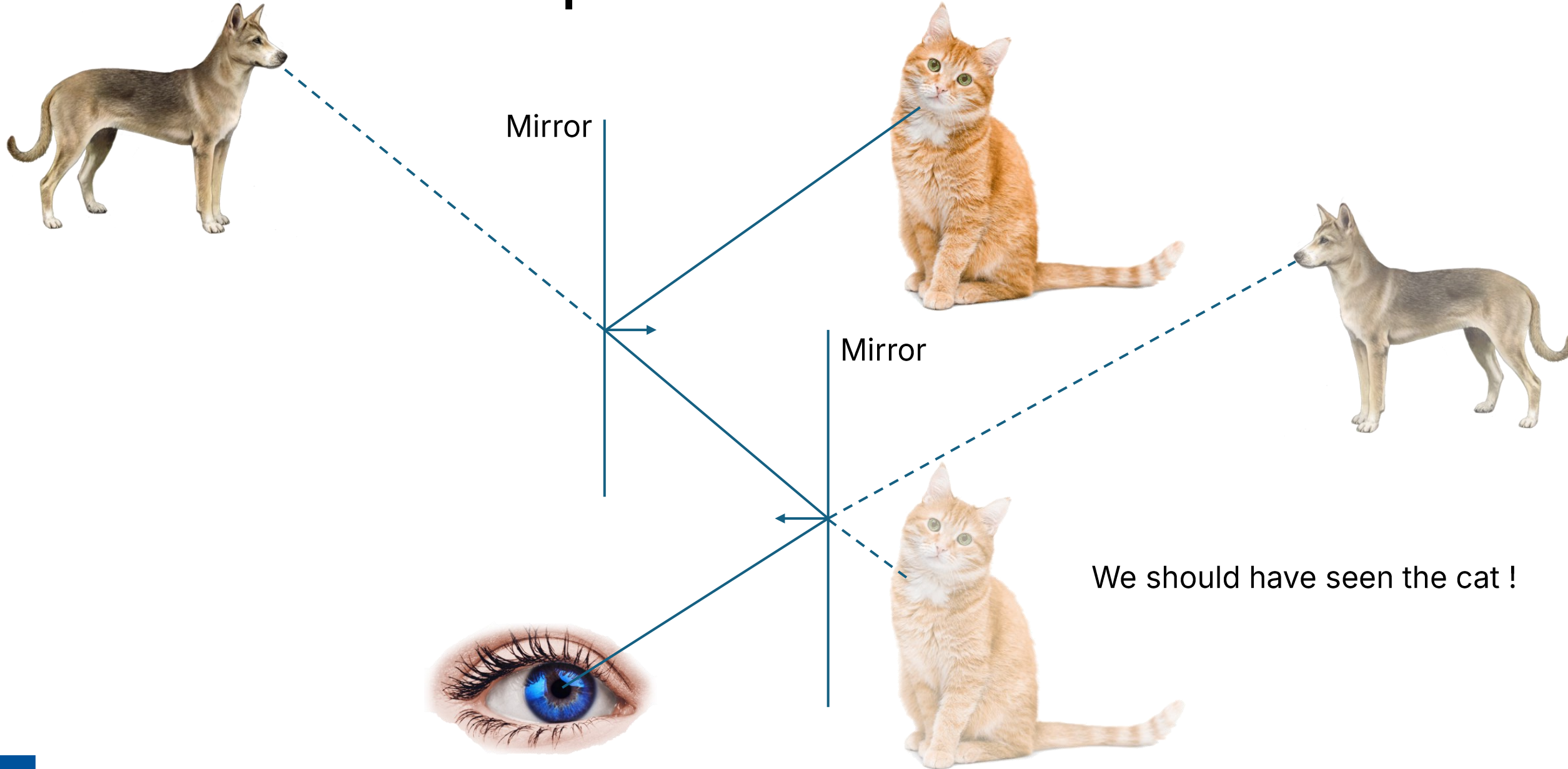
Mirror



What do we see in the screen?



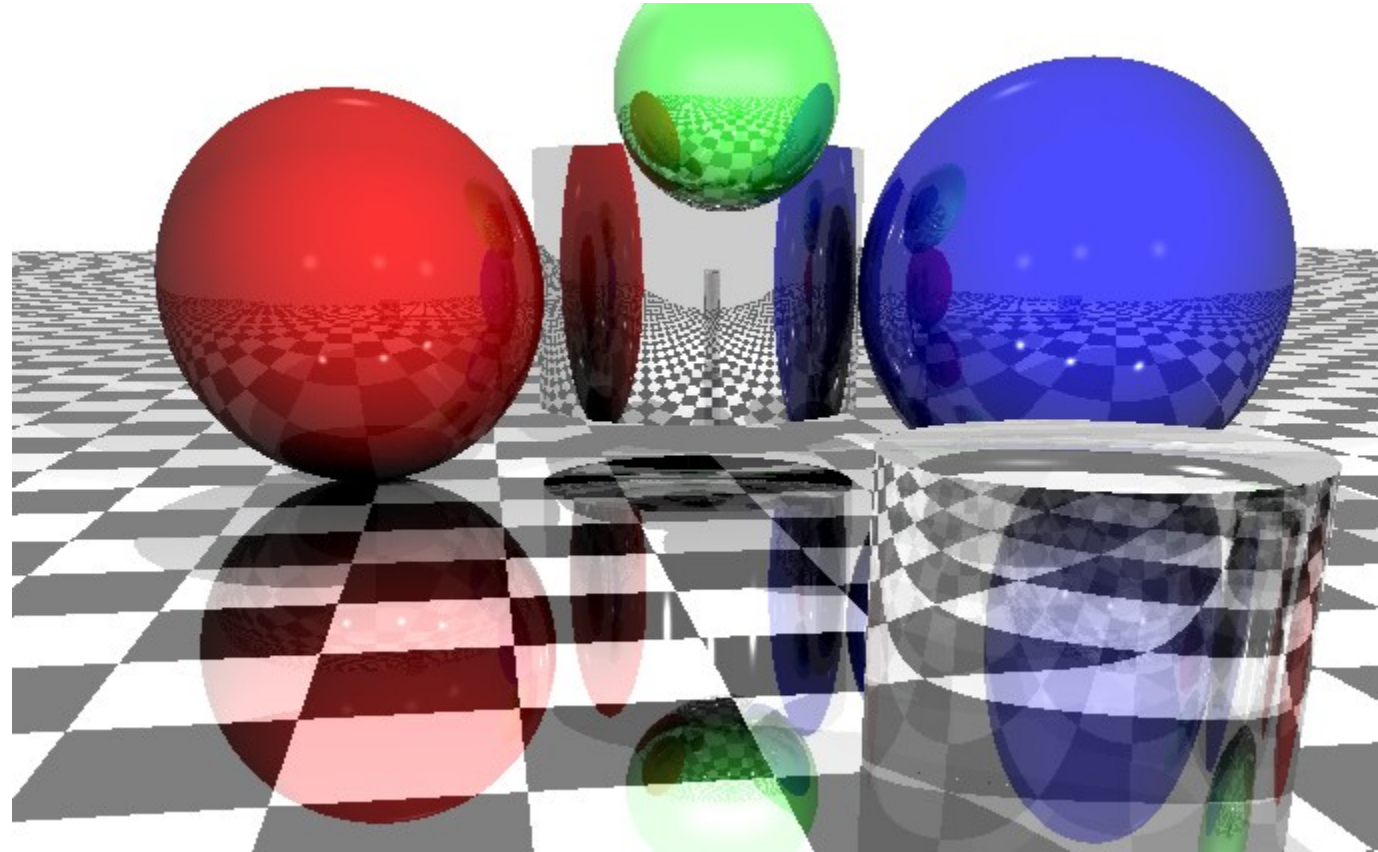
Reflections Multiple Mirrors



We should have seen the cat !

Reflections Multiple Mirrors

- We can only reflect one object.
 - Multiple reflections are not allowed (no recursion).
 - Same for refractions.
-
- Then.. How can this be possible?
 - Its certainly generated by computer graphics..
 - So.. Somehow it must be possible

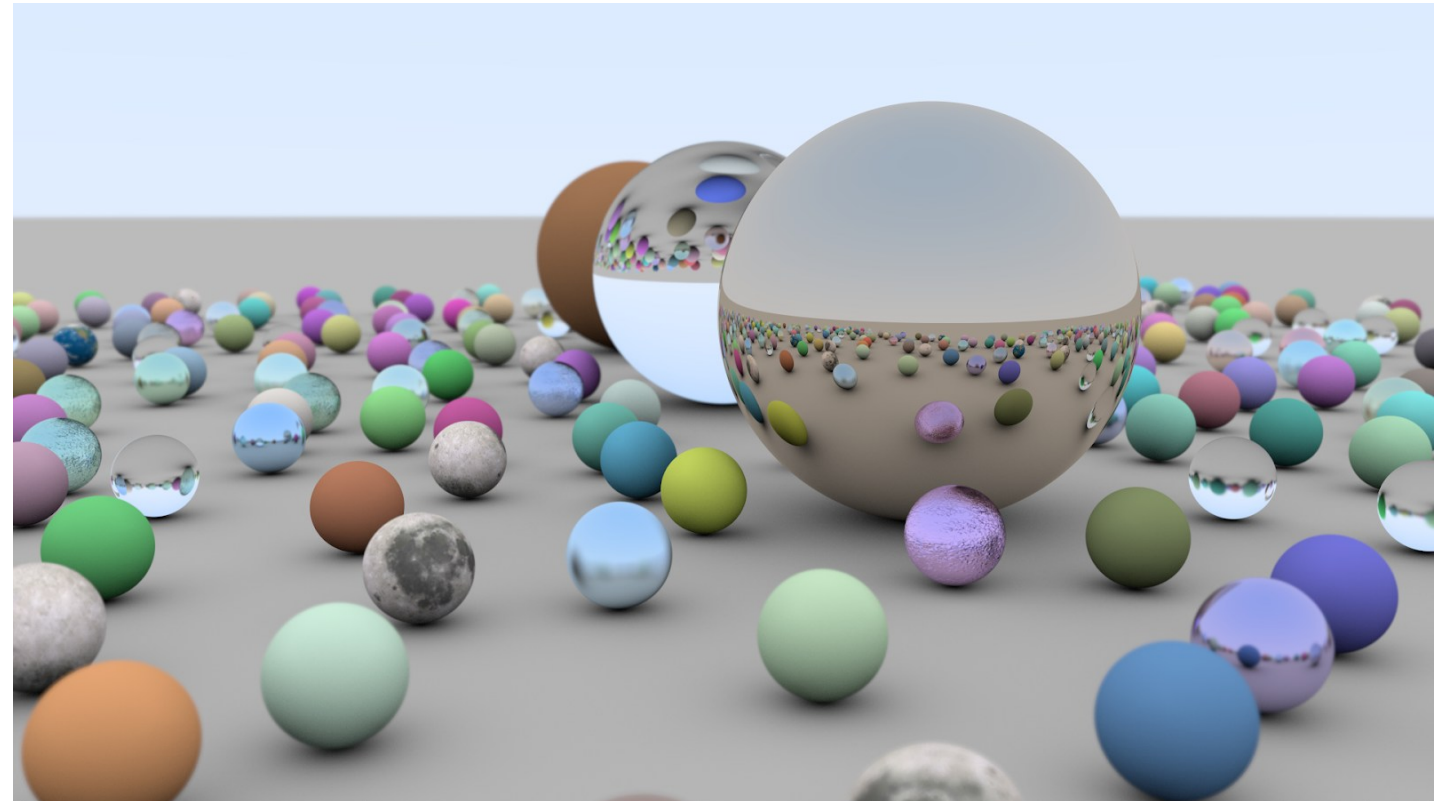


Reflections Multiple Mirrors

- One way is to use **Frame Buffer Objects** (FBOs) (rendering on a texture not displayed)
- We render the scene one time from the point of view of the reflective object.
- Then, we use it as a texture over another object.

- Not extremely efficient.

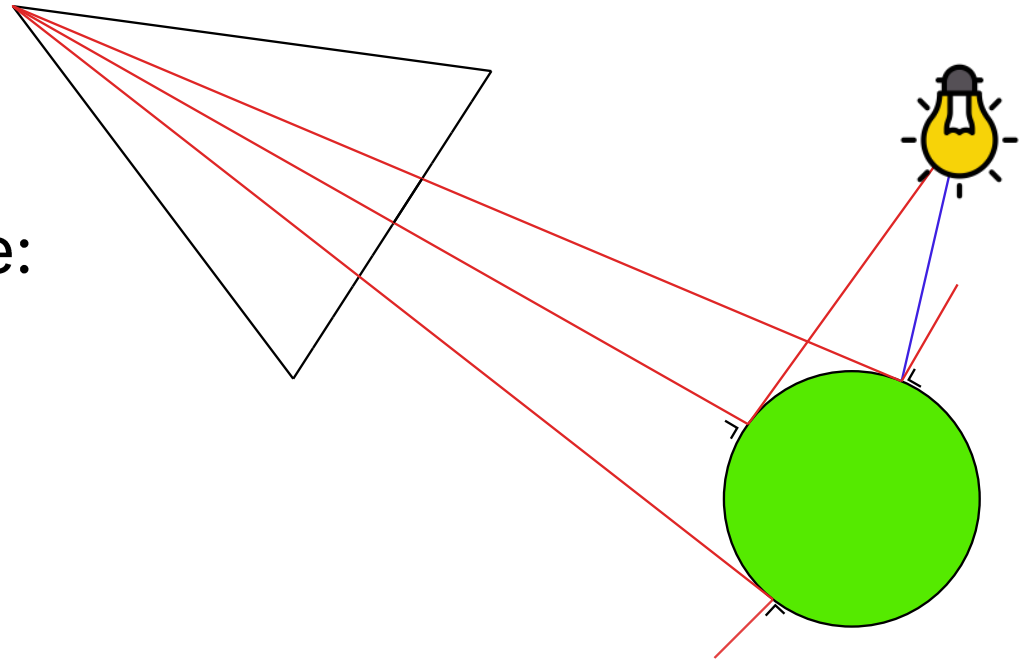
- Can we do better?



Reflections Multiple Mirrors

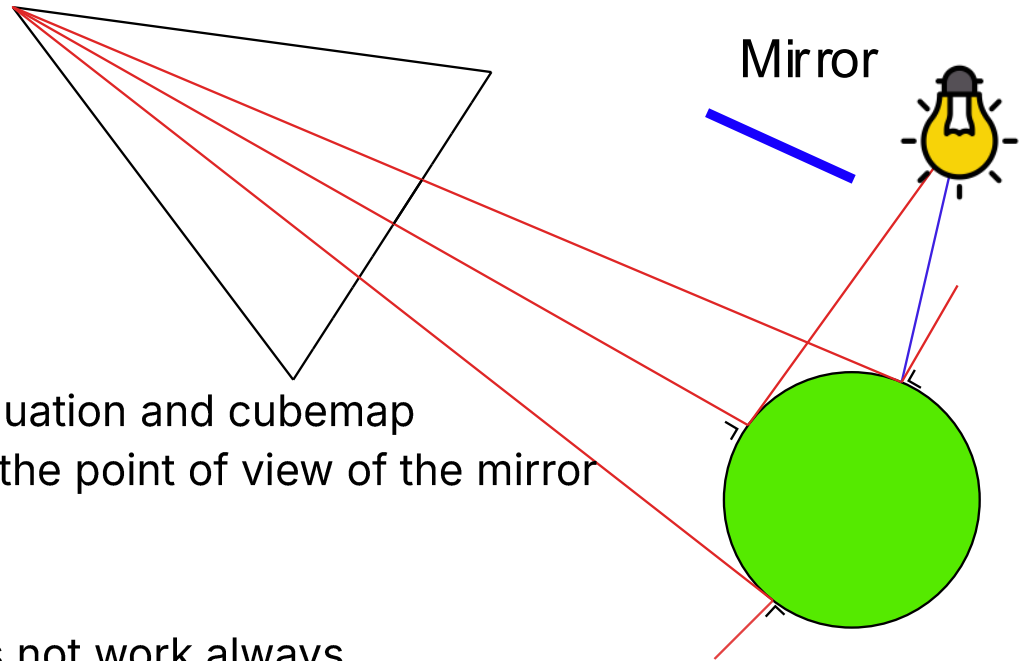
- Can we do better? Analysis of how it works from a ray perspective.

- Up to now, using the OpenGL pipeline:
 - For each light:
 - Compute color at vertices
 - Project vertices to screen
 - Divide in fragments
 - Interpolate colors



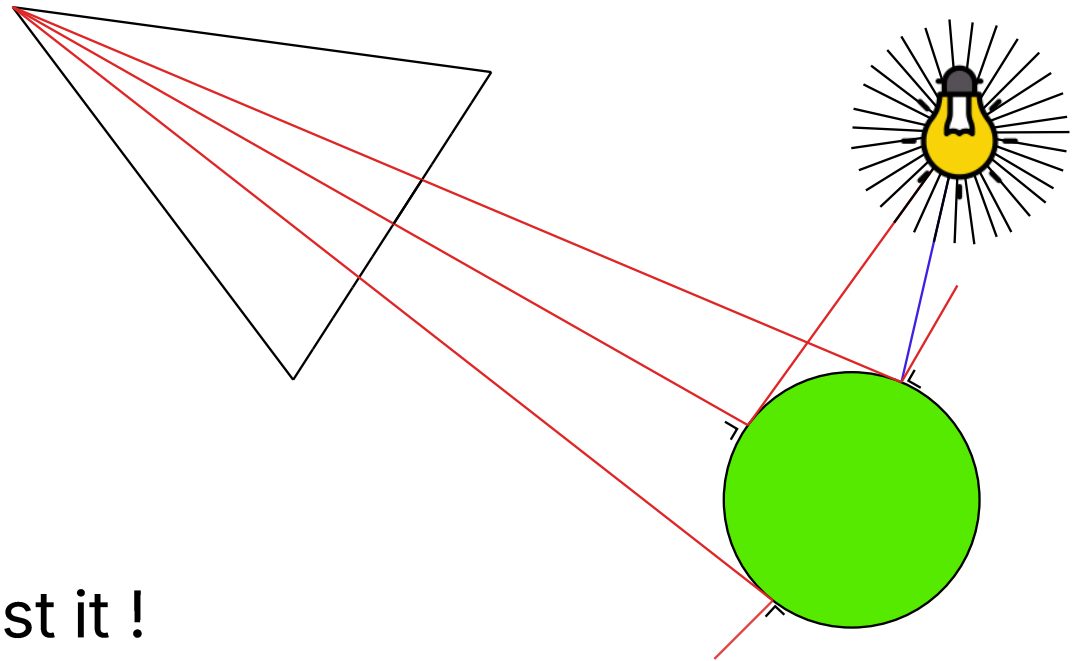
OpenGL Pipeline and Simple Scene

- Up to now:
 - For each line:
 - Compute color at vertices
 - Project vertices to screen
 - Divide in fragments
 - Interpolate colors
- We would like to add a mirror(s)
 - Everything is fine:
 - If one mirror:
 - If no objects in front: use the reflection equation and cubemap
 - If objects in front: Render the scene from the point of view of the mirror
 - Store it as a texture (FBO)
 - If multiple:
 - Recursively apply the FBO trick.. But does not work always..
 - Render the scene from the point of view of the camera
- But, as we saw, it does not scale ! (adding other mirrors)



OpenGL Pipeline and Simple Scene

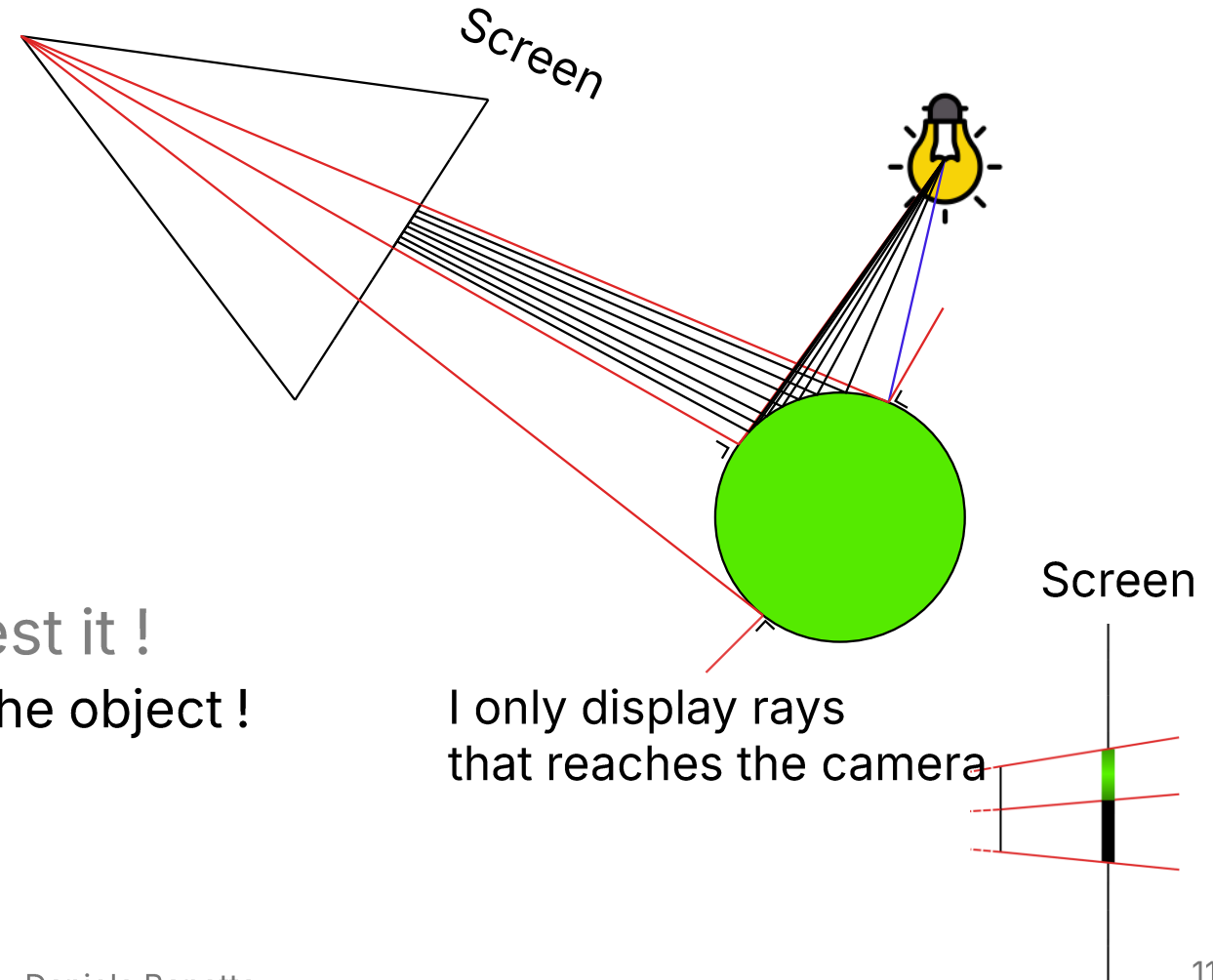
- Idea:
 - Let's use $L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$ recursively!
 - The final rendered scene depends on the light
 - We could trace rays from the light and let them bounce on objects using the:
 - Reflection equation
 - Refraction equation
 - This approach is recursive
 - We need to keep track of each ray
 - Accumulate colors along the rays
 - When a ray reaches the camera
 - We store its color as rendered
- The idea seems good enough, let's test it !



OpenGL Pipeline and Simple Scene

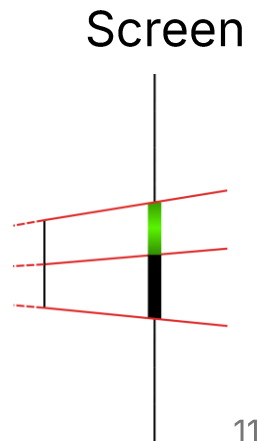
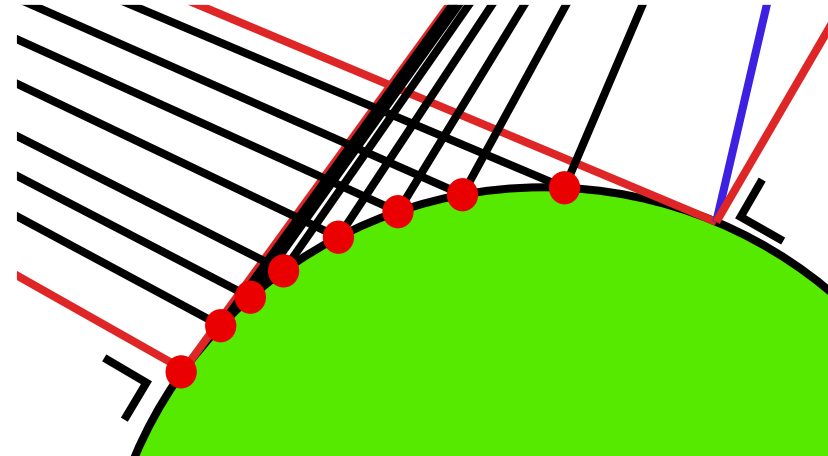
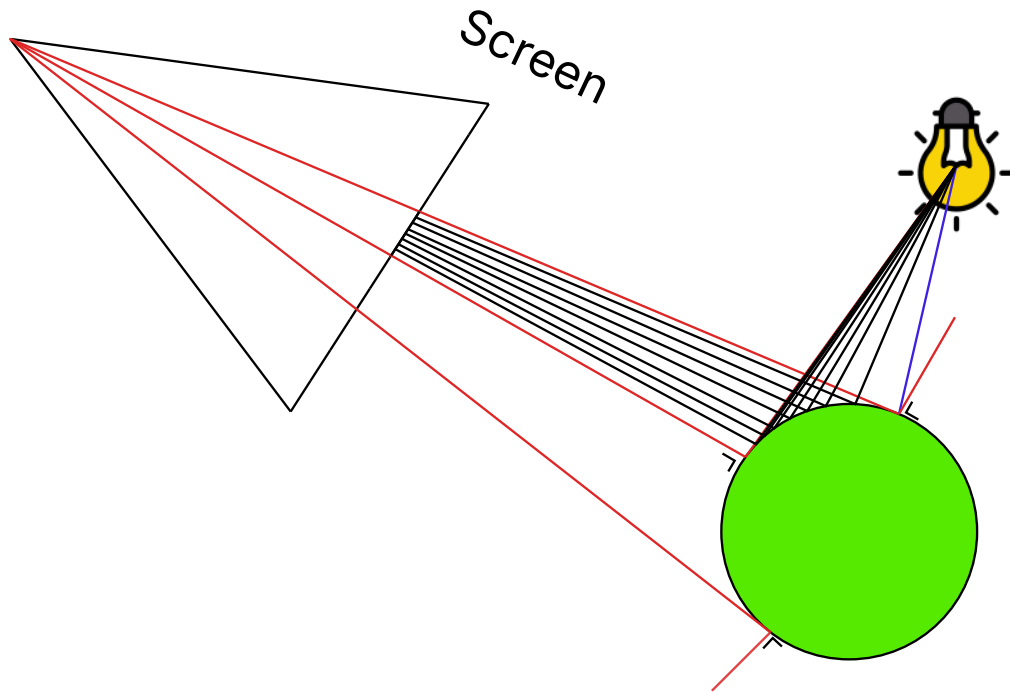
- Idea:

- Let's use $L_o(\vec{x}, \omega_o, \lambda, t) = L_e(\vec{x}, \omega_o, \lambda, t) + \int_{\Omega} f_r(\vec{x}, \omega_i, \omega_o, \lambda, t) L_i(\vec{x}, \omega_i, \lambda, t) (\omega_i \cdot \vec{n}) d\omega_i$ recursively!
- The final rendered scene depends on the light
- We could trace rays from the light and let them bounce on objects using the:
 - Reflection equation
 - Refraction equation
- This approach is recursive
 - We need to keep track of each ray
 - Accumulate colors along the rays
- When a ray reaches the camera
 - We store its color as rendered
- The idea seems good enough, let's test it !
 - Sure enough, we can draw on the screen the object !
 - Is that statement accurate..?



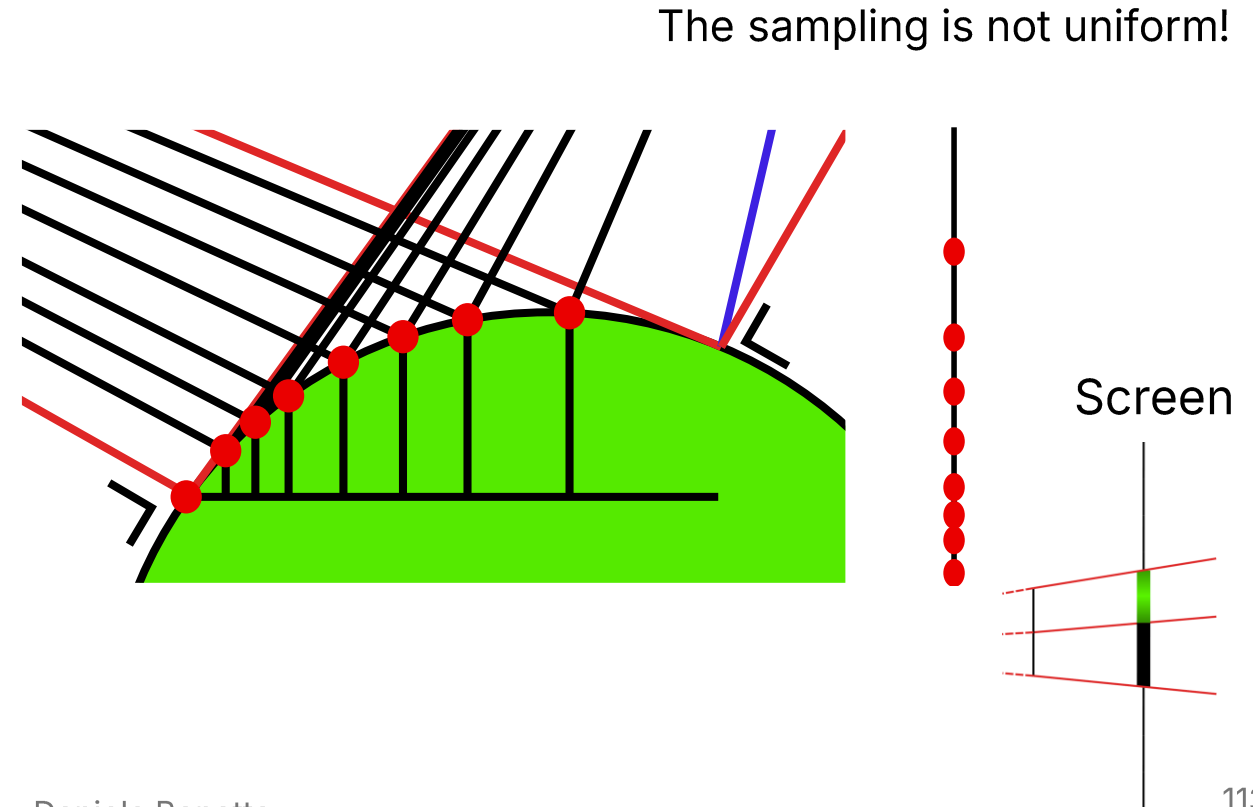
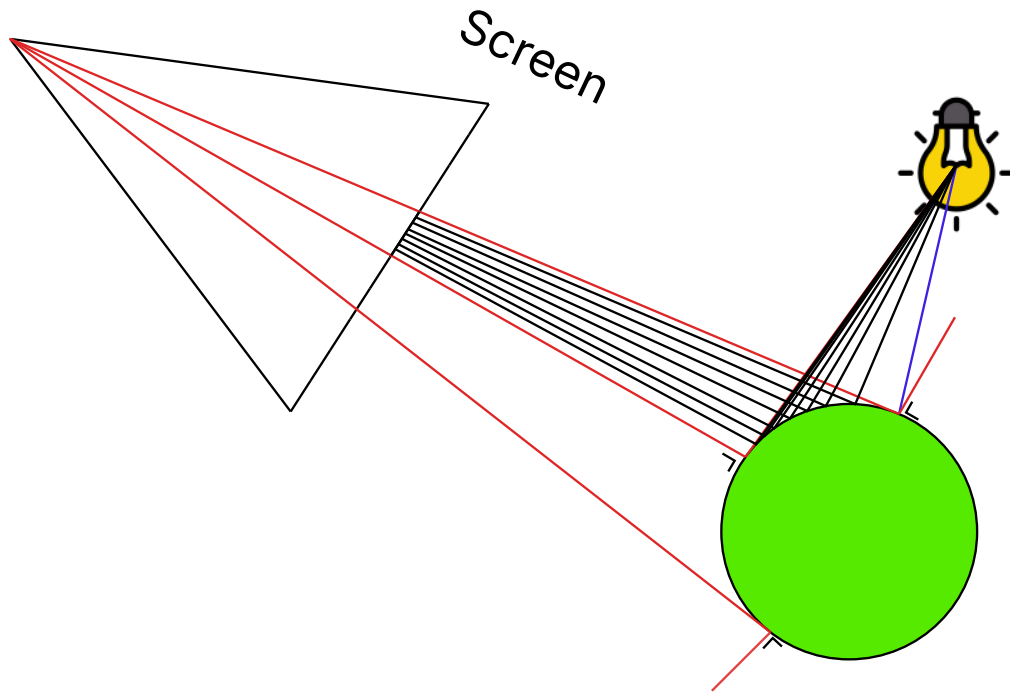
OpenGL Pipeline and Simple Scene

- The idea seems good enough, let's test it !
 - Sure enough, we can draw on the screen the object !
 - Is that statement accurate.. ?
 - Let's zoom in
 - Notice something ?



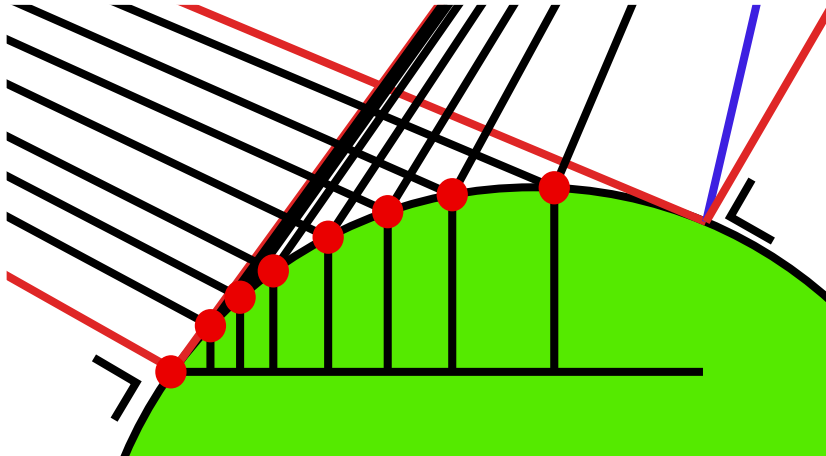
OpenGL Pipeline and Simple Scene

- The idea seems good enough, let's test it !
 - Sure enough, we can draw on the screen the object !
 - Is that statement accurate.. ?
 - Let's zoom in
 - Notice something ?

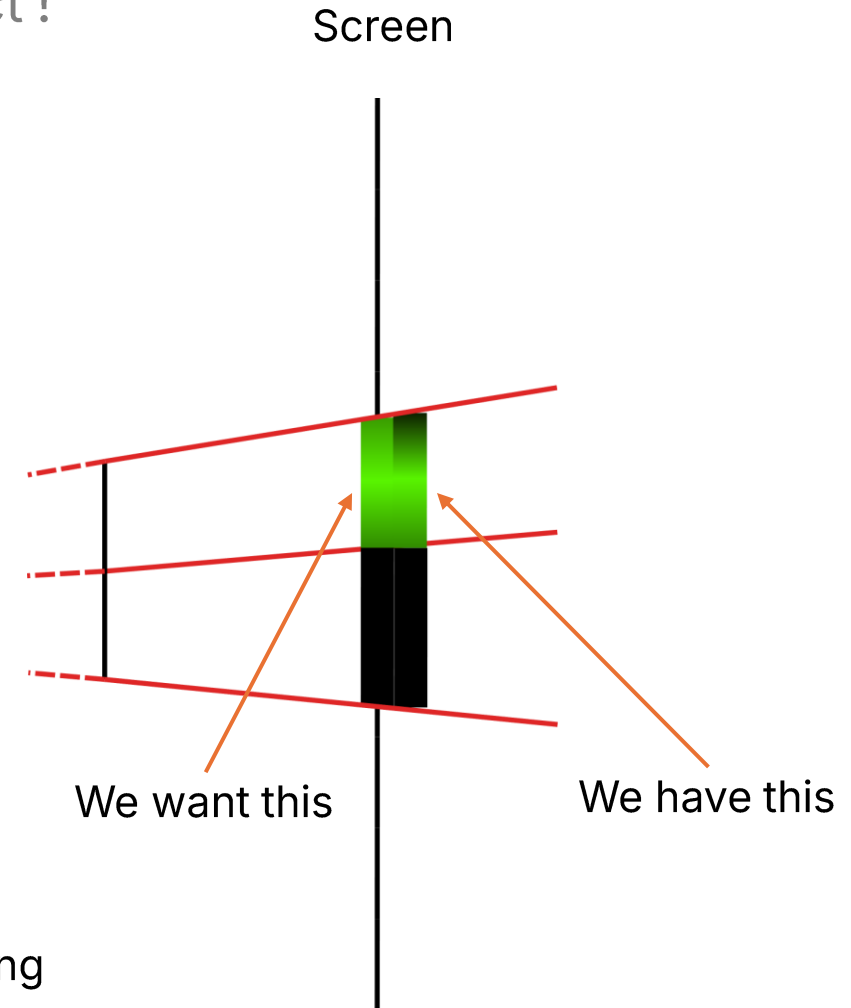


OpenGL Pipeline and Simple Scene

- The idea seems good enough, let's test it !
 - Sure enough, we can draw on the screen the object !
 - Is that statement accurate.. ?
 - Let's zoom in
 - Notice something ?
 - Not enough light rays on the top of the sphere !
 - We can just add more !



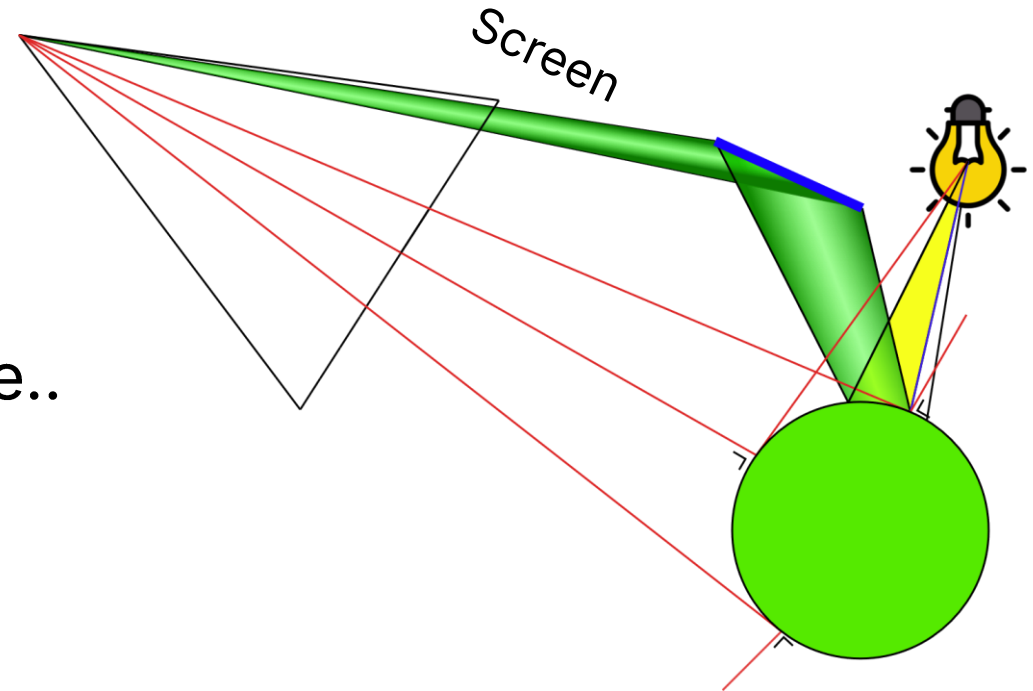
Non uniform sampling



OpenGL Pipeline and Mirror Scene

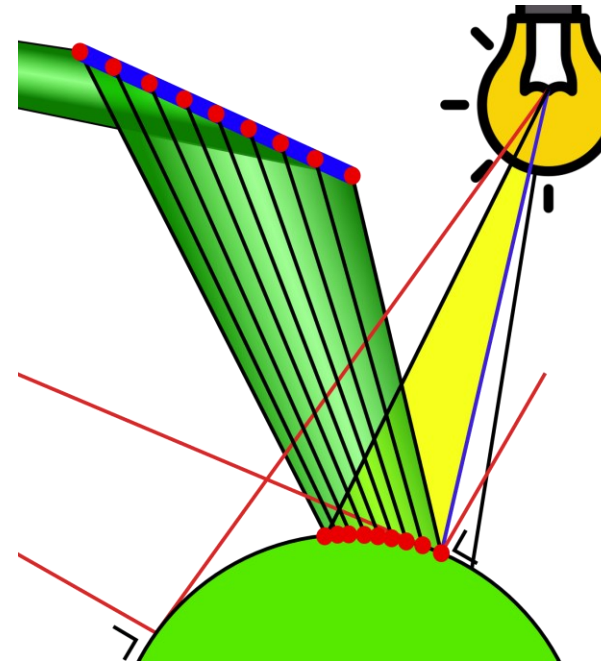
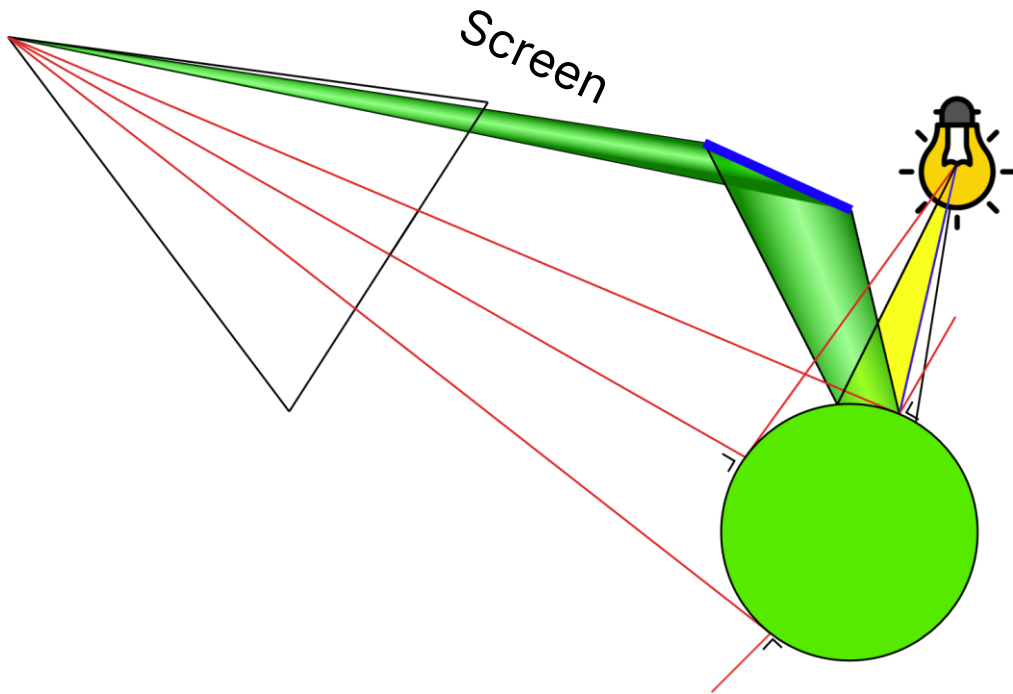
- But then.. The Mirror comes in.

- Okay... Everything seems to work fine..
- Or does it ?
 - Let's zoom in.

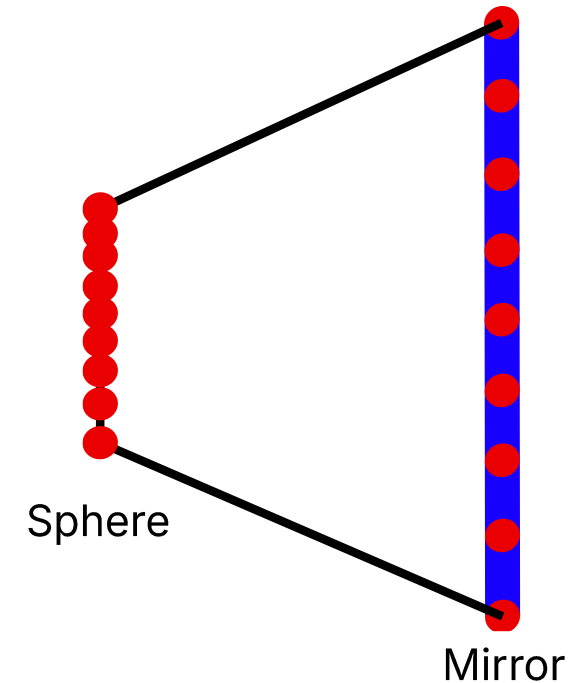


OpenGL Pipeline and Mirror Scene

- But then.. The Mirror comes in.
- Even if we have increased the numbers of rays from the light
- On the mirror, it is still sparse..



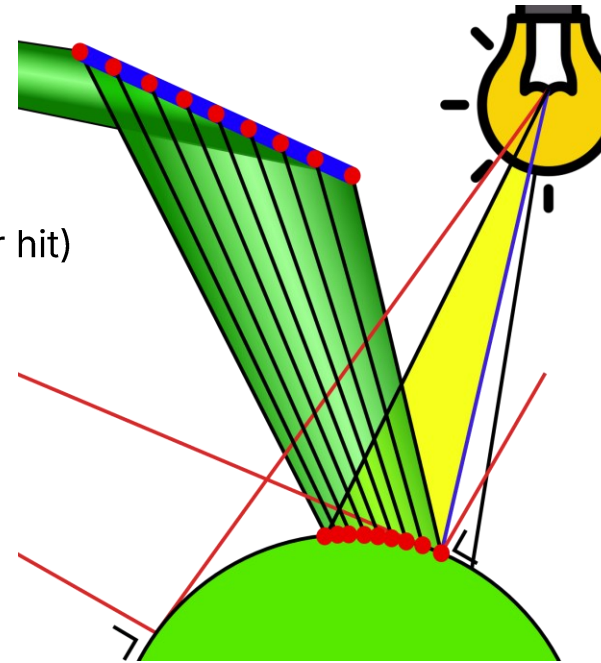
The sampling is sparse on the mirror



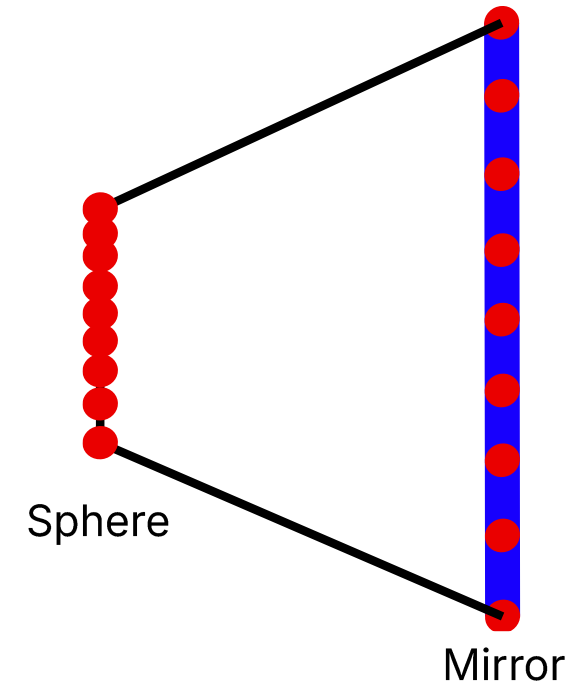
OpenGL Pipeline and Mirror Scene

- But then.. The Mirror comes in.
- Even if we have increased the numbers of rays from the light
- On the mirror, it is still sparse..

- Increasing even more rays from the light:
 - Exponentially increases the number of bounces on objects (worst case 2 bounces per hit)
 - Computational explosion
 - Does not guarantee to have a rendering without gaps on the screen
 - We will never have enough rays when we bounce on many objects



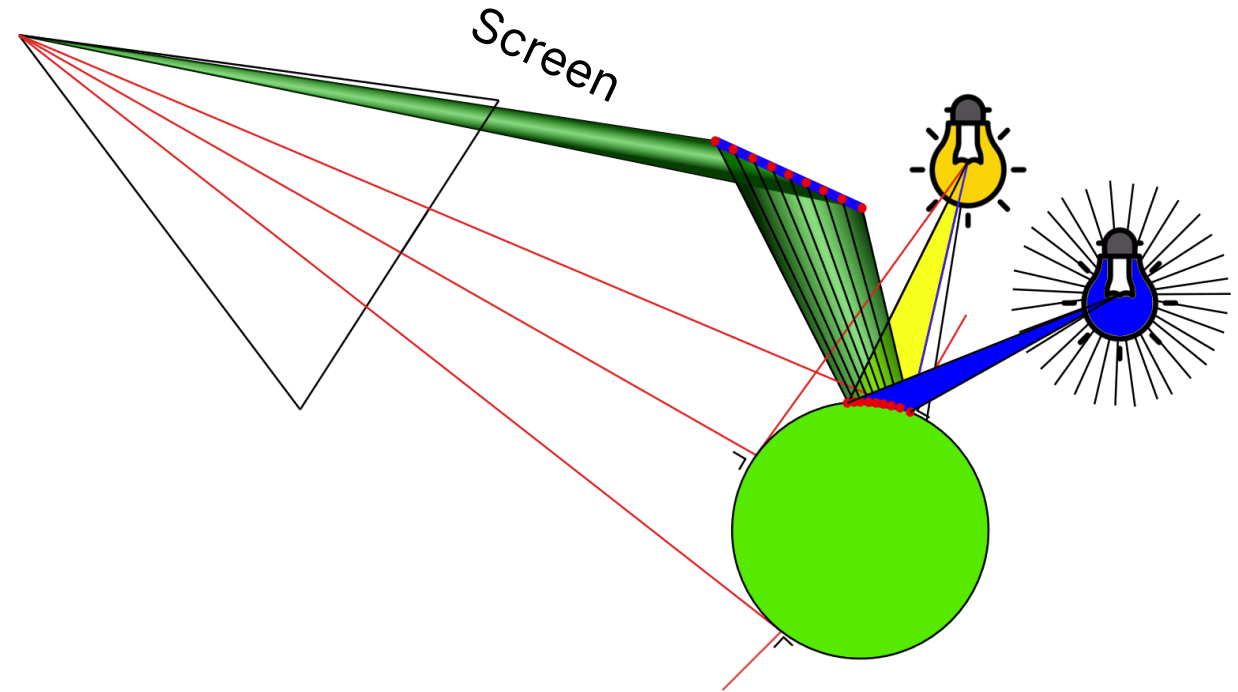
The sampling is sparse on the mirror



OpenGL Pipeline and Mirror and Lights Scene

- It get even worse...
- If we add more lights !

- This idea does not work..
 - What is the problem?



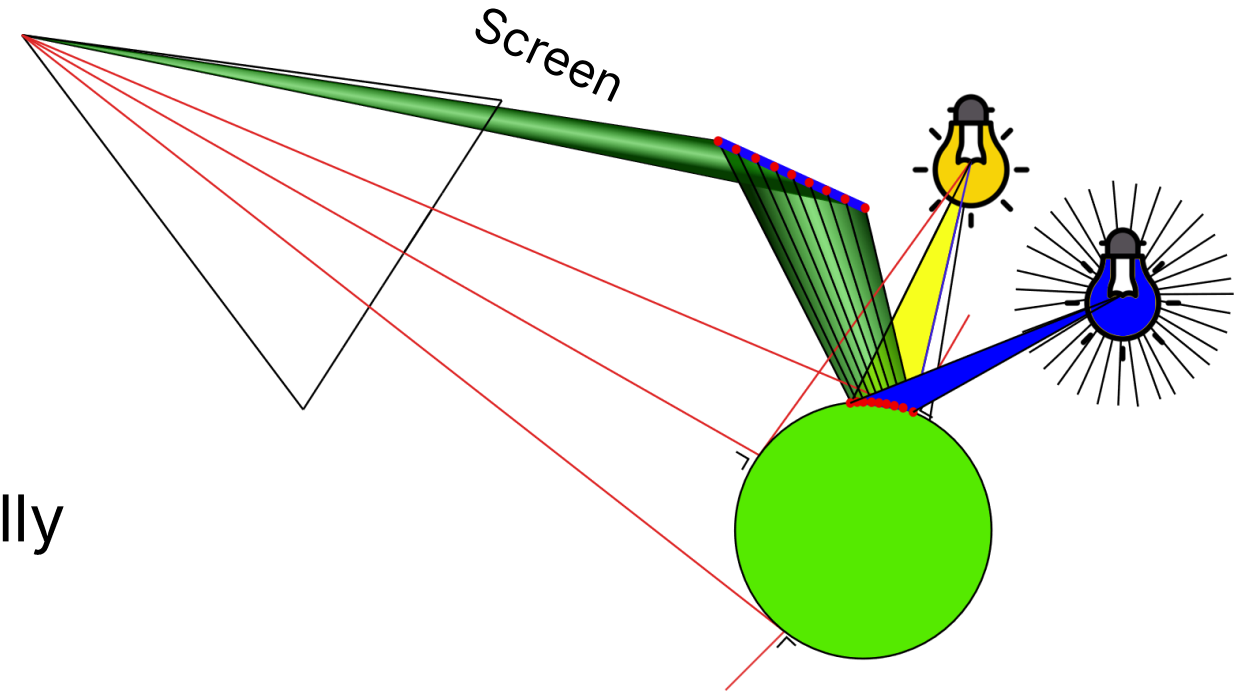
OpenGL Pipeline and Mirror and Lights Scene

- It get even worse...
- If we add more lights !

- This idea does not work..
 - What is the problem?

- The number of rays grow exponentially
 - And the number of bounces follows..

- Too computationally expensive and no guarantee on the rendering
- We would like to bound the number of rays we launch ! ...but how.. ?

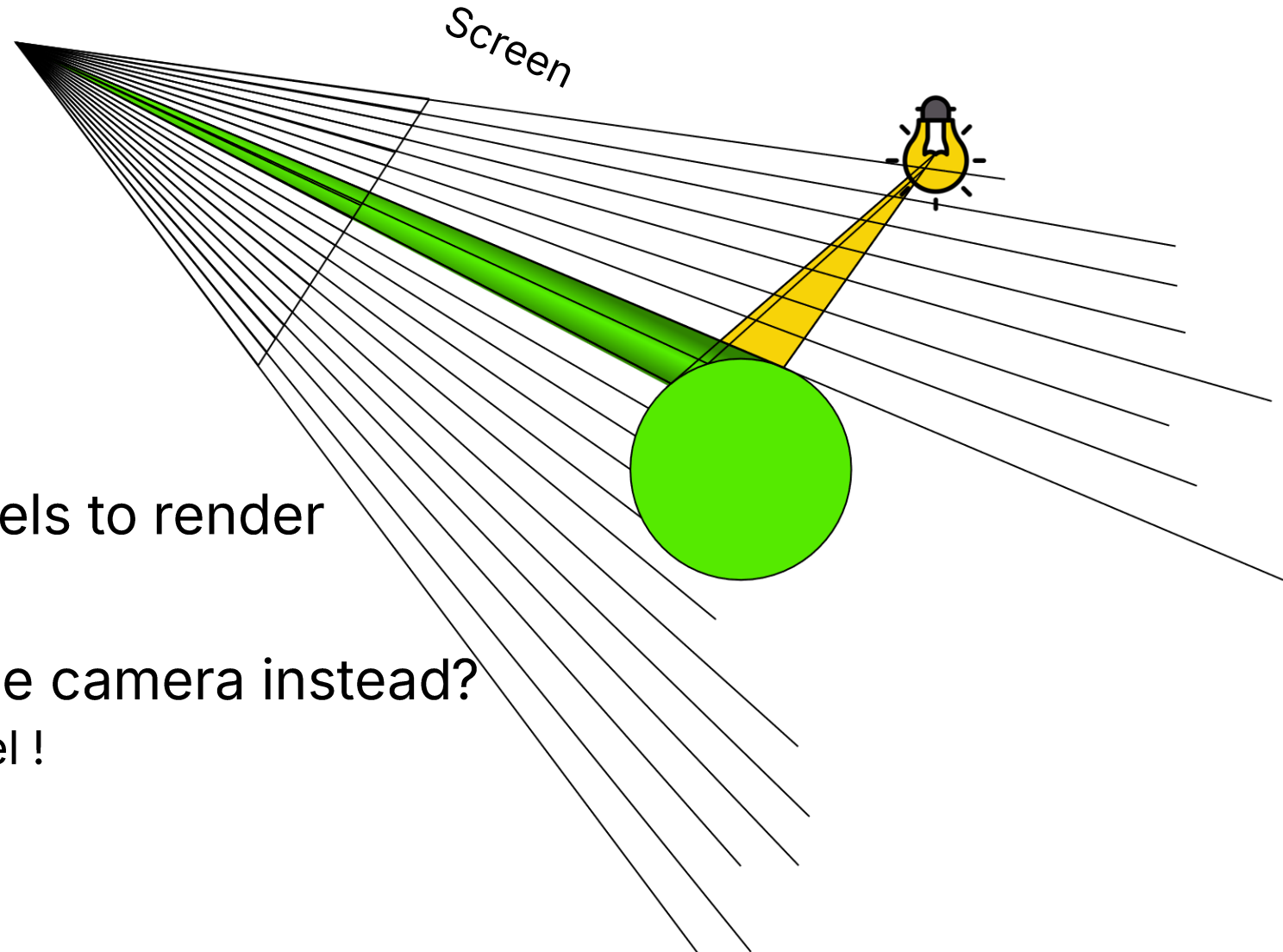


Raytracing

- We would like to bound the number of rays we launch !

- Entering: Raytracing

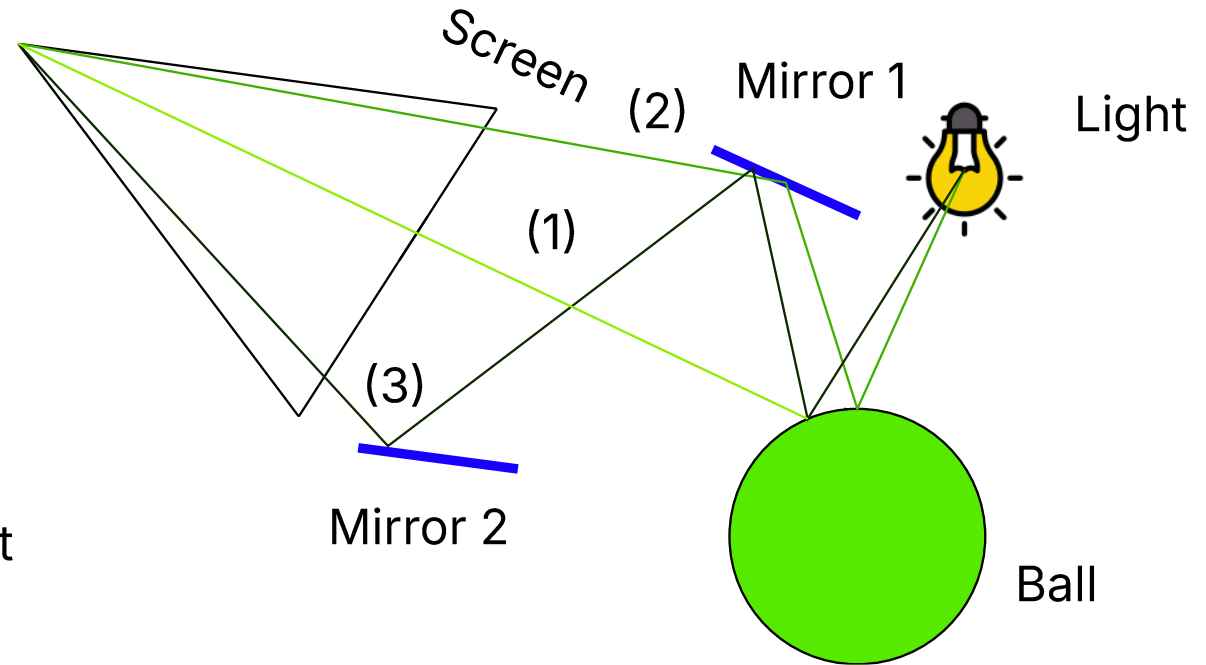
- We have a fixed number of pixels to render
- What if we launch rays from the camera instead?
 - We have at least one color per pixel !
 - Computations are bound !



Raytracing

- We can even compute bounces !

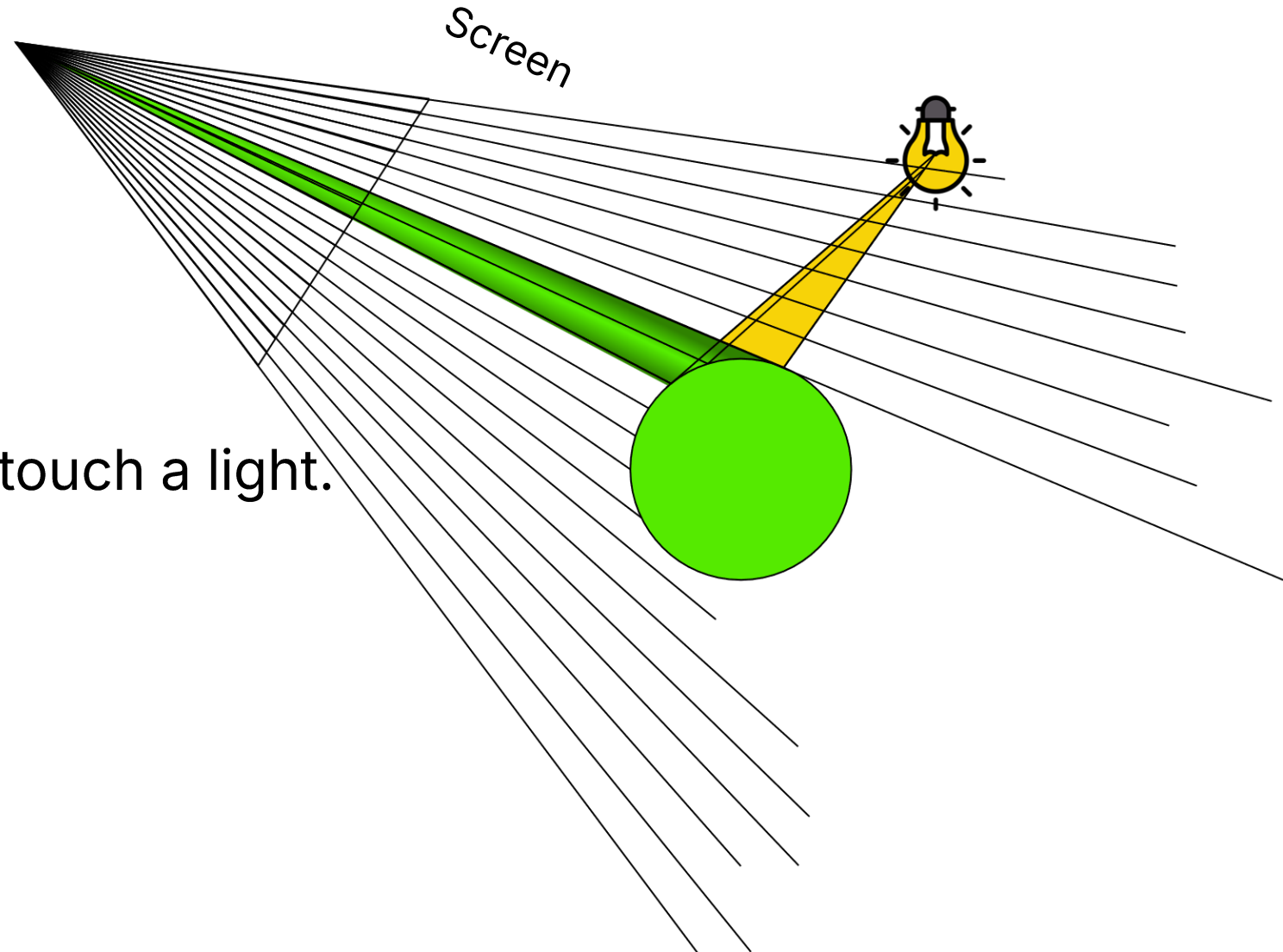
- (1) Direct object hit (bounces to light)
- (2) Hit mirror 1
 - bounces to ball
 - bounces to light
- (3) Hit mirror 2
 - bounces to mirror 1
 - bounces to ball
 - bounces to light



Raytracing

We recursively compute the final color starting from the pixels.

- If a ray does not hit a lamp
 - we set black
- We limit the number of recursive bounces to N
- Caveat: Too many rays do not touch a light.
- What to do?

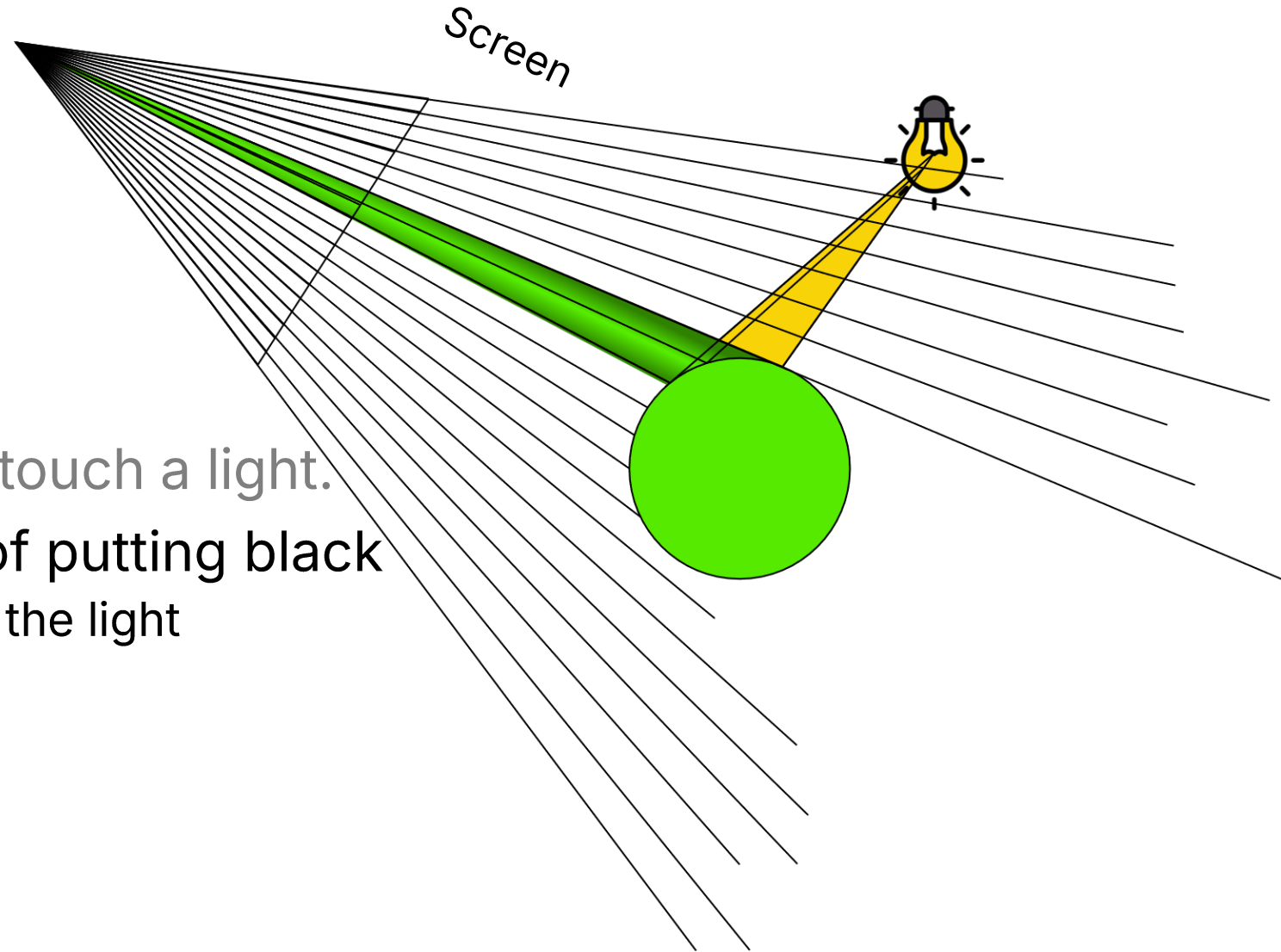


Raytracing

We recursively compute the final color starting from the pixels.

- If a ray does not hit a lamp
 - we set black
- We limit the number of recursive bounces to N

- Caveat: Too many rays do not touch a light.
- We add ambient light instead of putting black
 - It simulates the global bouncing of the light on every object.

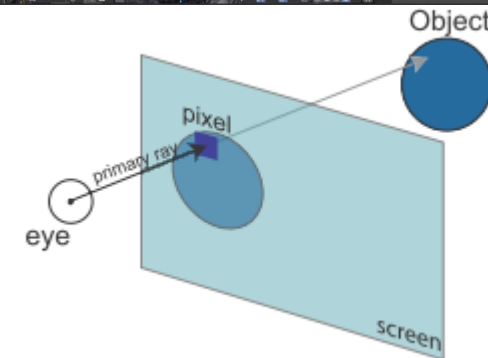
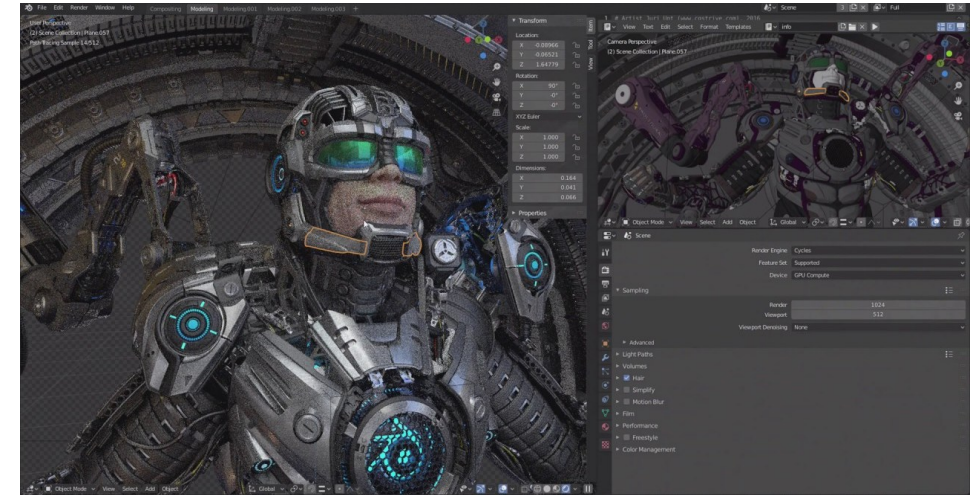


Raytracing – Pseudo code

```
class Scene(object):
    def __init__(self, blender_file):
        b = blender_file
        self.mesh = parse_mesh(b)
        self.lights = parse_lights(b)
        # attenuation could also be parsed from scene
        self.refl_att = 0.5f # reflection attenuation
        self.refr_att = 0.2f # refraction attenuation
        self.background_color = parse_bg(b)
        self.ambient_color = parse_amb(b)
```

```
class Screen(object):
    def __init__(self, eye_pos):
        # screen pixels
        self.pixels = [[], ...]
        # eye position in world space
        self.eye_pos = eye_pos
```

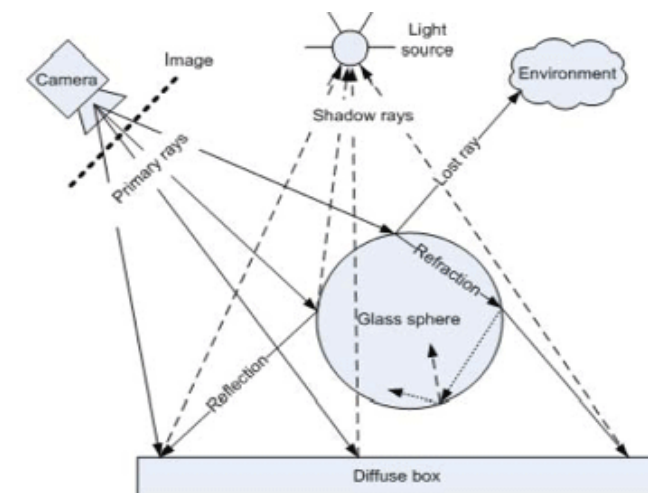
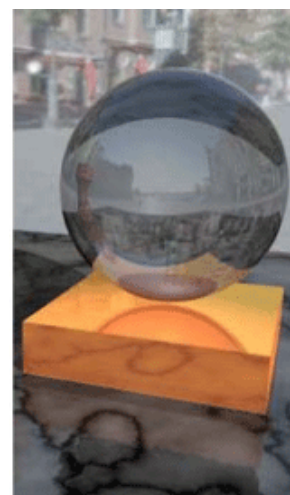
```
def topLevelRoutine(screen, scene):
    pos_eye = screen.eye_pos
    color = [[], ...] #2D array with same shape as screen
    for pixel in screen.pixels:
        # unit vec from pos_eye to the pixel
        pix_dir = normalize(pixel - pos_eye)
        color[pixel] = raytrace(pos_eye, pix_dir, max_depth, scene);
```



Raytracing – Pseudo code

```
def raytrace(pos_eye, pix_dir, depth, scene):  
    r = ray(pos_eye, pix_dir) # ray from eye toward pix_dir  
    # intersect needs to be defined  
    mesh = scene.mesh  
    intersection, q = get_intersection(r, mesh)  
    if not intersection:  
        return scene.background_color  
  
    color = scene.ambient_color  
    for light in scene.lights:  
        L = normalize(light - q) # unit vec, q to light  
        # using the rendering equation:  
        color += compute_color(L, q)  
  
    if depth > 0:  
        dir_refl = reflect(r, mesh[q].normal)  
        dir_refr = refract(r, mesh[q].normal, refr_index)  
        # the new eye is at the position q and looks toward  
        # the reflection or the refraction  
        color += scene.refl_att * raytrace(q, dir_refl, depth - 1)  
        color += scene.refr_att * raytrace(q, dir_refr, depth - 1)  
  
    return color
```

```
def get_intersection(ray, mesh):  
    # compute intersection between the ray and the mesh  
    # usually with spatial structures  
    if intersection:  
        q = point of first intersection  
        return True, q  
  
    return False, (0, 0, 0)
```

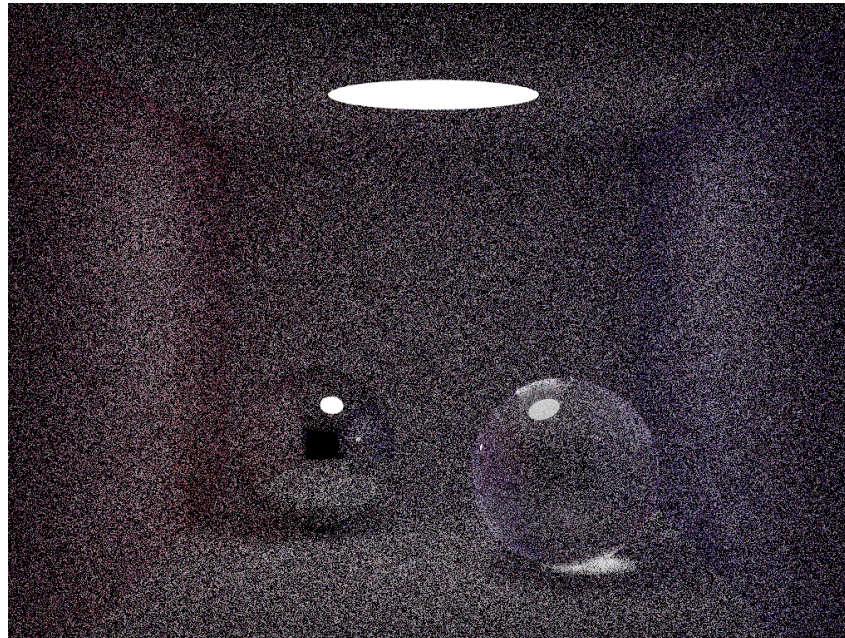


Raytracing – One ray per pixel

- Okay, we launched one ray per pixel
- Computed all the bounces on objects
 - If it hits a light, we propagate the color
 - If not, we put ambient light
- But wait.. Then we have only one color computed per pixel.. Is it enough?

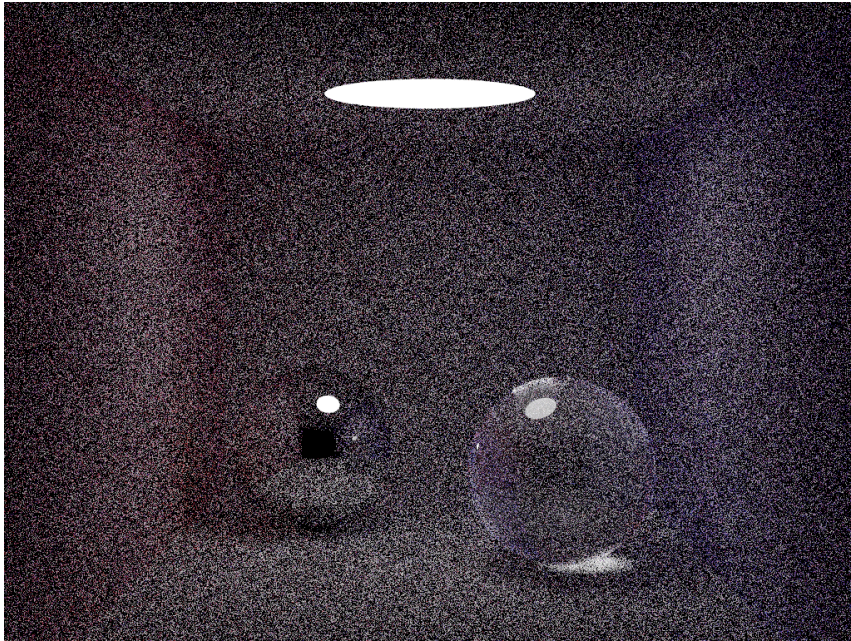
Raytracing – One ray per pixel

- Okay, we launched one ray per pixel
- Computed all the bounces on objects
 - If it hits a light, we propagate the color
 - If not, we put ambient light
- But wait.. Then we have only one color computed per pixel.. Is it enough?
- No ☹️
- What can we do ?



Raytracing – Samples per pixel

- One ray per pixel of the screen is not enough
- We send multiple rays per pixel !

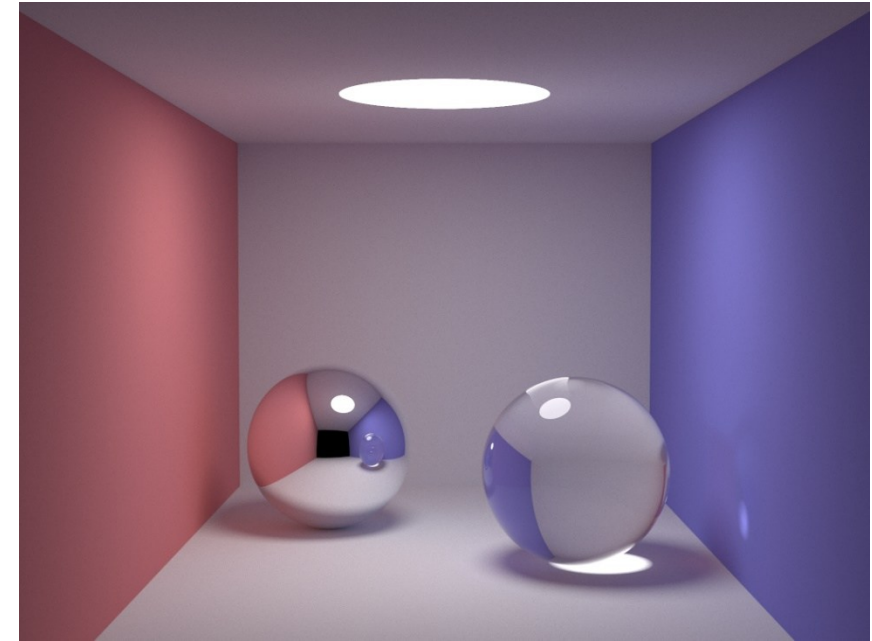


8 samples per pixel: 13 sec

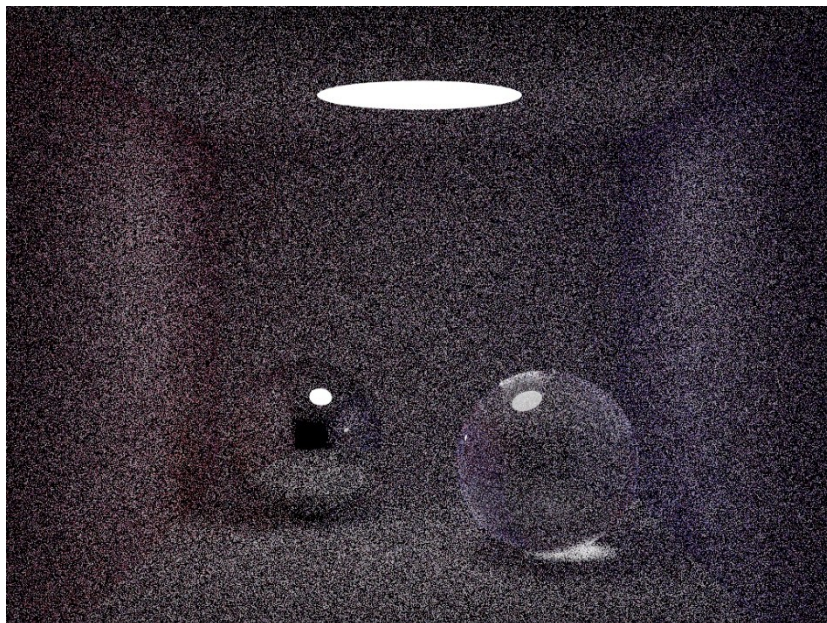
However..
How many rays?

2.4 GHz Intel Core 2 Quad CPU
using 4 threads

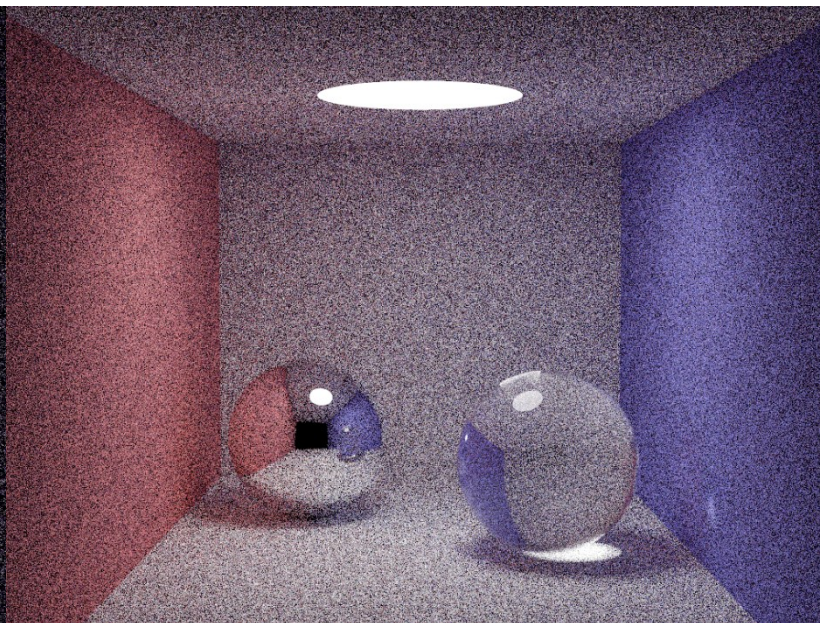
<http://www.kevinbeason.com/smallpt/>



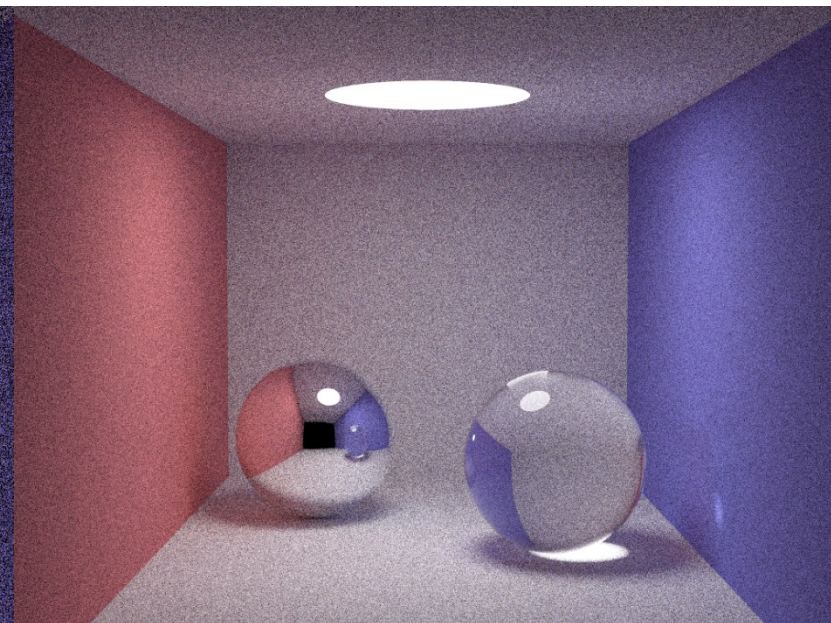
25000 samples per pixel: 10,3 hours



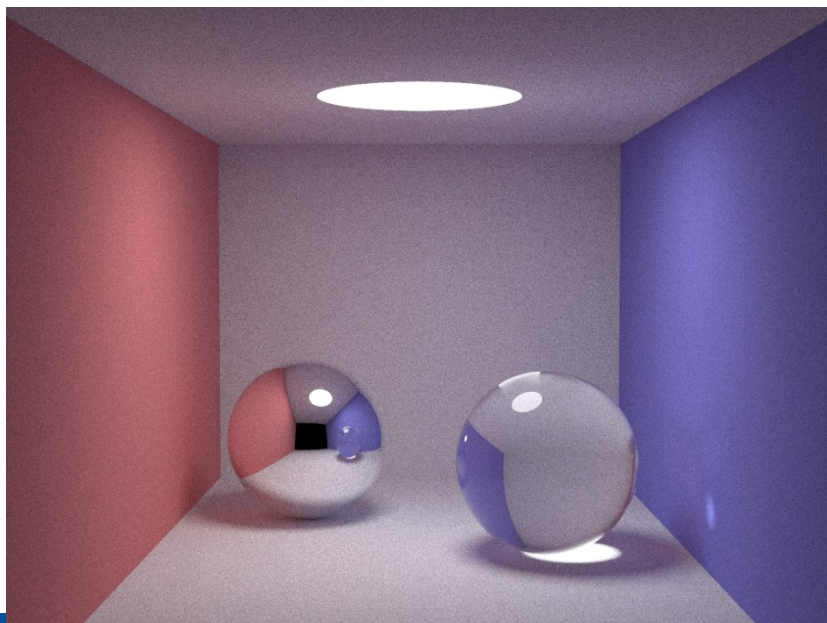
8 samples per pixel: 13 sec



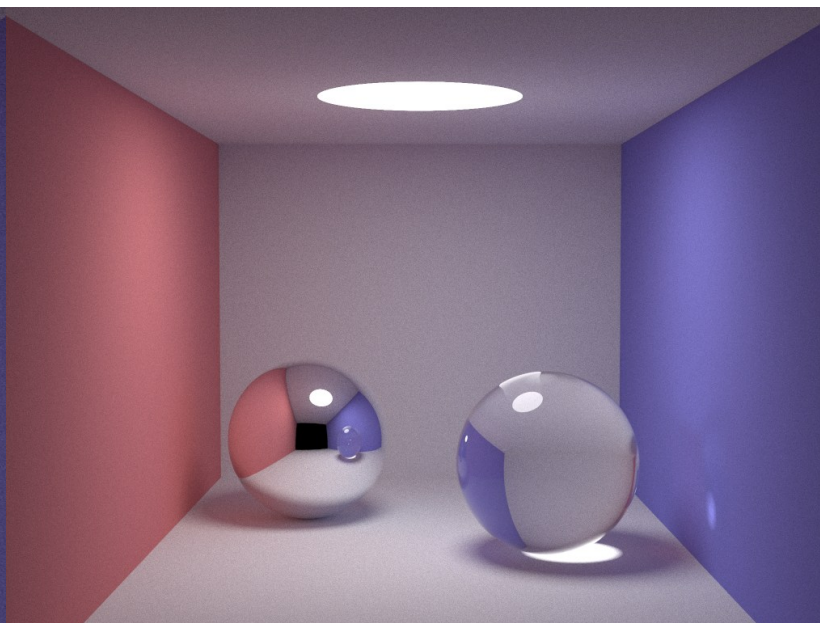
40 samples per pixel: 63 sec



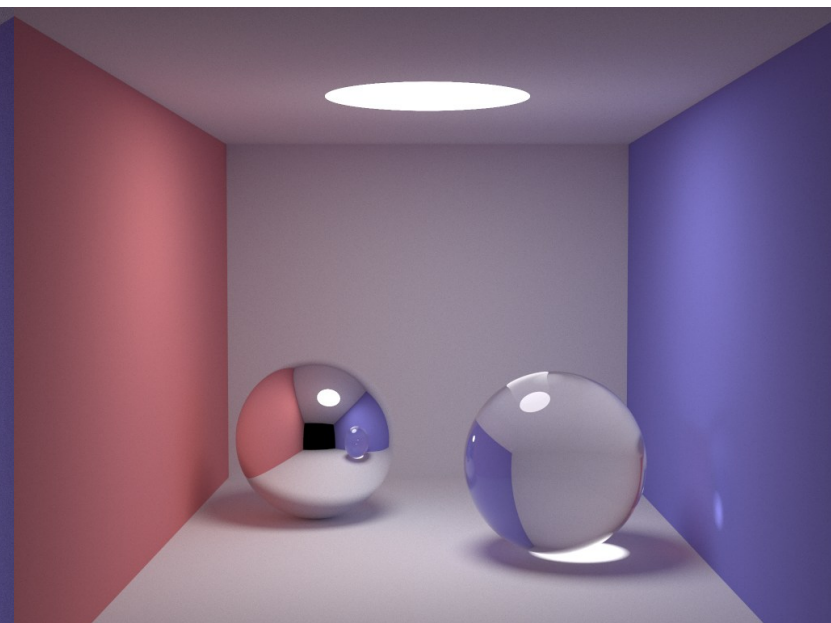
200 samples per pixel: 5 min



1000 samples per pixel: 25 min



5000 samples per pixel: 124 min



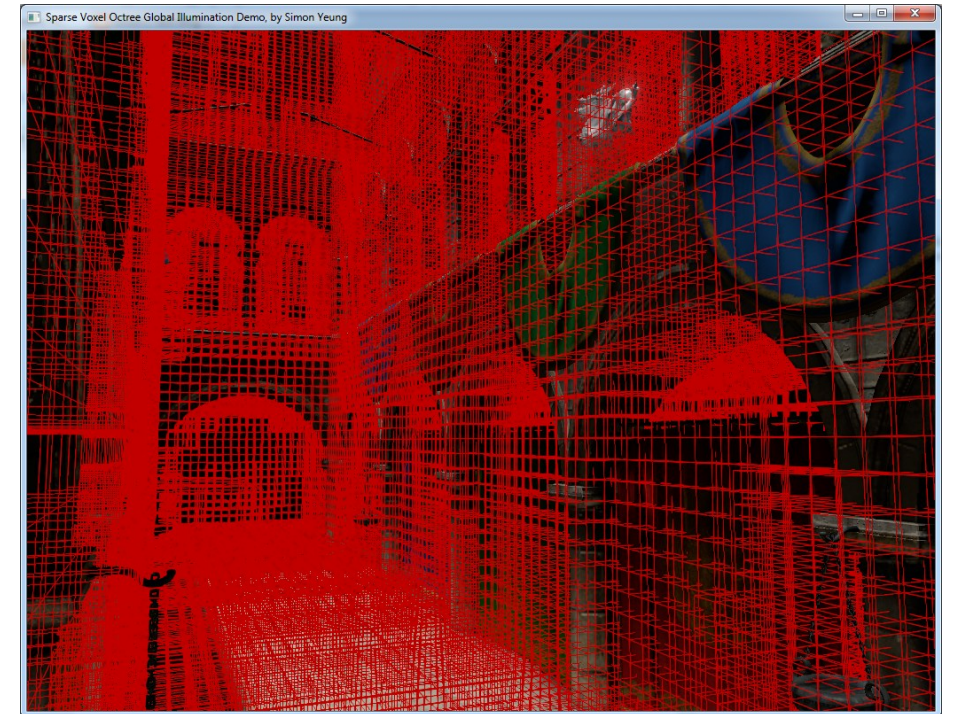
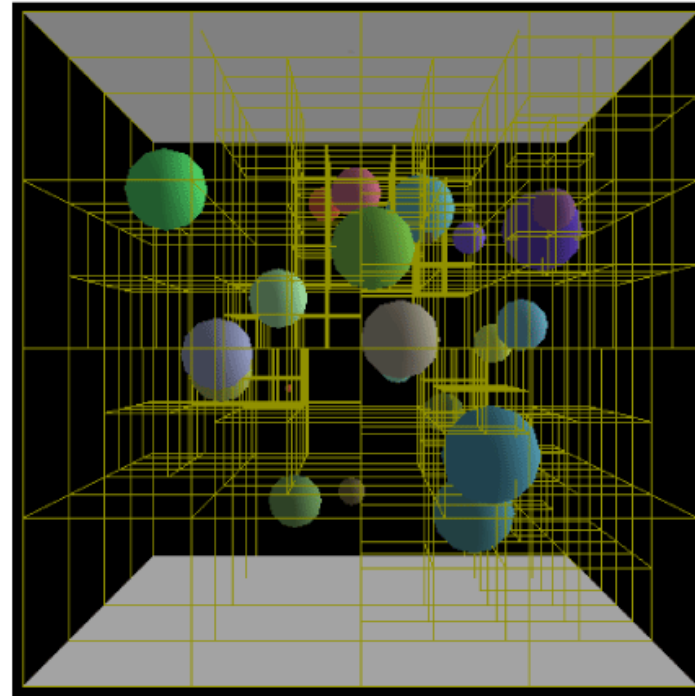
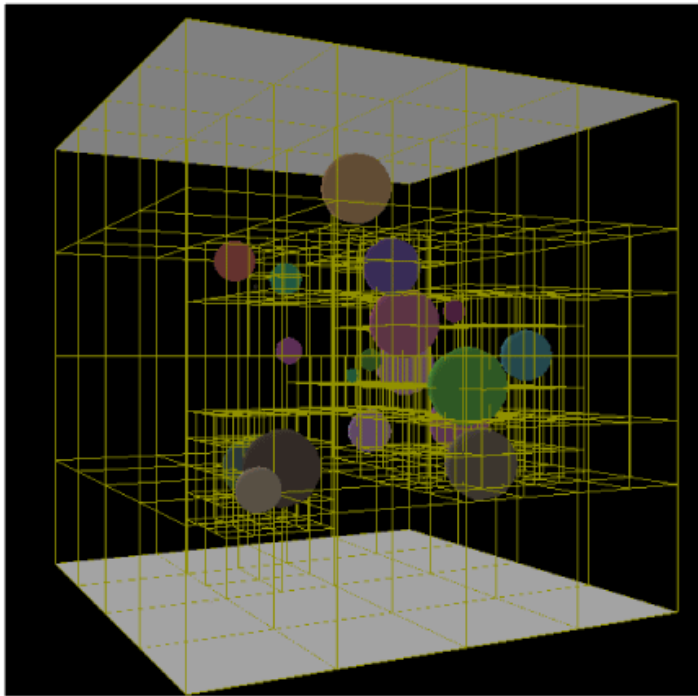
25000 samples per pixel: 10,3 hours

Intersection Ray-Object

- We need to perform intersection computations between the ray and the objects of the scene.
- We use classical intersection equations (geometry)
- But.. That means that we need to compute the intersection between each ray and..... All the vertices of the scene
- It seems impossible 😞 what should we do ?

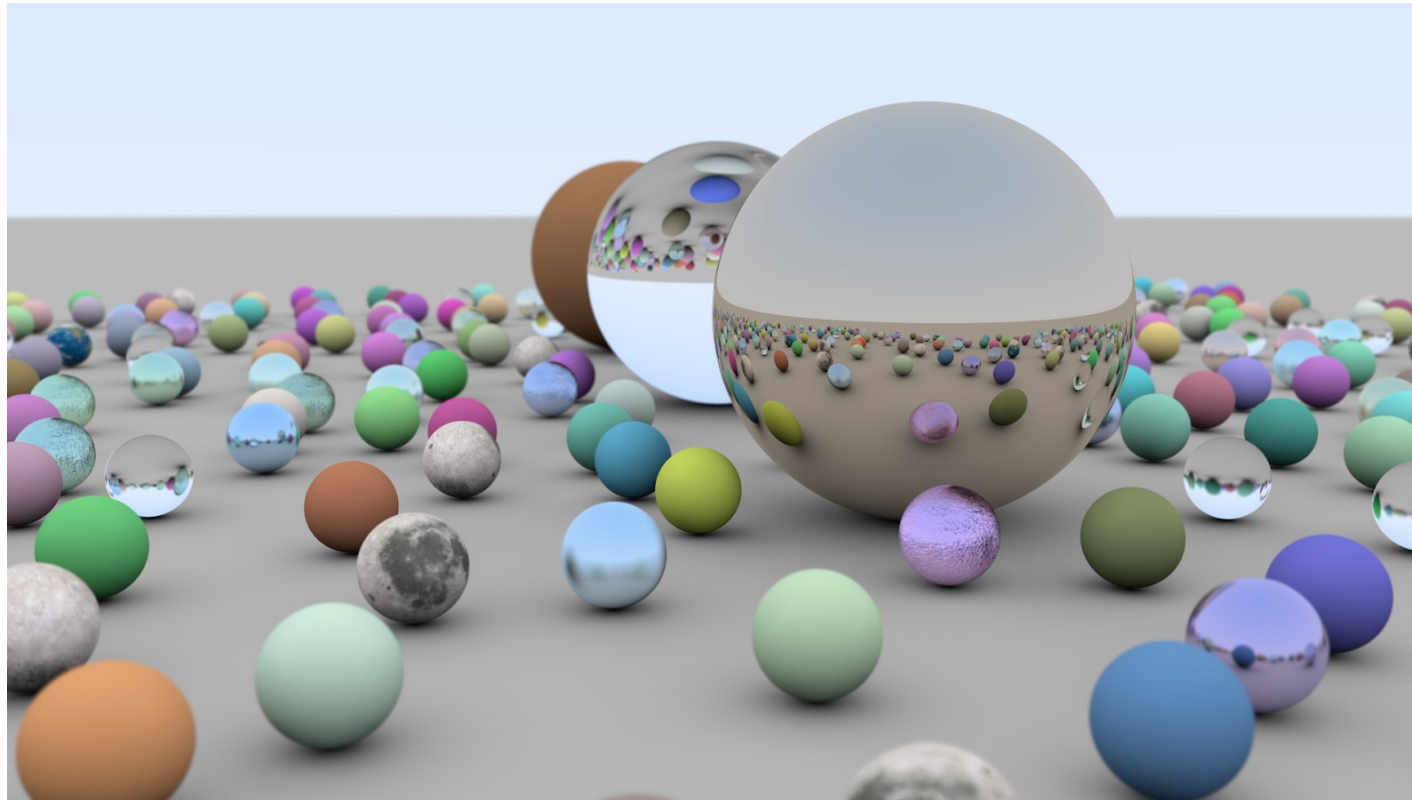
How to accelerate computations ?

- Computing `get_intersection(ray, every point in the mesh)` is slow
- We use space partitioning data structures instead
 - Octree
 - kd-tree
 - ...



Implement your own !

- <https://raytracing.github.io/books/RayTracingInOneWeekend.html>
- A very nice website to implement a complete raytracer in 2 days.
- Lot's of useful information if you want to dig deeper.



This image comes from there

What I Expect for the exam

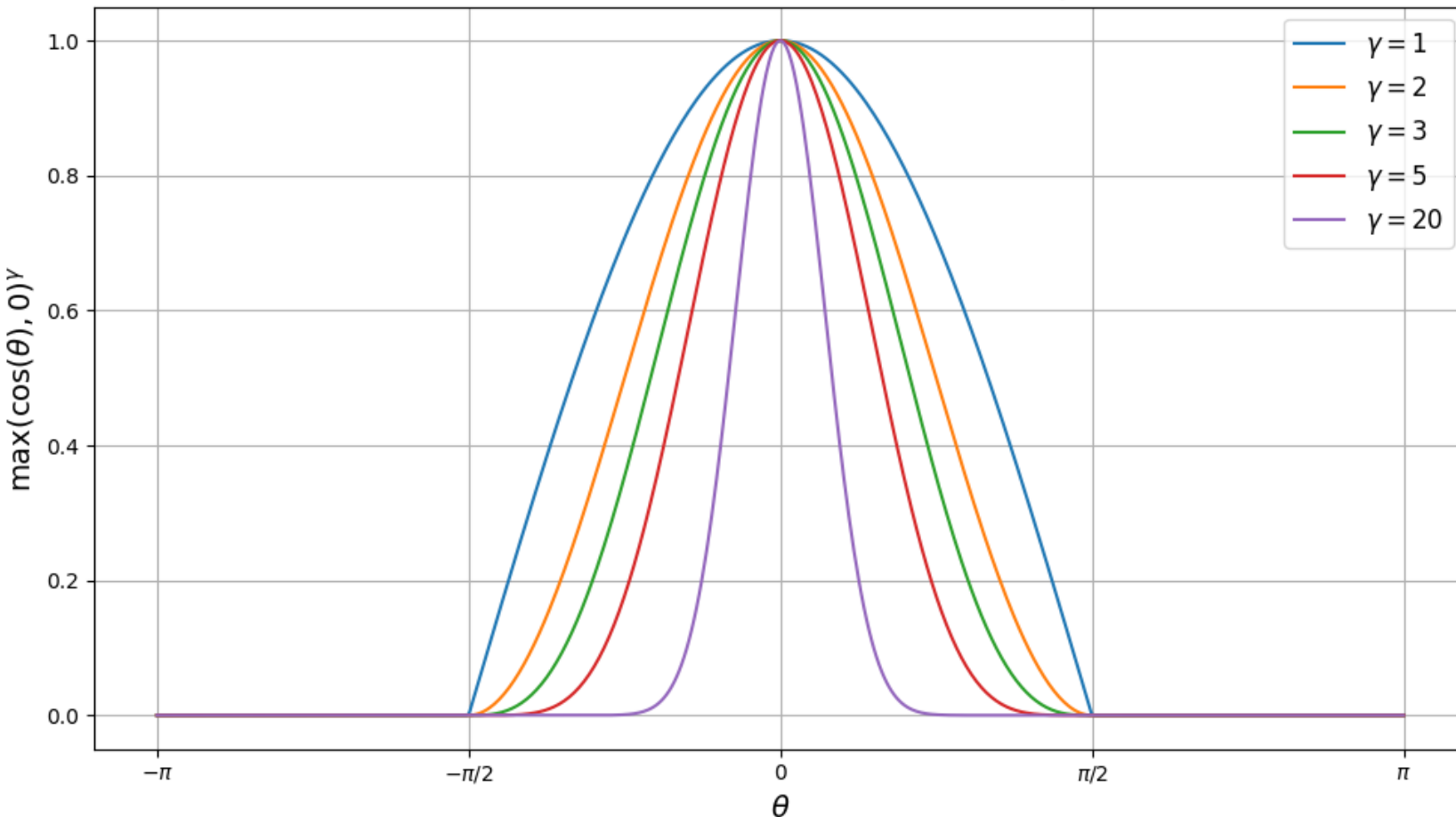
- After this lesson I expect that you
 - understand each part of the rendering equation
 - No need to remember all the formulas, but if I ask a question about a part of the equation, you should be able to explain it
 - know the main simplifications to the rendering equation
 - understand the different shading models (flat, Gouraud, Phong)
 - know the link between the rendering equation and raytracing
 - understand how raytracing works
 - are able to write some pseudo code on how a raytracer works
 - understand why we need raytracing and when to use them
 - understand the tradeoffs when you implement a raytracer
- I do not expect that you
 - know all the different models of the light equation
 - can rederive the refraction formulas

Next time

- Textures !



Appendix: Variation of $\cos^\gamma \theta$



```
import numpy as np
import matplotlib.pyplot as plt

def cos_gamma(t, gamma):
    return np.maximum(np.cos(t), 0)** gamma

t = np.linspace(-np.pi, np.pi, 500)
gamma_values = [1, 2, 3, 5, 20]

plt.figure(figsize=(10, 6))

for gamma in gamma_values:
    y = cos_gamma(t, gamma)
    plt.plot(t, y, label=f"$\\gamma = {gamma}$")

plt.title("Plot of $\\max(\\cos(\\theta), 0)^\\gamma$ " \
          "for various $\\gamma$", fontsize=16)
plt.xlabel("$\\theta$", fontsize=14)
plt.ylabel("$\\max(\\cos(\\theta), 0)^\\gamma$", fontsize=14)
plt.xticks(
    ticks=[-np.pi, -np.pi/2, 0, np.pi/2, np.pi],
    labels=[r"$-\pi$", r"$-\pi/2$",
            "$0$", r"$\pi/2$", r"$\pi$"]
)
plt.legend(fontsize=12)
plt.grid()
plt.show()
```

Appendix: The Rendering Equation – Enough ?

Although the equation is very general, it does not capture every aspect of light reflection. Some missing aspects include the following:

- **Transmission**, which occurs when light is transmitted through the surface, such as when it hits a **glass** object or a **water** surface,
- **Subsurface scattering**, where the spatial locations for incoming and departing light are different. Surfaces rendered without accounting for subsurface scattering may appear unnaturally opaque — however, it is not necessary to account for this if transmission is included in the equation, since that will effectively include also light scattered under the surface,
- **Polarization**, where different light polarizations will sometimes have different reflection distributions, for example when light bounces at a water surface,
- **Phosphorescence**, which occurs when light or other **electromagnetic radiation** is **absorbed** at one moment in time and emitted at a later moment in time, usually with a longer **wavelength** (unless the absorbed electromagnetic radiation is very intense),
- **Interference**, where the wave properties of light are exhibited,
- **Fluorescence**, where the absorbed and emitted light have different **wavelengths**,
- **Non-linear** effects, where very intense light can increase the **energy level** of an **electron** with more energy than that of a single **photon** (this can occur if the electron is hit by two photons at the same time), and **emission** of light with higher frequency than the frequency of the light that hit the surface suddenly becomes possible, and
- **Relativistic Doppler effect**, where light that bounces on an object that is moving in a very high speed will get its wavelength changed; if the light bounces at an object that is moving towards it, the impact will compress the **photons**, so the wavelength will become shorter and the light will be **blueshifted** and the photons will be packed more closely so the photon flux will be increased; if it bounces at an object that is moving away from it, it will be **redshifted** and the photons will be packed more sparsely so the photon flux will be decreased.

For scenes that are either not composed of simple surfaces in a vacuum or for which the travel time for light is an important factor, researchers have generalized the rendering equation to produce a *volume rendering equation*^[5] suitable for **volume rendering** and a *transient rendering equation*^[6] for use with data from a **time-of-flight camera**.

https://en.wikipedia.org/wiki/Rendering_equation

Appendix: The Rendering Equation – Other Models

- The Blinn-Phong model we saw is just **one** visually pleasing **model**!
- There is always a tradeoff between visual accuracy and difficulty to compute

- Lambertian:

$$f(\omega_i, x, \omega_o) = \frac{a}{\pi}$$

- Lafortune:

$$f(\omega_i, x, \omega_o) = \frac{a}{\pi} + b(-\omega_i^T A \omega_o)^k$$

- Blinn:

$$f(\omega_i, x, \omega_o) = \frac{a}{\pi} + b \cos^c b \langle \omega_i | \omega_o \rangle$$

- Phong:

$$f(\omega_i, x, \omega_o) = \frac{a}{\pi} +$$

$$b \cos^c (2 \langle \omega_i | n \rangle \langle \omega_o | n \rangle - \langle \omega_i | \omega_o \rangle)$$

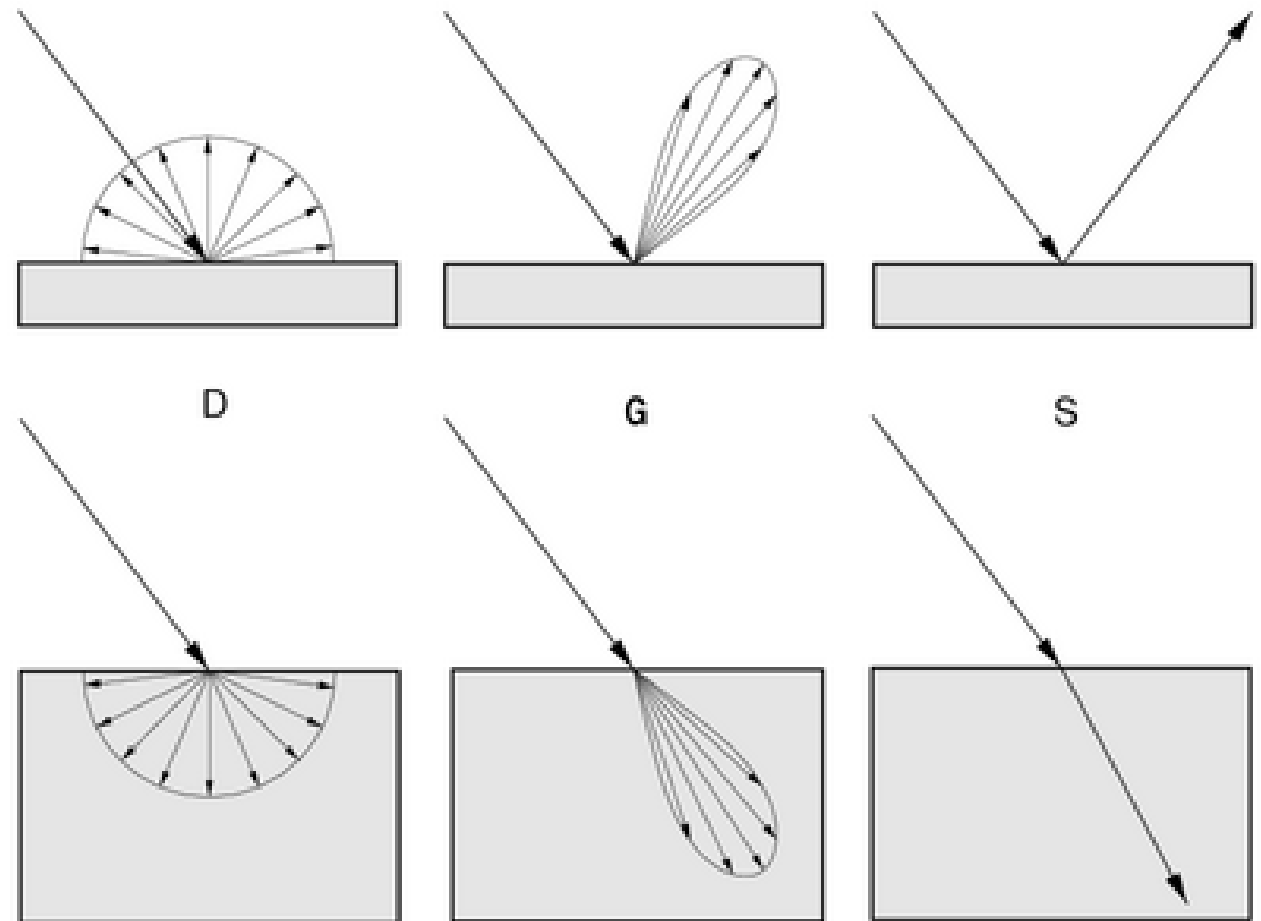
- Ward:

$$f(\omega_i, x, \omega_o) = \frac{a}{\pi} +$$

$$\frac{b}{4\pi c^2 \sqrt{\langle \omega_i | n \rangle \langle \omega_o | n \rangle}} \exp\left(-\frac{\tan^2 b \langle \omega_i | \omega_o \rangle}{c^2}\right)$$

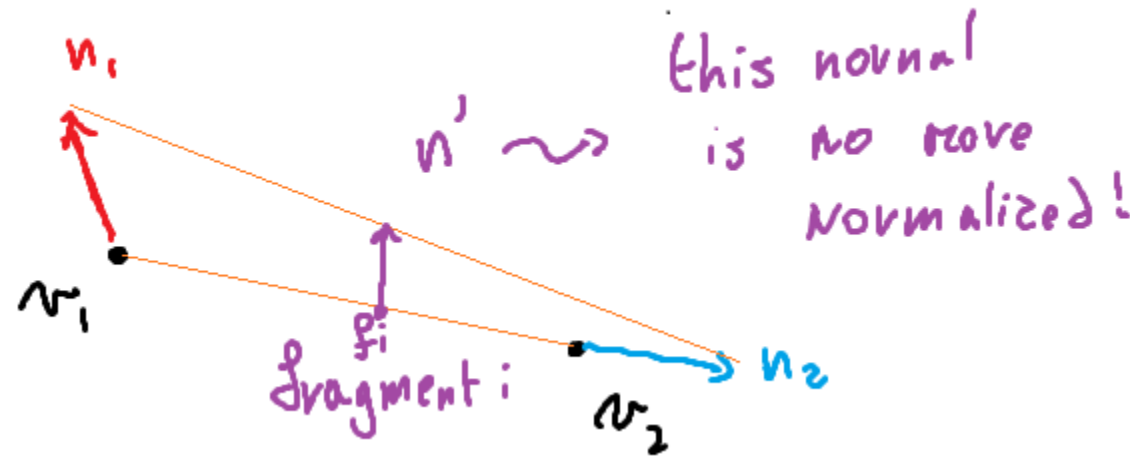
Appendix: BRDF Not Alone

- BRDF is for Reflectance
- BTDF is for Transmittance (light inside the material!)
- BSDF is both of them



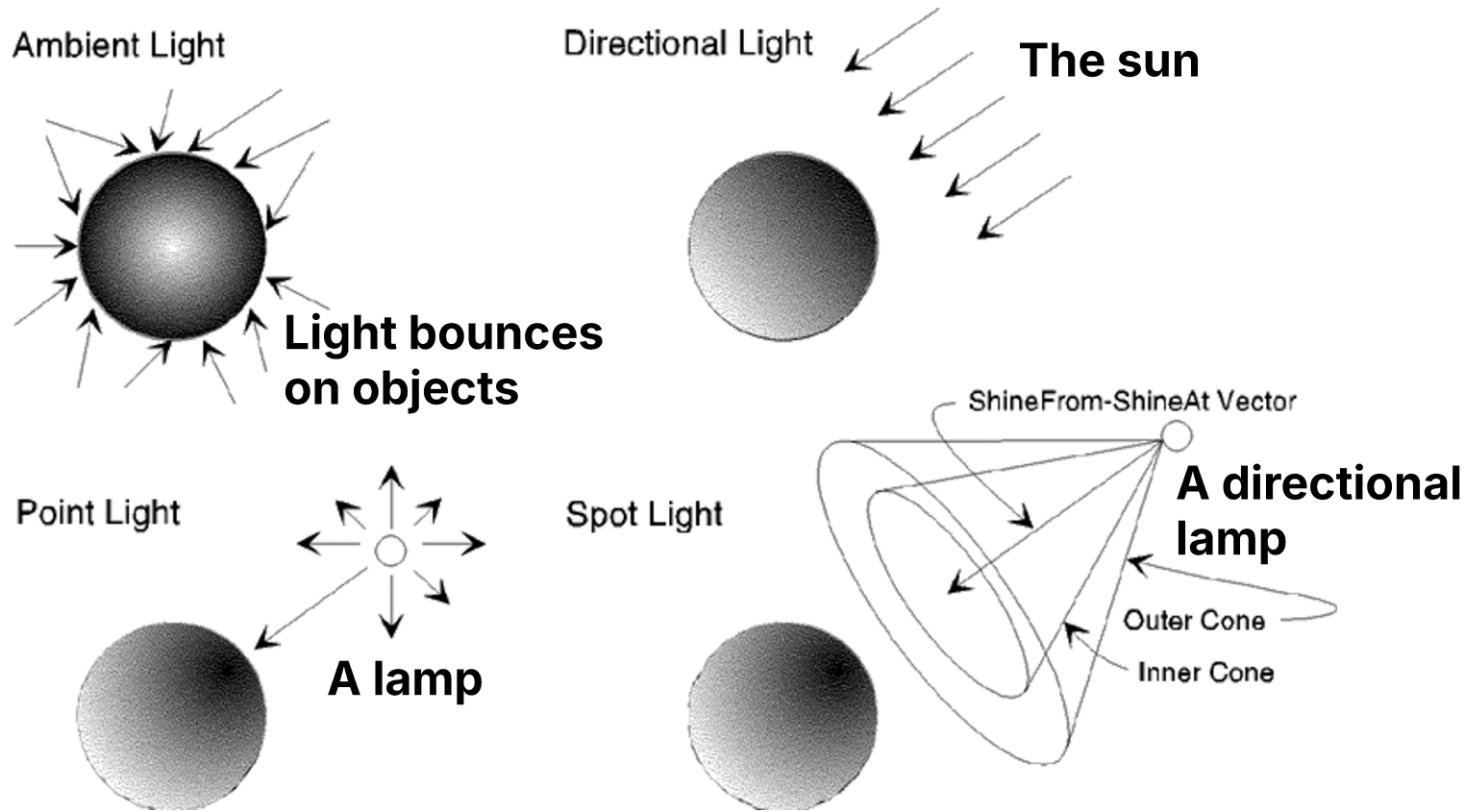
Appendix: Phong and the non-normalized normals

- When you interpolate a normal between the vertex and the fragment shader
 - The "normality" condition is not necessarily fulfilled
 - You need to re-normalize your normal when you receive them in the fragment shader!
 - $n = \text{normalize}(n)$



Appendix: Kind of light sources

- We did not discuss that we can implement several types of light sources
- For the moment they were all considered infinitesimal points but..



3D Lighting: History, Concepts, and Techniques
Arnold Gallardo
2000

Digital Lighting and Rendering
Jeremy Birn
2013

<https://stackoverflow.com/questions/16406861/difference-between-positioned-and-direct-light-in-c-opengl>

Appendix: Cosine between normalized vectors

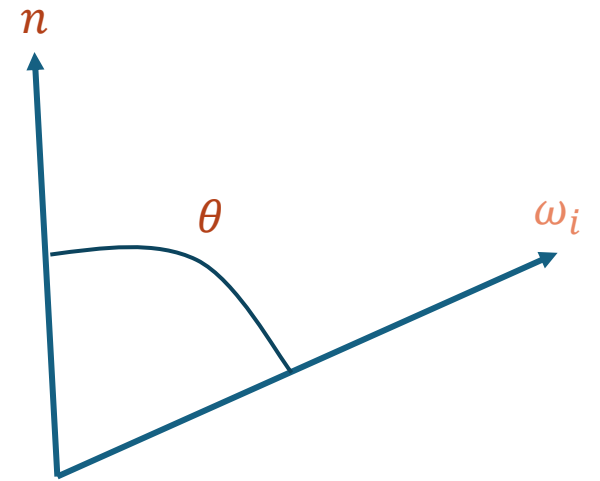
- Why did we had that $\omega_i \cdot \vec{n} = \cos \theta$?

- ω_i and $\vec{n}$ are normalized ! $|\omega_i| = 1$, $|\vec{n}| = 1$

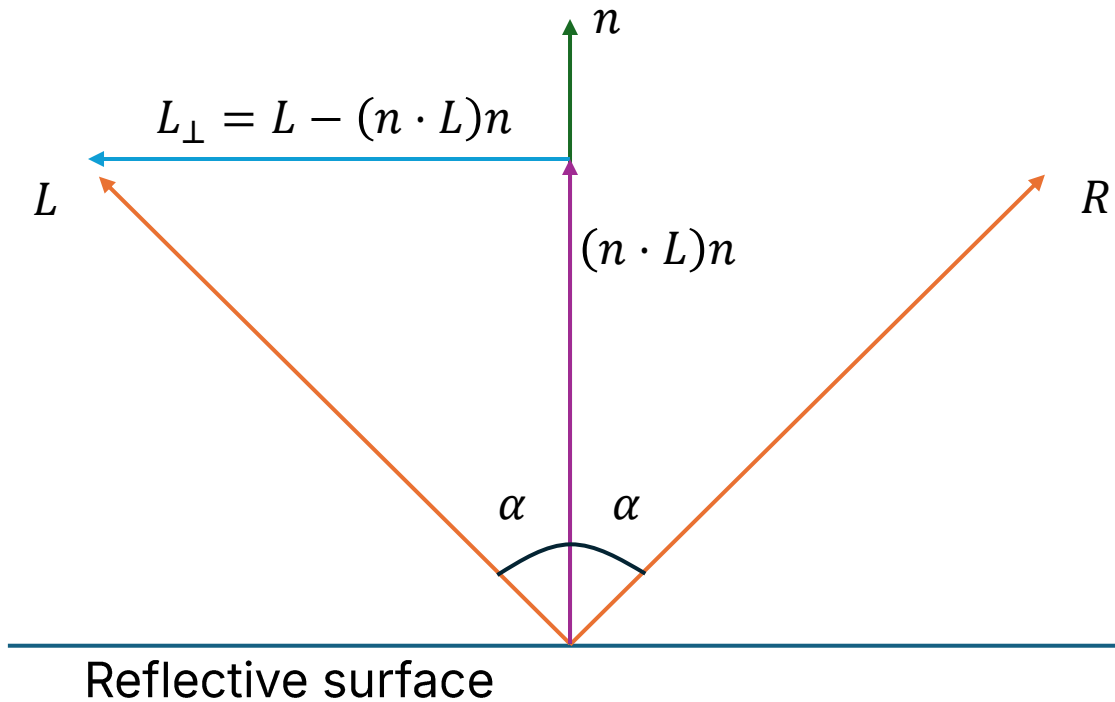
$$\omega_i \cdot \vec{n} = |\omega_i| |\vec{n}| \cos \theta = 1 \cdot 1 \cdot \cos \theta$$

- So,

$$\omega_i \cdot \vec{n} = \cos \theta$$

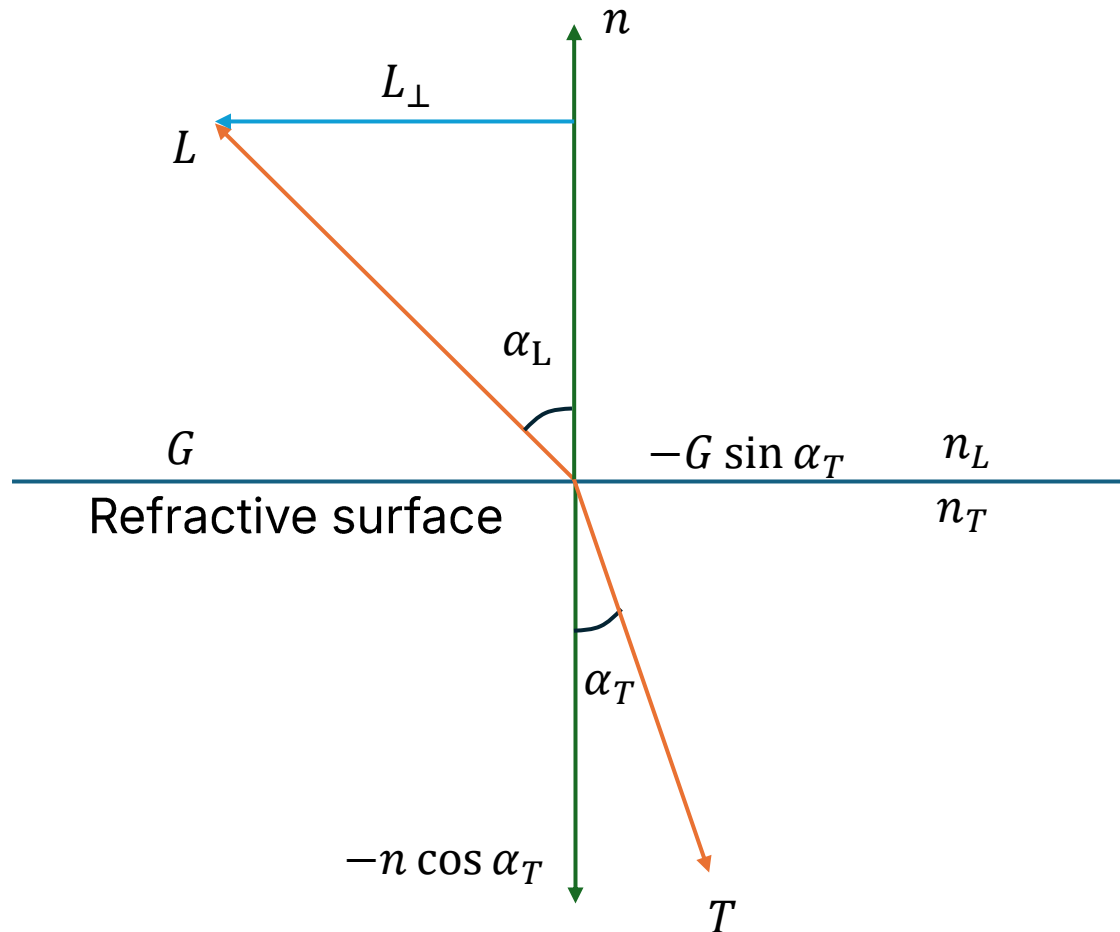


Appendix: Reflection ray proof



$$\begin{aligned} R &= L - 2L_{\perp} \\ &= L - 2(L - (n \cdot L)n) \\ &= 2(n \cdot L)n - L \end{aligned}$$

Appendix: Refraction ray proof



$$G = \frac{L_{\perp}}{\sin \alpha_L} = \frac{L - (n \cdot L)n}{\sin \alpha_L} \quad \eta_L \sin \alpha_L = \eta_T \sin \alpha_T$$

$$\begin{aligned} T &= -n \cos \alpha_T - G \sin \alpha_T \\ &= -n \cos \alpha_T - \frac{\sin \alpha_T}{\sin \alpha_L} (L - (n \cdot L)n) \\ &= -n \cos \alpha_T - \frac{\eta_L}{\eta_T} (L - (n \cdot L)n) \\ &= -n \sqrt{1 - \frac{\eta_L^2}{\eta_T^2} \sin^2 \alpha_L} - \frac{\eta_L}{\eta_T} (L - (n \cdot L)n) \\ &= \left(\frac{\eta_L}{\eta_T} n \cdot L - \sqrt{1 - \frac{\eta_L^2}{\eta_T^2} (1 - (n \cdot L)^2)} \right) n - \frac{\eta_L}{\eta_T} L \end{aligned}$$

Appendix: Raytracing – On the GPU

- No recursion in the GPU!
- → derecursify the algorithm recursive_function → while + data structure

```
# For each fragment
def raytrace():
    to_explore = [ray0]
    i = 0
    color = scene.ambient_color
    while len(to_explore) > 0 and i < maxIter:
        ray = to_explore[0]
        del to_explore[0]    # Remove it
        intersection, q = get_intersect(ray, mesh)
        if not intersection:
            color += scene.background_color
            pass

        color += compute_color(lights, q)

        # Add reflection and refraction to the list
        to_explore.push(reflect(ray, mesh[q].normal))
        to_explore.push(refract(ray, mesh[q].normal, refr_inde))
    return color
```

Appendix: GPU – NVIDIA RTX Cards

- Recently NVIDIA added support for raytracing computing cores in its graphics cards
- They let you:
 - launch rays in shaders
 - Compute intersections fast
- New games are becoming raytraced in real time!
- Not yet available in OpenGL :'(ul>- Vulkan and DirectX only (2019.08)
- <https://community.khronos.org/t/opengl-will-it-ever-have-rtx-raytracing-support/104489/5>



Show Cases:

- <https://www.youtube.com/watch?v=quRO1QEZ1Rc>
- <https://www.youtube.com/watch?v=pNmhJx8yPLk>
- <https://www.youtube.com/watch?v=Efo-YDWnnpw>
- <https://www.youtube.com/watch?v=bFUWu387ErM>
- <https://www.youtube.com/watch?v=hTf1otkzTz4>
- <https://www.youtube.com/watch?v=ttghcA4kUQU>
- https://www.youtube.com/watch?v=NgcYLIvlp_k